

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique

Université de Monastir
Faculté des Sciences de Monastir

Projet de Fin d'Études

Pour l'obtention du diplôme de Licence / Master en Informatique

LexAI

Plateforme IA d'Analyse Automatisée
de la Conformité des Contrats Juridiques



Réalisé par :	Ala Missaoui
Entreprise d'accueil :	e-Tafakna
Encadrant académique :	Badreddine Guizani
Encadrante professionnelle :	Dhekra Ben Mahmoud

Table des matières

Liste des Abréviations	7
Introduction Générale	1
Chapitre 1 : Contexte général et étude préalable	4
1.1 Introduction	4
1.2 Cadre général	4
1.3 Présentation de l'organisme d'accueil	4
1.4 Présentation du projet	5
1.4.1 Problématique	5
1.4.2 Étude de l'existant	6
1.4.3 Critique des solutions existantes	7
1.4.4 Solution proposée	7
1.5 Méthodologie de gestion de projet	9
1.5.1 Framework Scrum	9
1.5.2 Les rôles de Scrum	10
1.5.3 Les artefacts de Scrum	11
1.5.4 Les événements de Scrum	11
1.5.5 Planning du projet	11
1.6 Conclusion	12
Chapitre 2 : Analyse et spécification des besoins	15
2.1 Introduction	15
2.2 Analyse et identification des besoins	15
2.2.1 Identification des acteurs	15
2.2.2 Capture des besoins fonctionnels	16
2.2.3 Capture des besoins non fonctionnels	17
2.2.4 Planification du travail avec Scrum	18
2.3 Composition du backlog produit	18
2.3.1 Backlog de produit	18
2.3.2 Planification des releases et sprints	18
2.4 Architecture de l'application	20
2.4.1 Architecture logique — Pipeline multi-agents	20
2.4.2 Architecture physique	21
2.5 Environnement logiciel du projet	22
2.5.1 Outils de gestion de projet	22
2.5.2 Technologies de développement	23
2.6 Diagramme de cas d'utilisation global	24
2.7 Contraintes et exigences de qualité	26
2.7.1 Contraintes techniques	26
2.7.2 Exigences de sécurité	26
2.7.3 Exigences de performance	27
2.8 Infrastructure et déploiement	27

2.8.1	Conteneurisation avec Docker Compose	27
2.8.2	Volumes et persistance	28
2.8.3	Variables d'environnement	28
2.8.4	Lancement de la plateforme	29
2.9	Conclusion	30
Chapitre 3 : Release 1 : Extraction documentaire et Analyse NLP		32
3.1	Introduction	32
3.2	Planification des sprints de la Release 1	32
3.3	Sprint 1 : Agent 1 — Pipeline d'extraction documentaire	32
3.3.1	Backlog du Sprint 1	32
3.3.2	Analyse des besoins du Sprint 1	34
3.3.3	Conception du Sprint 1	36
3.3.4	Réalisation du Sprint 1	39
3.4	Sprint 2 : Agent 2 — Analyse NLP des clauses	42
3.4.1	Backlog du Sprint 2	42
3.4.2	Analyse des besoins du Sprint 2	44
3.4.3	Conception du Sprint 2	47
3.4.4	Construction du dataset et entraînement du modèle	49
3.4.5	Réalisation du Sprint 2	55
3.5	Gestion des migrations de base de données	56
3.5.1	Outil : Alembic	56
3.5.2	Historique des migrations de la Release 1	56
3.5.3	Exécution automatique au démarrage	56
3.6	Diagrammes de classes de la Release 1	57
3.6.1	Diagramme de classes — Agent 1 : Extraction documentaire	57
3.6.2	Diagramme de classes — Agent 2 : Analyse NLP	57
3.7	Récapitulatif des endpoints REST de la Release 1	58
3.8	Tests et validation de la Release 1	59
3.8.1	Stratégie de tests	59
3.8.2	Résultats de validation	61
3.9	Conclusion	61
Chapitre 4 : Release 2 : Évaluation de conformité et Recommandations		64
4.1	Introduction	64
4.2	Planification des sprints de la Release 2	64
4.3	Sprint 3 : Agent 3 — Moteur d'évaluation de conformité	66
4.3.1	Backlog du Sprint 3	66
4.3.2	Analyse des besoins du Sprint 3	67
4.3.3	Conception du Sprint 3	70
4.3.4	Réalisation du Sprint 3	72
4.4	Sprint 4 : Agent 4 — Génération de recommandations	73
4.4.1	Backlog du Sprint 4	73
4.4.2	Analyse des besoins du Sprint 4	73
4.4.3	Conception du Sprint 4	78
4.4.4	Réalisation du Sprint 4	80
4.5	Diagrammes de classes de la Release 2	81
4.5.1	Diagramme de classes — Agent 3 : Évaluation de conformité	81
4.5.2	Diagramme de classes — Agent 4 : Génération de recommandations	82

4.6	Intégration LLM — Enrichissement par l'API Groq	83
4.6.1	Architecture de l'enrichissement LLM	83
4.6.2	Prompt template pour la génération LLM	84
4.6.3	Configuration	84
4.7	Gestion des migrations de base de données — Release 2	85
4.8	Récapitulatif des endpoints REST de la Release 2	86
4.9	Tests et validation de la Release 2	86
4.9.1	Stratégie de tests	86
4.9.2	Résultats de validation	87
4.10	Conclusion	88
Chapitre 5 : Release 3 : Cloud, Conteneurisation & DevOps		90
5.1	Introduction	90
5.2	Sprint 6 : Déploiement sur VPS	90
5.2.1	Objectif du Sprint	90
5.2.2	Présentation du VPS	91
5.2.3	Provisionnement du VPS	92
5.2.4	Configuration Réseau et Sécurité	93
5.2.5	Règles de Sécurité	94
5.2.6	Déploiement des Services Applicatifs	95
5.2.7	Clonage du Projet depuis GitHub	96
5.2.8	Déploiement du Backend	97
5.2.9	Validation de l'Infrastructure	98
5.2.10	Gestion du Code Source avec Git	99
5.3	Sprint 7 : Conteneurisation avec Docker Compose	100
5.3.1	Objectif du Sprint	100
5.3.2	Docker et Conteneurisation	100
5.3.3	Dockerfile des Services	101
5.3.4	Docker Compose	102
5.4	Sprint 8 : Pipeline CI/CD avec GitHub Actions	102
5.4.1	Objectif du Sprint	102
5.4.2	Architecture du Pipeline CI/CD	103
5.4.3	Structure du Workflow GitHub Actions	103
5.4.4	Images Docker — Backend et Frontend	104
5.4.5	Orchestration avec Docker Compose	105
5.4.6	Déploiement Automatique sur VPS	106
5.4.7	Registre d'Images — GitHub Container Registry (GHCR)	107
5.4.8	Bilan DevOps	108
5.4.9	Conclusion	109
Conclusion Générale		110

Liste des tableaux

1.1	Critique des solutions existantes par rapport à LexAI — focus droit tunisien	8
1.2	Planning général du projet LexAI — 3 releases, 7 sprints	12
2.1	Acteurs de la plateforme LexAI	15
2.2	Backlog produit des fonctionnalités principales de LexAI	19
2.3	Stack technologique de LexAI — Backend et extraction (1/2)	23
2.4	Stack technologique de LexAI — NLP, IA, Frontend et DevOps (2/2)	24
2.5	Exigences de performance de la plateforme LexAI	27
2.6	Services Docker Compose de la plateforme LexAI	28
2.7	Variables d’environnement de LexAI	29
3.1	Backlog du Sprint 1 — Agent 1	34
3.2	Description textuelle du cas « Téléverser et extraire un contrat »	35
3.3	Dictionnaire des données du Sprint 1	37
3.4	Backlog du Sprint 2 — Agent 2	43
3.5	Description textuelle du cas « Analyser les clauses NLP »	46
3.6	Dictionnaire des données du Sprint 2	48
3.7	Sources du dataset d’entraînement — Agent 2	50
3.8	Paramètres d’entraînement du modèle XLM-RoBERTa	52
3.9	Migrations Alembic de la Release 1	56
3.10	Endpoints REST de la Release 1 (Agents 1 et 2)	59
3.11	Résultats de validation de la Release 1	61
4.1	Backlog du Sprint 3 — Agent 3	66
4.2	Description textuelle du cas « Évaluer la conformité réglementaire »	69
4.3	Structure d’une règle juridique JSON	71
4.4	Dictionnaire des données du Sprint 3	72
4.5	Backlog du Sprint 4 — Agent 4	74
4.6	Description textuelle du cas « Générer des recommandations »	76
4.7	Structure d’un template de recommandation	78
4.8	Dictionnaire des données du Sprint 4	79
4.9	Migrations Alembic de la Release 2	85
4.10	Endpoints REST de la Release 2 (Agents 3 et 4)	86
4.11	Résultats de validation de la Release 2	87
5.1	Bilan des pratiques DevOps adoptées pour LexAI	108
5.2	Bilan quantitatif des réalisations de LexAI	112

Table des figures

1.1	Logo de l'entreprise e-Tafakna	5
1.2	Cycle de fonctionnement du framework Scrum	10
2.1	Architecture physique à trois niveaux de LexAI	22
2.2	Diagramme de cas d'utilisation global de la plateforme LexAI	25
3.1	Diagramme de cas d'utilisation du Sprint 1	35
3.2	Diagramme de séquence : Téléverser et extraire un contrat	36
3.3	Interface de téléversement de contrat	40
3.4	Interface de suivi de progression du traitement	41
3.5	Interface de visualisation des résultats d'extraction	42
3.6	Diagramme de cas d'utilisation du Sprint 2	45
3.7	Diagramme de séquence : Analyser les clauses NLP	47
3.8	Pipeline de construction du dataset d'entraînement	51
3.9	Stratégie de classification multi-niveaux du ClauseClassifier	54
3.10	Interface de visualisation de l'analyse NLP	55
3.11	Diagramme de classes — Agent 1 : Extraction documentaire	57
3.12	Diagramme de classes — Agent 2 : Analyse NLP	58
4.1	Diagramme de cas d'utilisation du Sprint 3	68
4.2	Diagramme de séquence : Évaluer la conformité réglementaire	70
4.3	Interface de visualisation de l'évaluation de conformité	73
4.4	Diagramme de cas d'utilisation du Sprint 4	75
4.5	Diagramme de séquence : Générer des recommandations	77
4.6	Interface de visualisation des recommandations	80
4.7	Diagramme de classes — Agent 3 : Évaluation de conformité	82
4.8	Diagramme de classes — Agent 4 : Génération de recommandations	83
5.1	Vue d'ensemble de l'infrastructure VPS de production	92
5.2	Provisionnement de l'infrastructure VPS	93
5.3	Configuration réseau du VPS	94
5.4	Règles de sécurité du VPS	95
5.5	Déploiement des services applicatifs sur le VPS	96
5.6	Clonage du projet LexAI depuis GitHub sur le VPS	97
5.7	Déploiement du backend LexAI sur le VPS	98
5.8	Validation de la connectivité réseau et de l'accès à LexAI	99
5.9	Gestion du code source LexAI avec Git et GitHub	100
5.10	Exemple de Dockerfile	101
5.11	Configuration Docker Compose	102
5.12	Dockerfile du service Frontend (multi-stage build)	105
5.13	Extrait du fichier docker-compose.yml — services backend, redis et postgres	106

Liste des Abréviations

Abréviation	Signification
LexAI	Plateforme IA d'Analyse Juridique de Contrats
IA	Intelligence Artificielle
LNPDP	Loi Organique n°2004-63 portant sur la Protection des Données à Caractère Personnel (Tunisie)
RGPD	Règlement Général sur la Protection des Données (Union Européenne, 2016/679)
GDPR	General Data Protection Regulation (version anglaise du RGPD)
COC	Code des Obligations et Contrats tunisien
ISO	International Organization for Standardization
INPDP	Instance Nationale de Protection des Données Personnelles (Tunisie)
NLP	Natural Language Processing (Traitement du Langage Naturel)
NER	Named Entity Recognition (Reconnaissance d'Entités Nommées)
LLM	Large Language Model (Grand Modèle de Langage)
API	Application Programming Interface (Interface de Programmation d'Application)
REST	Representational State Transfer
ORM	Object-Relational Mapping (Correspondance Objet-Relationnel)
JSON	JavaScript Object Notation
DPA	Data Processing Agreement (Accord de Traitement de Données)
DPO	Délégué à la Protection des Données (Data Protection Officer)
PME	Petites et Moyennes Entreprises
CLM	Contract Lifecycle Management (Gestion du Cycle de Vie des Contrats)
RAG	Retrieval Augmented Generation
RBAC	Role-Based Access Control (Contrôle d'Accès Basé sur les Rôles)
PDF	Portable Document Format
DOCX	Document XML Open Format (Microsoft Word)

Abréviation	Signification
HTML	HyperText Markup Language
DB	Database (Base de Données)
MVC	Modèle-Vue-Contrôleur
CI/CD	Continuous Integration / Continuous Deployment
VPS	Virtual Private Server (Serveur Privé Virtuel)
GHCR	GitHub Container Registry
Scrum	Méthodologie agile de gestion de projet (cadre de travail itératif)
CUAD	Contract Understanding Atticus Dataset
FAISS	Facebook AI Similarity Search (bibliothèque de recherche vectorielle)

Introduction Générale

À l'ère de la transformation numérique, le domaine juridique connaît une mutation profonde portée par l'essor de l'intelligence artificielle. Les professionnels du droit, les responsables de conformité et les PME tunisiennes consacrent encore aujourd'hui une part considérable de leur temps à l'examen manuel des contrats, tâche longue, coûteuse et sujette aux erreurs humaines. La complexité croissante des obligations légales — notamment la Loi Organique n°2004-63 sur la Protection des Données à Caractère Personnel (LNPDP), le Règlement Général sur la Protection des Données (RGPD), le Code des Obligations et Contrats tunisien (COC) ainsi que les normes ISO 27001 et ISO 9001 — aggrave encore ce problème en multipliant les cadres réglementaires à surveiller simultanément.

C'est dans ce contexte que s'inscrit le projet **LexAI — Plateforme IA d'Analyse Juridique de Contrats**, réalisé par **Ala Missaoui** dans le cadre d'un projet de fin d'études à la Faculté des Sciences de Monastir, en collaboration avec l'entreprise **ETAFAKNA**. LexAI est une plateforme intelligente qui automatise l'analyse de conformité des contrats juridiques en s'appuyant sur un pipeline de quatre agents spécialisés orchestrés par LangGraph. Elle permet à ses utilisateurs de téléverser un contrat, d'en extraire et normaliser le texte, d'analyser automatiquement ses clauses par traitement du langage naturel, d'évaluer sa conformité aux référentiels légaux applicables et de générer des recommandations de mise en conformité sous forme de clauses révisées.

La plateforme répond à un besoin réel et urgent dans le contexte tunisien : les entreprises et cabinets juridiques ne disposent pas d'outils adaptés à la législation nationale. Les solutions internationales existantes ne couvrent ni le droit tunisien ni la langue française dans une dimension contractuelle juridique. LexAI comble ce vide en intégrant les articles clés de la LNPDP, du RGPD, de l'ISO 27001, de l'ISO 9001 et du COC directement dans son moteur de règles.

Ce rapport est structuré en cinq chapitres. Le **premier chapitre** présente le contexte général du projet, l'analyse de l'existant, la problématique et la méthodologie Scrum adoptée. Le **deuxième chapitre** détaille l'analyse des besoins, les acteurs, l'architecture globale et l'environnement technologique. Le **troisième chapitre** décrit la première release du projet, couvrant l'Agent 1 (extraction documentaire) et l'Agent 2 (analyse NLP). Le **quatrième chapitre** présente la deuxième release avec l'Agent 3 (évaluation de conformité) et l'Agent 4 (génération de recommandations). Le **cinquième chapitre** décrit la troisième release dédiée au déploiement Cloud, à la conteneurisation Docker et à la mise en place du pipeline CI/CD avec GitHub Actions. Une **conclusion générale** synthétise les résultats obtenus et trace les perspectives d'évolution future de la plateforme LexAI.

CHAPITRE 1

Contexte général et étude préalable

1.1 Introduction

Dans ce chapitre, nous présentons le contexte général de notre projet. Nous commençons par une présentation de l'organisme d'accueil, avant de détailler le cadre du projet, ses objectifs, sa problématique et l'impact attendu. Nous poursuivons avec une étude de l'existant à travers l'analyse de solutions similaires, afin d'identifier les lacunes auxquelles LexAI répond. Enfin, nous exposons la méthodologie de développement Scrum choisie pour conduire efficacement ce projet.

1.2 Cadre général

Ce projet a été réalisé par **Ala Missaoui**, étudiant à la **Faculté des Sciences de Monastir**, dans le cadre d'un projet de fin d'études effectué au sein de l'entreprise **ETAFAKNA**. Cette expérience a permis de mobiliser et d'approfondir des connaissances en génie logiciel, traitement du langage naturel, architecture logicielle et droit informatique dans le cadre d'un projet répondant à des besoins professionnels concrets.

1.3 Présentation de l'organisme d'accueil

ETAFAKNA est une startup tunisienne spécialisée dans les solutions technologiques pour les professionnels du droit et de la conformité réglementaire. Son nom, inspiré du terme arabe signifiant « notre accord », reflète sa vocation première : faciliter et sécuriser les processus contractuels entre les entreprises. Fondée à Monastir, ETAFAKNA accompagne les cabinets juridiques, les PME et les responsables conformité dans la digitalisation et l'automatisation de la gestion de leurs contrats.

L'entreprise s'inscrit dans l'écosystème LegalTech tunisien en plein essor, avec pour mission de combler le fossé entre les exigences légales croissantes — notamment la LNPDP, le RGPD et les normes ISO — et les outils mis à disposition des praticiens du droit. ETAFAKNA se distingue par son ancrage dans le contexte juridique tunisien et sa maîtrise des spécificités de la langue française dans les documents contractuels.

Dans le cadre de ce projet de fin d'études, ETAFAKNA a fourni le cadre applicatif,

les exigences métier et l'expertise juridique nécessaires à la conception de la plateforme LexAI.



FIGURE 1.1 – Logo de l'entreprise e-Tafakna

1.4 Présentation du projet

1.4.1 Problématique

En Tunisie, les PME, les cabinets juridiques et les responsables de conformité examinent encore manuellement leurs contrats pour s'assurer de leur conformité aux réglementations en vigueur. Cette activité représente un volume de travail considérable : une revue complète d'un contrat de prestation de services peut mobiliser plusieurs heures d'un juriste qualifié, multipliant les coûts opérationnels et les risques d'erreur.

La multiplication des référentiels réglementaires aggrave cette situation. Les contrats doivent désormais être analysés au regard de la **LNPDP** (Loi Organique n°2004-63 portant sur la protection des données à caractère personnel), du **RGPD** pour les contrats impliquant des parties européennes, du **Code des Obligations et Contrats (COC)** tunisien, ainsi que des normes **ISO 27001** et **ISO 9001**. La vérification manuelle de l'ensemble de ces exigences est non seulement lente et coûteuse, mais aussi intrinsèquement limitée par les capacités humaines.

De plus, aucune solution existante sur le marché ne couvre le contexte juridique tunisien en langue française, laissant les professionnels du droit sans outil adapté à leurs besoins spécifiques.

La problématique principale de ce projet est donc la suivante :

Comment concevoir une plateforme intelligente capable d'automatiser l'analyse de conformité des contrats juridiques en langue française, en intégrant les spécificités du droit tunisien (LNPDP, COC) et des normes internationales

(RGPD, ISO 27001, ISO 9001) ?

1.4.2 Étude de l'existant

Afin de mieux situer notre projet, nous avons analysé plusieurs solutions existantes intervenant dans des domaines proches : la gestion documentaire juridique, l'analyse automatisée de contrats et la vérification de conformité réglementaire.

1. iManage Work

Domaine : Gestion documentaire pour cabinets juridiques et services juridiques d'entreprise.

Objectif : Centraliser et organiser les documents juridiques, faciliter la recherche et la collaboration.

Principales fonctionnalités :

- Stockage sécurisé et indexation des contrats et documents juridiques.
- Recherche full-text et gestion des versions.
- Intégration avec les outils bureautiques (Microsoft Office, Outlook).

2. Kira Systems

Domaine : Extraction automatisée d'informations contractuelles par intelligence artificielle.

Objectif : Identifier et extraire automatiquement les clauses clés d'un contrat pour accélérer la due diligence.

Principales fonctionnalités :

- Reconnaissance automatique de clauses (indemnités, résiliation, confidentialité, etc.).
- Modèles d'apprentissage automatique entraînés sur des contrats anglophones.
- Exportation structurée des données extraites.

3. ContractPodAi

Domaine : Gestion du cycle de vie des contrats (CLM — Contract Lifecycle Management).

Objectif : Couvrir toutes les phases d'un contrat, de la rédaction initiale à la renégociation.

Principales fonctionnalités :

- Rédaction assistée et bibliothèque de clauses approuvées.
- Alertes sur les dates d'échéance et obligations contractuelles.
- Tableau de bord de supervision des contrats actifs.

4. LawGeex

Domaine : Analyse automatisée de contrats par intelligence artificielle.

Objectif : Permettre aux équipes juridiques d'entreprise de comparer et valider des contrats par rapport à des politiques internes prédéfinies, en identifiant automatiquement les clauses acceptables et non acceptables.

Principales fonctionnalités :

- Comparaison automatique de clauses contractuelles par rapport à des modèles approuvés.
- Mise en évidence des clauses à risque avec des recommandations de redline.
- Intégration aux systèmes de gestion de documents des grandes entreprises.
- Reporting sur les taux d'approbation et les temps de révision.

1.4.3 Critique des solutions existantes

Les solutions existantes présentées ci-dessus couvrent partiellement les besoins en analyse contractuelle, mais aucune n'est adaptée au contexte juridique tunisien. LexAI comble ce vide en intégrant nativement la **LNPDP**, le **COC** et le **RGPD**, avec un support complet du français juridique.

1.4.4 Solution proposée

Pour répondre à cette problématique, nous proposons **LexAI**, une plateforme d'analyse juridique basée sur l'intelligence artificielle, conçue spécifiquement pour le contexte tunisien. La plateforme repose sur un **pipeline multi-agents** orchestré par LangGraph, où chaque

Solution	Points forts	Limites	Apport de LexAI (focus tunisien)
iManage Work	Gestion centralisée des documents ; sécurité éprouvée ; collaboration en temps réel.	Aucune analyse NLP ; pas de détection de violations ; pas de recommandations.	Pipeline complet extraction → scoring → recommandations ; adapté au droit tunisien (LNPDP, COC).
Kira Systems	Extraction de clauses performante ; interface intuitive ; modèles ML entraînés.	Anglophone uniquement ; aucune couverture du droit tunisien ; pas d'évaluation de conformité réglementaire.	Support du français juridique ; règles LNPDP, RGPD, ISO 27001 intégrées ; scoring multi-référentiel.
ContractPodAi	ICLM complet ; alertes contractuelles ; tableau de bord actif.	Pas d'analyse de conformité légale ; pas de contexte juridique tunisien ; coût prohibitif pour les PME.	Solution open source ; moteur de règles juridiques local ; accessible aux PME tunisiennes.
LawGeex	IA pour la révision contractuelle ; politique interne configurable ; gain de temps avéré.	Anglophone uniquement ; pas de couverture LNPDP ; réservé aux grandes entreprises.	Support du français et de la LNPDP ; recommandations avec justification légale ; accessible aux PME.

TABLE 1.1 – Critique des solutions existantes par rapport à LexAI — focus droit tunisien

agent remplit une mission précise et passe ses résultats à l’agent suivant de manière automatique et traçable.

LexAI permet de téléverser un contrat dans n’importe quel format courant (PDF, DOCX, TXT, HTML), d’en extraire et normaliser le texte, de segmenter automatiquement ses clauses et d’en extraire les entités juridiques pertinentes (parties, rôles, durées, références légales). La plateforme évalue ensuite la conformité du contrat vis-à-vis de quatre référentiels — LNPDP, RGPD, ISO 27001 et ISO 9001 — et produit pour chaque violation identifiée une recommandation concrète accompagnée d’une clause de remplacement rédigée en français.

Elle repose sur des technologies modernes : *FastAPI*, *Celery*, *Redis*, *PostgreSQL*, *spaCy*, *HuggingFace Transformers*, *LangGraph*, *React* et *Docker*, garantissant performance, modularité et maintenabilité.

1.5 Méthodologie de gestion de projet

Il existe naturellement plusieurs méthodologies de gestion de projet, chacune présentant ses avantages et ses limites. Dans le cadre de ce projet, nous avons opté pour le framework **Scrum** en raison de sa grande flexibilité, de son adaptabilité aux exigences évolutives et de l’importance qu’il accorde à la qualité du produit final.

1.5.1 Framework Scrum

L’approche agile, et plus particulièrement le framework **Scrum**, a été adoptée dans notre projet en raison de sa capacité à découper le travail en plusieurs itérations courtes appelées *sprints*. Cette approche favorise une gestion efficace du projet, une amélioration continue de la productivité ainsi qu’une meilleure adéquation avec les besoins fonctionnels définis. Dans notre contexte, Scrum a été adapté à nos exigences spécifiques : chaque sprint correspond au développement d’un agent du pipeline LexAI.

Scrum est un cadre de travail destiné au développement de produits logiciels complexes. Bien qu’il ait été initialement conçu pour les projets informatiques, il peut être appliqué à différents types de projets. Les projets menés selon Scrum sont structurés en cycles

de travail courts. Ces derniers permettent à l'équipe de planifier efficacement les étapes suivantes, de suivre régulièrement l'avancement du projet et d'apporter, si nécessaire, des ajustements ou des réorientations.

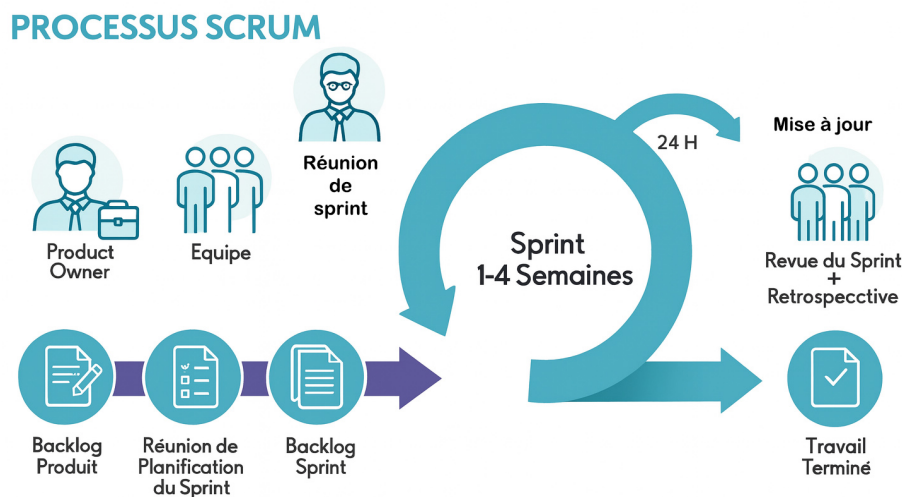


FIGURE 1.2 – Cycle de fonctionnement du framework Scrum

1.5.2 Les rôles de Scrum

L'équipe **Scrum** est constituée de trois rôles principaux : le **Scrum Master**, le **Product Owner** et l'équipe de développement.

- **Scrum Master** : Veille à la bonne compréhension et à la correcte application du framework Scrum. Il anime et accompagne l'équipe de développement, favorise la collaboration entre ses membres et s'assure du respect du processus Scrum tout au long du projet.
- **Product Owner** : Responsable de la gestion du *Product Backlog* et de la définition du produit final. Il assure la communication entre l'équipe technique et les parties prenantes afin de garantir la prise en compte des besoins exprimés. Dans notre cas, ce rôle est assuré par l'encadrant académique.
- **Équipe de développement** : Composée de **Ala Missaoui**, travaillant de manière autonome et organisée pour livrer les incréments du produit à chaque sprint,

avec des compétences couvrant le développement backend, frontend, NLP et droit informatique.

1.5.3 Les artefacts de Scrum

Les artefacts **Scrum** fournissent des informations essentielles permettant à l'équipe ainsi qu'aux parties prenantes de comprendre l'état du produit en cours de développement.

- **Backlog produit** : Liste priorisée de l'ensemble des fonctionnalités à développer, correspondant dans notre cas aux quatre agents de la plateforme LexAI et aux composants associés.
- **Backlog de sprint** : Sous-ensemble du backlog produit sélectionné pour le sprint en cours, constituant le plan de travail de l'équipe pour cette itération.
- **Burn Down Chart** : Graphique permettant de visualiser l'avancement du projet au fil du temps, en montrant la réduction progressive du travail restant.

1.5.4 Les événements de Scrum

Le framework **Scrum** repose sur quatre événements principaux se déroulant au cours de chaque sprint.

- **Planification de sprint** : Organisée avant le début du sprint, elle définit les objectifs du sprint et les éléments du backlog à réaliser durant l'itération.
- **Daily Scrum** : Réunion quotidienne courte (15 minutes) permettant à l'équipe de partager l'avancement et d'identifier les obstacles.
- **Revue de sprint** : Réunion tenue à la fin de chaque sprint pour présenter les fonctionnalités réalisées et valider les incréments du produit.
- **Rétrospective de sprint** : Réunion d'analyse du déroulement du sprint afin d'identifier les axes d'amélioration pour les itérations suivantes.

1.5.5 Planning du projet

Le tableau 1.2 présente la planification temporelle du projet LexAI, répartie sur douze semaines de développement actif organisées en trois releases et sept sprints. Chaque release

correspond à une étape fonctionnelle majeure du pipeline multi-agents.

Semaines	Release / Sprint	Objectif	Livrables
S1 – S2	Release 1 Sprint 1	Agent 1 : ingestion documentaire et extraction de texte (PDF, DOCX, TXT, HTML)	API upload, Celery, ExtractionViewer
S3 – S4	Release 1 Sprint 2	Agent 2 : segmentation des clauses, classification NLP et extraction d'entités juridiques	NLP pipeline, AnalysisViewer
S5 – S6	Release 2 Sprint 3	Agent 3 : moteur de règles juridiques (LNPD, RGP, ISO 27001) et scoring de conformité	Moteur de règles JSON, EvaluationViewer
S7 – S8	Release 2 Sprint 4	Agent 4 : génération de recommandations par templates et LLM (Llama 3 / Groq)	RecommendationViewer, export DOCX/PDF
S9 – S10	Release 3 Sprint 5–6	Déploiement Cloud IaaS (MicroStack), conteneurisation Docker et orchestration des services	docker-compose.yml, images Docker
S11 – S12	Release 3 Sprint 7	Pipeline CI/CD GitHub Actions : build automatique, publication GHCR, auto-déploiement VPS	Workflow CI/CD, script auto-deploy

TABLE 1.2 – Planning général du projet LexAI — 3 releases, 7 sprints

Chaque sprint a été conduit selon le cycle Scrum standard : planification initiale, réunions daily, revue à la livraison et rétrospective. Cette organisation a permis de maintenir un rythme de développement soutenu tout en assurant la qualité et la cohérence du pipeline multi-agents.

1.6 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet LexAI, identifié la problématique liée à la revue manuelle des contrats dans le contexte juridique tunisien, analysé les solutions existantes et mis en évidence leur incapacité à couvrir le droit tunisien (LNPD, COC) et la langue française. Nous avons justifié notre approche par la mise en

place d'un pipeline multi-agents orchestré par LangGraph, couvrant l'extraction documentaire, l'analyse NLP, l'évaluation de conformité et la génération de recommandations.

Le chapitre suivant sera consacré à l'analyse détaillée des besoins fonctionnels et non fonctionnels, à la définition des acteurs du système, à l'architecture globale de la solution et à l'environnement technologique adopté.

CHAPITRE 2

Analyse et spécification des besoins

2.1 Introduction

Ce chapitre constitue la phase de planification et d'architecture de la méthodologie Scrum. Les travaux accomplis durant cette phase permettent d'avoir une vision claire du produit à construire, de définir les rôles des utilisateurs, d'identifier les fonctionnalités principales et de produire le backlog initial. Nous détaillons également l'architecture logicielle adoptée et l'environnement technologique de la plateforme LexAI.

2.2 Analyse et identification des besoins

2.2.1 Identification des acteurs

Le tableau 2.1 suivant présente les acteurs interagissant avec la plateforme LexAI :

Acteur	Rôle
Juriste / Responsable légal	Utilisateur principal de la plateforme. Il téléverse les contrats à analyser, consulte les résultats d'extraction, visualise la segmentation en clauses et les entités juridiques identifiées, consulte l'évaluation de conformité et les recommandations générées. Il peut également déclencher manuellement le retraitement d'un document ou supprimer un document de la file.
Responsable conformité	Utilisateur spécialisé dans la vérification réglementaire. Il consulte les scores de conformité par référentiel (LNPDP, RGPD, ISO 27001, ISO 9001), examine les violations détectées et leurs niveaux de sévérité, et exploite les recommandations pour mettre en conformité les contrats. Il peut également exporter le contrat révisé au format DOCX ou PDF.
Administrateur	Utilisateur responsable de l'administration de la plateforme. Il supervise la file d'attente de traitement des documents, consulte les indicateurs de santé du système, gère les erreurs de traitement et peut relancer les tâches échouées. Il dispose d'une vue globale sur tous les documents en cours de traitement.

TABLE 2.1 – Acteurs de la plateforme LexAI

2.2.2 Capture des besoins fonctionnels

Les besoins fonctionnels définissent les fonctionnalités que la plateforme LexAI doit fournir pour répondre aux attentes de ses utilisateurs.

- **Téléverser un contrat juridique** : Permettre au juriste de déposer un fichier contractuel aux formats PDF, DOCX, DOC, TXT ou HTML pour démarrer le pipeline d'analyse.
- **Extraire et normaliser le texte du contrat** : Extraire automatiquement le contenu textuel du document, normaliser le texte (encodage, retours à la ligne, espaces) et persister le résultat pour les agents aval.
- **Suivre l'état d'avancement du traitement** : Permettre à l'utilisateur de visualiser en temps réel l'état d'avancement du traitement du contrat (file d'attente, extraction, analyse, évaluation, recommandations) avec un indicateur de progression.
- **Consulter les résultats d'extraction** : Permettre au juriste de visualiser le texte brut extrait, le texte normalisé, la structure du document et les métadonnées d'extraction.
- **Analyser les clauses par NLP** : Détecter automatiquement la langue du document, segmenter le texte en clauses individuelles, classifier chaque clause selon une taxonomie de 12 types (obligation, résiliation, confidentialité, traitement de données, etc.) et extraire les entités juridiques nommées (parties, rôles, durées, montants, références légales).
- **Consulter l'analyse NLP** : Permettre au juriste de parcourir les clauses segmentées avec leurs labels, scores de confiance, flags de conformité et entités extraites.
- **Évaluer la conformité réglementaire** : Appliquer automatiquement le moteur de règles pour détecter les violations par rapport au LNPDP, RGPD, ISO 27001 et ISO 9001, calculer un score de conformité par référentiel et global, et déterminer le niveau de risque juridique.
- **Consulter l'évaluation de conformité** : Permettre au responsable conformité de visualiser le score de conformité global, les scores par référentiel, la liste des violations triées par sévérité et les clauses manquantes obligatoires.

- **Générer des recommandations de mise en conformité** : Produire pour chaque violation une recommandation concrète en français, une clause de remplacement rédigée et une justification juridique citant l'article de loi applicable.
- **Consulter et exporter les recommandations** : Permettre au juriste de consulter les recommandations classées par priorité et d'exporter le contrat révisé avec les clauses acceptées au format DOCX ou PDF.
- **Relancer le traitement d'un document échoué** : Permettre à l'administrateur de relancer la chaîne de traitement d'un document ayant échoué à n'importe quelle étape.
- **Supprimer un document** : Permettre à l'utilisateur de supprimer un document et toutes ses données associées de la plateforme.

2.2.3 Capture des besoins non fonctionnels

Les besoins non fonctionnels de la plateforme LexAI concernent les caractéristiques globales du système garantissant sa qualité, sa fiabilité et son adéquation aux usages professionnels :

- **Performance** : Le pipeline complet de traitement (extraction + NLP + évaluation + recommandations) doit s'exécuter en arrière-plan sans bloquer l'interface. Le temps de réponse de l'API REST ne doit pas dépasser 200 ms pour les requêtes de consultation.
- **Modularité** : Chaque agent est un composant autonome. L'ajout d'un nouveau format de document, d'une nouvelle règle légale ou d'un nouveau référentiel ne doit pas nécessiter de modification du code des autres agents.
- **Maintenabilité** : Les règles juridiques sont définies dans des fichiers JSON indépendants du code Python, permettant aux juristes de les mettre à jour sans intervention des développeurs.
- **Disponibilité et résilience** : En cas d'interruption du serveur, le service de récupération au démarrage détecte les documents bloqués en cours de traitement et les réinitialise pour retraitement.

- **Traçabilité** : Chaque étape du traitement est journalisée avec son statut, son pourcentage d'avancement, son message d'état et son éventuel message d'erreur, permettant un audit complet du traitement de chaque document.
- **Extensibilité** : L'architecture multi-agents LangGraph permet d'ajouter de nouveaux agents dans le pipeline sans modifier la logique existante.
- **Fiabilité des analyses** : Le classifieur de clauses utilise une stratégie en trois niveaux (modèle fine-tuné, zero-shot XLM-RoBERTa, heuristique par mots-clés) garantissant un résultat dans tous les cas, même sans connexion à un modèle externe.

2.2.4 Planification du travail avec Scrum

Le **backlog produit** constitue un artefact fondamental du framework Scrum. Il regroupe l'ensemble des fonctionnalités à développer, exprimées sous forme de *User Stories* et organisées selon leur niveau de priorité et de valeur pour l'utilisateur. Notre backlog est structuré autour des quatre agents du pipeline LexAI, chacun correspondant à un sprint de deux semaines.

2.3 Composition du backlog produit

2.3.1 Backlog de produit

Le tableau ci-dessous présente le backlog produit de la plateforme LexAI, avec les fonctionnalités à développer, les *User Stories* associées, leur priorité et leur durée estimée.

2.3.2 Planification des releases et sprints

Notre solution adopte une approche itérative et incrémentale, organisée en **trois releases** composées de sept sprints au total, couvrant l'ensemble du pipeline multi-agents ainsi que le déploiement Cloud et le pipeline CI/CD.

- **Release 1 (4 semaines)** — Pipeline d'ingestion et d'analyse NLP :
 - **Sprint 1 (2 semaines)** : Agent 1 — Pipeline d'extraction documentaire (PDF, DOCX, TXT, HTML)

ID	Feature	User Story	Priorité	Durée
US01	Extraction documentaire	En tant que juriste, je veux téléverser un contrat (PDF, DOCX, TXT, HTML ou image) et obtenir son texte extrait, normalisé et structuré afin de le soumettre au pipeline d'analyse.	Haute	2 semaines
US02	Analyse NLP des clauses	En tant que juriste, je veux que le contrat soit automatiquement segmenté en clauses, que chaque clause soit classifiée (obligation, confidentialité, données personnelles, etc.) et que les entités juridiques soient extraites (parties, rôles, durées, références légales).	Haute	2 semaines
US03	Évaluation de conformité	En tant que responsable conformité, je veux obtenir un score de conformité global et par référentiel (LNPDP, RGPD, ISO 27001, ISO 9001) ainsi que la liste détaillée des violations identifiées, afin d'évaluer le risque juridique du contrat dans le contexte tunisien.	Critique	2 semaines
US04	Recommandations de mise en conformité	En tant que juriste, je veux recevoir pour chaque violation une recommandation concrète accompagnée d'une clause de remplacement rédigée en français, avec sa justification légale (article LNPDP, RGPD ou ISO), afin de corriger le contrat directement.	Haute	2 semaines
US05	Déploiement et CI/CD	En tant qu'administrateur système, je veux que l'application soit conteneurisée via Docker, déployée sur un VPS Linux et mise à jour automatiquement via un pipeline CI/CD GitHub Actions à chaque modification du code source.	Moyenne	4 semaines

TABLE 2.2 – Backlog produit des fonctionnalités principales de LexAI

- **Sprint 2** (2 semaines) : Agent 2 — Segmentation des clauses, classification NLP et extraction d’entités juridiques
- **Release 2 (4 semaines)** — Pipeline d’évaluation et de recommandation :
 - **Sprint 3** (2 semaines) : Agent 3 — Moteur de règles juridiques (LNPDP, RGPD, ISO 27001) et scoring de conformité
 - **Sprint 4** (2 semaines) : Agent 4 — Génération de recommandations par templates et LLM (Llama 3 / Groq)
- **Release 3 (4 semaines)** — Déploiement Cloud, conteneurisation et CI/CD :
 - **Sprint 5–6** (2 semaines) : Infrastructure Cloud IaaS (MicroStack), conteneurisation Docker et orchestration via Docker Compose
 - **Sprint 7** (2 semaines) : Pipeline CI/CD GitHub Actions — build automatique, publication GHCR, auto-déploiement VPS

2.4 Architecture de l’application

2.4.1 Architecture logique — Pipeline multi-agents

La plateforme LexAI repose sur une **architecture multi-agents** orchestrée par **LangGraph**, un framework de graphes d’état conçu pour les systèmes d’intelligence artificielle à agents multiples. Chaque agent est un nœud autonome qui lit depuis un état partagé (**AnalysisState**), effectue son traitement et écrit ses résultats dans cet état avant de passer le relais à l’agent suivant.

Le pipeline complet se déroule comme suit :

1. **Agent 1 — Extractor** : Reçoit le fichier téléversé, valide le format, extrait le texte brut selon le type MIME (PDF via PyMuPDF, DOCX via python-docx, HTML via BeautifulSoup4, TXT direct), normalise le texte et persiste le résultat dans la table **extractions**.
2. **Agent 2 — NLP Analyzer** : Lit le texte normalisé, détecte la langue, segmente le document en clauses par détection de structures, d’en-têtes d’articles ou de para-

graphes, classe chaque clause (12 types) et extrait les entités nommées juridiques. Persiste les résultats dans la table `nlp_analyses`.

3. **Agent 3 — Evaluator** : Lit les clauses annotées de l'Agent 2, applique le moteur de règles JSON (44 règles couvrant LNPDP, RGPD, ISO 27001, ISO 9001), calcule le score de conformité pondéré et le niveau de risque juridique. Persiste les résultats dans la table `evaluations`.
4. **Agent 4 — Recommender** : Lit les violations de l'Agent 3, applique la bibliothèque de 24 templates de recommandations avec slot-filling depuis les entités NLP, et produit une clause révisée par violation. Persiste les résultats dans la table `recommendations`.

Chaque agent est exécuté de manière asynchrone via **Celery**, avec le statut et le pourcentage d'avancement mis à jour en temps réel dans la base de données PostgreSQL. L'API FastAPI expose ces informations au frontend React, qui interroge l'API toutes les deux secondes pendant le traitement.

2.4.2 Architecture physique

La plateforme adopte une **architecture à trois niveaux** offrant une séparation claire des responsabilités, une meilleure évolutivité et une facilité de maintenance.

- **Niveau de présentation** : Interface utilisateur développée avec *React 18* et *TypeScript*, stylisée avec *Tailwind CSS*. Elle permet la gestion du téléversement, le suivi de la progression, la visualisation des résultats de chaque agent et l'export du contrat révisé. Les échanges avec le backend s'effectuent via des requêtes HTTP vers l'API REST.

- **Niveau application** : Backend développé avec *FastAPI* exposant des endpoints REST sécurisés. Les tâches longues (extraction, NLP, évaluation, recommandations) sont déléguées à des workers *Celery* communiquant via un broker *Redis*. Les agents du pipeline sont orchestrés par *LangGraph*.

- **Niveau de données** : Base de données relationnelle *PostgreSQL 16* gérant la persistance de tous les artefacts du pipeline (documents, extractions, analyses NLP, évaluations, recommandations). Les migrations sont gérées par *Alembic*.

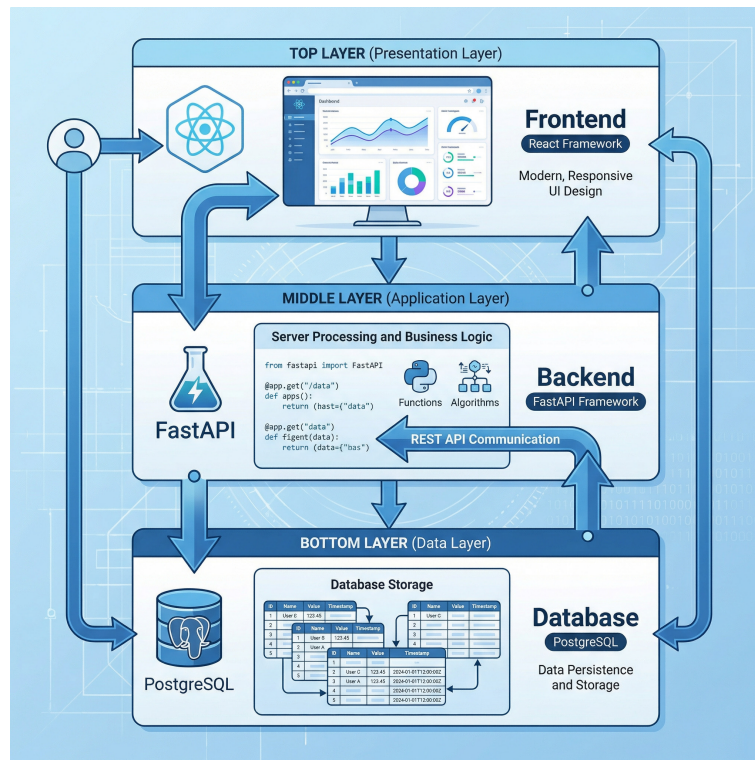


FIGURE 2.1 – Architecture physique à trois niveaux de LexAI

2.5 Environnement logiciel du projet

Dans cette partie, nous présentons l'environnement logiciel adopté pour la réalisation de la plateforme LexAI.

2.5.1 Outils de gestion de projet

- **Git** : Système de gestion de versions distribué pour le suivi de l'évolution du code source et la gestion des branches de développement.
- **GitHub** : Plateforme d'hébergement du dépôt du projet, utilisée pour la collaboration et le partage du code.
- **Docker & Docker Compose** : Conteneurisation de tous les services (FastAPI, Celery, Redis, PostgreSQL) pour un environnement de développement reproductible et cohérent.

2.5.2 Technologies de développement

Le choix des technologies de LexAI a été guidé par trois critères principaux : la performance du traitement asynchrone, la modularité du pipeline multi-agents et la compatibilité avec le traitement du langage naturel en français. L'ensemble de la pile technique est open source, déployable sous Docker et organisée en couches clairement séparées (API, tâches, NLP, frontend).

Le tableau ?? présente l'ensemble des technologies adoptées pour la réalisation de LexAI, classées par couche fonctionnelle.

Couche	Technologie	Rôle
API REST	FastAPI 0.109+	Endpoints REST, validation Pydantic, gestion asynchrone
File d'attente	Celery 5.3+	Tâches d'extraction et d'analyse asynchrones
Broker	Redis 7	Broker de messages Celery et backend de résultats
Base de données	PostgreSQL 16	Stockage persistant de tous les artefacts
ORM / Migrations	SQLAlchemy 2.0 + Alembic	Accès DB typé et migrations versionnées
Orchestration multi-agents	LangGraph 0.2+	Graphe d'état pour le pipeline 4 agents
Extraction PDF	PyMuPDF (fitz)	Extraction page par page avec métadonnées
Extraction DOCX	python-docx	Extraction Word avec détection de titres
Extraction HTML	BeautifulSoup4	Nettoyage HTML et extraction textuelle

TABLE 2.3 – Stack technologique de LexAI — Backend et extraction (1/2)

Couche	Technologie	Rôle
NLP	spaCy 3.7+	Tokenisation, NER, segmentation de clauses
Transformers	HuggingFace Transformers	Classification zero-shot et fine-tuning
Modèles NLP	XLM-RoBERTa, CamemBERT	Classification multi-label de clauses en français
LLM	Llama 3 via API Groq	Enrichissement et réécriture des recommandations
Frontend	React 18 + TypeScript	Interface utilisateur réactive et typée
CSS	Tailwind CSS 3	Stylisation utilitaire cohérente
Bundler	Vite 5	Builds frontend rapides
Conteneurisation	Docker + Docker Compose	Isolation des services et déploiement reproductible
CI/CD	GitHub Actions + GHCR	Build automatique et déploiement continu sur VPS
Serveur web	Nginx	Reverse proxy frontend et routage des requêtes API

TABLE 2.4 – Stack technologique de LexAI — NLP, IA, Frontend et DevOps (2/2)

2.6 Diagramme de cas d'utilisation global

La figure 2.2 présente le diagramme de cas d'utilisation global de la plateforme LexAI, synthétisant l'ensemble des interactions entre les trois acteurs (Juriste, Responsable conformité, Administrateur) et les neuf fonctionnalités principales de la plateforme.

Diagramme de cas d'utilisation global - Plateforme LexAI

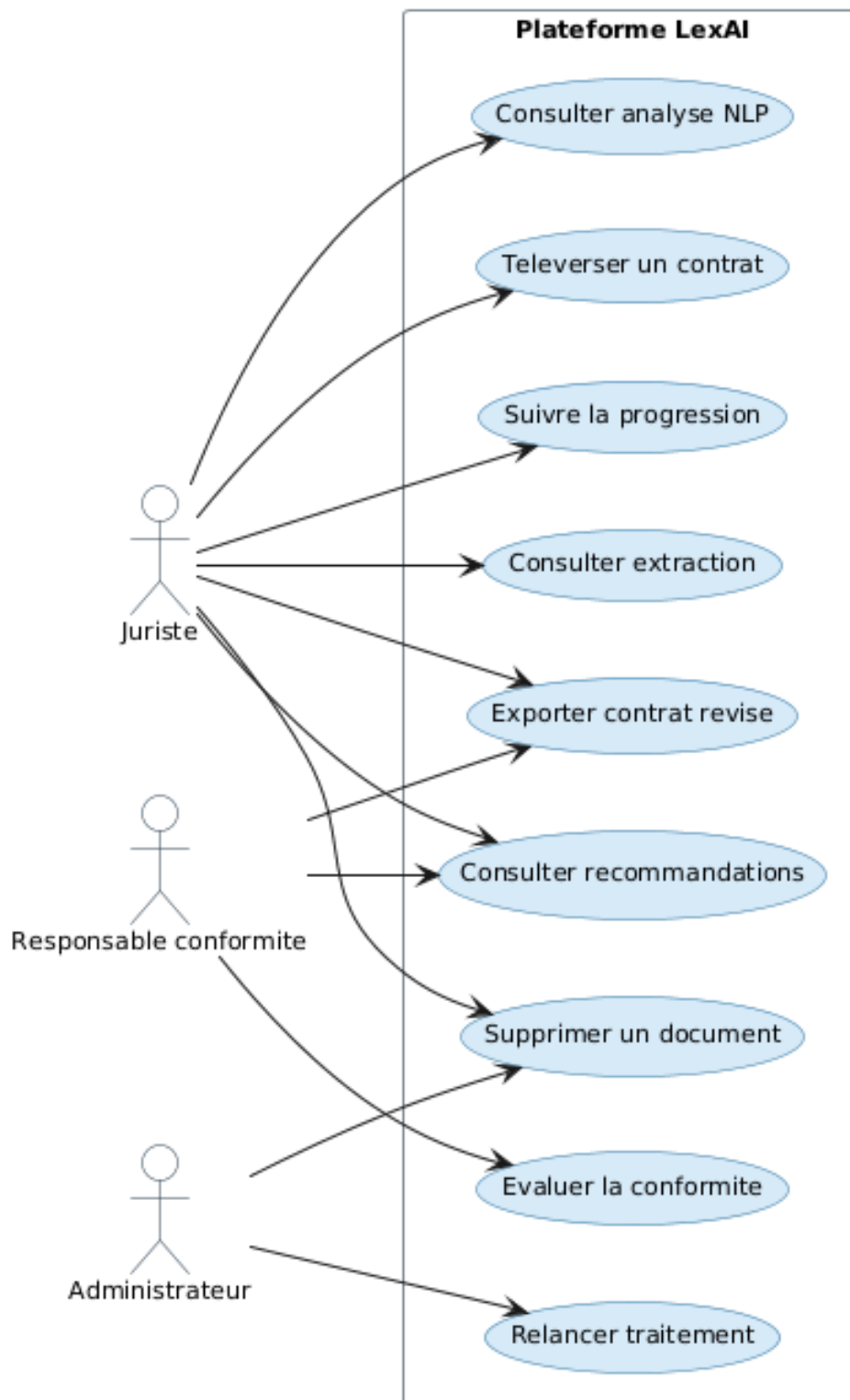


FIGURE 2.2 – Diagramme de cas d'utilisation global de la plateforme LexAI

Ce diagramme illustre clairement la séparation des responsabilités entre les acteurs :

- Le **Juriste** est l'acteur central, interagissant avec toutes les étapes du pipeline

(téléversement, consultation des résultats, recommandations, export).

- Le **Responsable conformité** est focalisé sur l'évaluation réglementaire et l'exploitation des recommandations.
- L'**Administrateur** assure la gestion opérationnelle (relance, suppression).

2.7 Contraintes et exigences de qualité

2.7.1 Contraintes techniques

La plateforme LexAI doit respecter les contraintes techniques suivantes :

- **Compatibilité des formats** : Les formats PDF, DOCX, DOC, TXT et HTML doivent être pris en charge sans dépendances externes non-open source.
- **Taille maximale des fichiers** : Les contrats ne doivent pas dépasser 50 Mo par téléversement.
- **Isolation des traitements** : Chaque tâche d'analyse est exécutée dans un worker Celery isolé pour éviter les interférences entre documents.
- **Déploiement local** : La plateforme doit fonctionner entièrement en local via Docker Compose, sans dépendance à un service cloud externe (hormis le LLM optionnel).
- **Compatibilité Python** : Le backend est développé pour Python 3.10+ afin d'assurer la compatibilité avec spaCy 3.7 et HuggingFace Transformers 4.x.

2.7.2 Exigences de sécurité

- **Isolation des données** : Les contrats téléversés sont stockés sur le système de fichiers local du serveur. Aucune donnée contractuelle ne transite vers des services tiers en dehors du LLM optionnel.
- **Validation des entrées** : Le type MIME et l'extension du fichier sont vérifiés côté serveur (liste blanche) pour prévenir les injections de fichiers malveillants.
- **En-têtes CORS** : L'API FastAPI est configurée avec une liste blanche d'origines autorisées pour prévenir les requêtes cross-origin non autorisées.

- **Aucune exécution de code** : Le contenu des contrats est traité exclusivement comme du texte — aucune évaluation de code ou de formule n'est effectuée.

2.7.3 Exigences de performance

Opération	Cible	Condition
Réponse API (consultation)	< 200 ms	Requêtes GET /documents
Extraction PDF (10 pages)	< 5 secondes	Fichier non scanné
Analyse NLP (50 clauses)	< 30 secondes	Mode zero-shot XLM-R
Évaluation de conformité	< 3 secondes	44 règles, 50 clauses
Génération de recommandations	< 5 secondes	24 templates, 10 violations
Pipeline complet (4 agents)	< 60 secondes	Contrat standard 20 pages

TABLE 2.5 – Exigences de performance de la plateforme LexAI

2.8 Infrastructure et déploiement

2.8.1 Conteneurisation avec Docker Compose

La plateforme LexAI est entièrement conteneurisée via **Docker Compose**, permettant un déploiement reproductible et cohérent en un seul commande sur n'importe quelle machine disposant de Docker.

L'infrastructure est composée des services suivants :

Service	Image	Port	Rôle
backend	Python 3.10 (custom)	8000	API FastAPI — point d'entrée de tous les appels REST
worker	Python 3.10 (custom)	—	Worker Celery — exécute les 4 tâches d'analyse
redis	redis :7-alpine	6379	Broker Celery + backend de résultats
postgres	postgres :16-alpine	5432	Base de données relationnelle persistante
frontend	node :20-alpine	5173	Serveur de développement Vite / React

TABLE 2.6 – Services Docker Compose de la plateforme LexAI

2.8.2 Volumes et persistance

Deux volumes Docker nommés garantissent la persistance des données entre les redémarrages :

- `postgres_data` : stocke les données PostgreSQL (documents, extractions, analyses, évaluations, recommandations).
- `uploads_data` : stocke les fichiers contractuels téléversés sur le serveur.

2.8.3 Variables d'environnement

La configuration de la plateforme est entièrement pilotée par des variables d'environnement définies dans le fichier `.env` à la racine du projet :

Variable	Valeur par défaut	Description
DATABASE_URL	(postgres local)	URL de connexion PostgreSQL
REDIS_URL	(redis local)	URL du broker Redis
UPLOAD_DIR	/app/uploads	Répertoire de stockage des fichiers
MAX_FILE_SIZE_MB	50	Limite de taille des fichiers
LLM_ENABLED	false	Active l'enrichissement LLM (Groq)
LLM_API_KEY	–	Clé API Groq (si LLM activé)
LLM_MODEL	mixtral-8x7b	Modèle LLM à utiliser
FRONTEND_ORIGINS	localhost :5173	Origines CORS autorisées

TABLE 2.7 – Variables d’environnement de LexAI

2.8.4 Lancement de la plateforme

Le lancement complet de la plateforme s’effectue en une seule commande :

```
docker-compose up --build
```

Cette commande construit les images Docker du backend et du frontend, démarre tous les services dans l’ordre correct (PostgreSQL et Redis avant le backend), exécute automatiquement les migrations Alembic au démarrage, et démarre le worker Celery en parallèle du serveur FastAPI.

L’interface utilisateur est alors accessible à <http://localhost:5173> et l’API REST à <http://localhost:8000>.

2.9 Conclusion

Dans ce chapitre, nous avons défini les acteurs de la plateforme LexAI, spécifié les besoins fonctionnels et non fonctionnels, établi le backlog produit et planifié les deux releases de développement. Nous avons présenté le diagramme de cas d'utilisation global, décrit l'architecture logique multi-agents LangGraph, l'architecture physique à trois niveaux, les contraintes de qualité et l'infrastructure Docker Compose assurant le déploiement reproductible de la plateforme. Le chapitre suivant décrit en détail la première release, couvrant le développement de l'Agent 1 (extraction documentaire) et de l'Agent 2 (analyse NLP des clauses).

CHAPITRE 3

**Release 1 : Extraction documentaire et
Analyse NLP**

3.1 Introduction

Dans ce chapitre, nous présentons le travail réalisé lors de la première release, composée de deux sprints. Le **Sprint 1** couvre l'Agent 1, responsable de l'ingestion et de l'extraction documentaire. Le **Sprint 2** couvre l'Agent 2, responsable de l'analyse NLP des clauses contractuelles. Pour chaque sprint, nous détaillons le backlog, l'analyse fonctionnelle avec les diagrammes UML, la conception de la base de données et la réalisation des interfaces.

3.2 Planification des sprints de la Release 1

La Release 1 est planifiée en deux sprints successifs de deux semaines chacun. Le Sprint 1 porte sur l'ingestion et l'extraction documentaire (Agent 1), et le Sprint 2 sur l'analyse NLP des clauses (Agent 2). Cette organisation permet une mise en œuvre progressive du pipeline d'ingestion.

3.3 Sprint 1 : Agent 1 — Pipeline d'extraction documentaire

3.3.1 Backlog du Sprint 1

Nom	Description	Priorité	Estimation	Tâches de développement
Téléversement de contrat	En tant que juriste, je veux déposer un fichier contractuel sur la plateforme.	Haute	2 jours	- Validation MIME et extension - Limite taille (50 Mo) - Sauvegarde sur disque - Création enregistrement DB
Extraction PDF	En tant que système, je veux extraire le texte d'un PDF via PyMuPDF.	Haute	2 jours	- Implémentation PdfProvider - Extraction page par page - Détection PDF scanné - Collecte métadonnées
Extraction DOCX/DOC	En tant que système, je veux extraire le texte d'un Word via python-docx.	Haute	1 jour	- Implémentation DocxProvider - Extraction paragraphes - Détection titres/sections
Extraction TXT/HTML	En tant que système, je veux extraire le texte d'un fichier TXT ou HTML.	Moyenne	1 jour	- Implémentation TxtProvider - Implémentation HtmlProvider - Nettoyage scripts/CSS

Nom	Description	Priorité	Estimation	Tâches de développement
Normalisation du texte	En tant que système, je veux normaliser le texte extrait pour le préparer à l'analyse NLP.	Haute	1 jour	- UTF-8, CRLF → LF - Collapsage espaces multiples - Collapsage sauts de ligne
Tâche Celery asynchrone	En tant que système, je veux exécuter l'extraction en arrière-plan avec suivi de progression.	Haute	2 jours	- Tâche Celery avec statuts - Mise à jour % progression - Gestion des erreurs et retry
API REST & Frontend	En tant que juriste, je veux suivre l'avancement et consulter les résultats d'extraction.	Haute	2 jours	- Endpoints GET /documents - Composant Progress-Card - Composant ExtractionViewer

TABLE 3.1 – Backlog du Sprint 1 — Agent 1

3.3.2 Analyse des besoins du Sprint 1

Diagramme de cas d'utilisation du Sprint 1

Le diagramme ci-dessous illustre les interactions entre les acteurs et le système lors du Sprint 1, couvrant le téléversement, le suivi et la consultation des résultats d'extraction.

Diagramme de cas d'utilisation — Sprint 1 : Agent 1 — Extraction documentaire

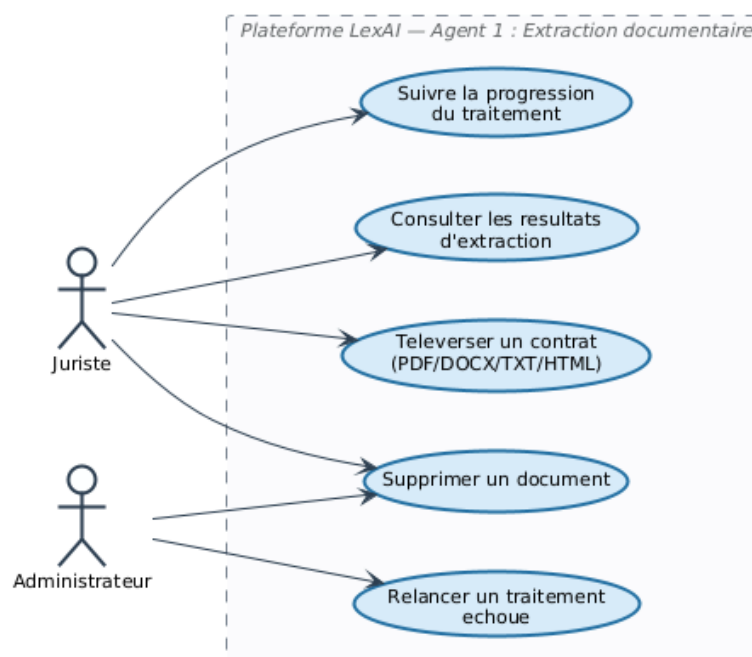


FIGURE 3.1 – Diagramme de cas d'utilisation du Sprint 1

Description du cas d'utilisation : « Téléverser et extraire un contrat »

Cas d'utilisation	Téléverser et extraire un contrat
Acteur	Juriste / Responsable légal
Pré-condition	L'utilisateur dispose d'un contrat au format PDF, DOCX, TXT ou HTML
Post-condition	Le texte du contrat est extrait, normalisé et disponible pour l'analyse NLP
Description du scénario principal	1- Le juriste sélectionne un fichier via l'interface de téléversement 2- Le système valide le type MIME, l'extension et la taille du fichier 3- Le fichier est sauvegardé sur le serveur et un enregistrement <i>Document</i> est créé en base avec le statut queued 4- Une tâche Celery est déclenchée ; le statut passe à extracting 5- Le provider approprié extrait le texte brut 6- Le texte est normalisé 7- Les résultats sont persistés dans la table extractions 8- Le statut passe à extracted et la progression atteint 100%
Exception	- Format non supporté : message d'erreur retourné à l'utilisateur - Fichier trop volumineux (> 50 Mo) : rejet immédiat - Échec d'extraction : statut failed, message d'erreur persisté, option de relance disponible

TABLE 3.2 – Description textuelle du cas « Téléverser et extraire un contrat »

Diagramme de séquence « Téléverser et extraire un contrat »

Le diagramme de séquence ci-dessous illustre le déroulement temporel des interactions entre le juriste, l'API FastAPI, le worker Celery, le registre de providers et la base de données lors de l'extraction d'un contrat.

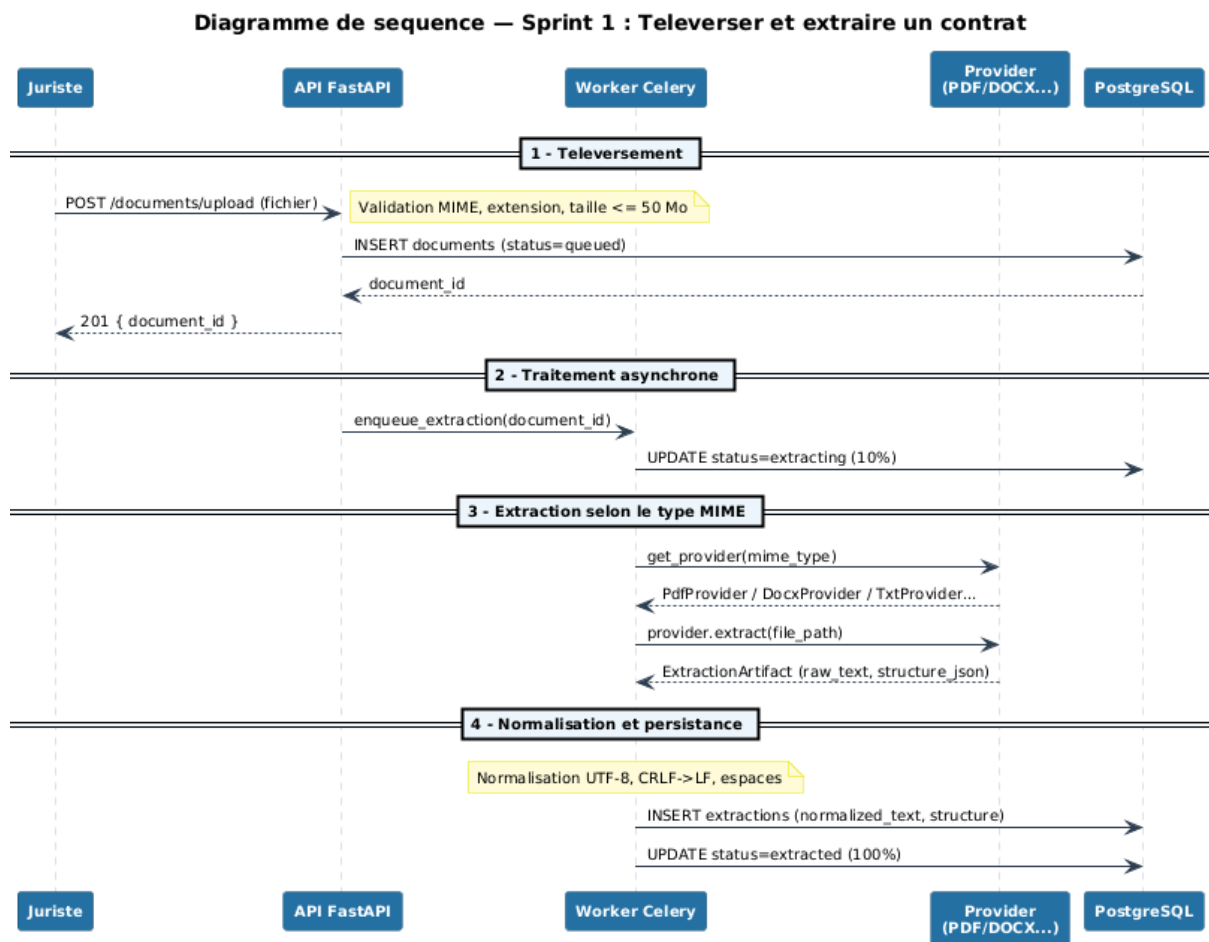


FIGURE 3.2 – Diagramme de séquence : Téléverser et extraire un contrat

3.3.3 Conception du Sprint 1

Dictionnaire des données du Sprint 1

Le tableau 3.3 présente la structure des tables créées lors du Sprint 1.

TABLE 3.3 – Dictionnaire des données du Sprint 1

Table	Attribut	Type	Contraintes	Description
	id	SERIAL	PK	Identifiant unique du document
documents	filename	VARCHAR(255)	NOT NULL	Nom de fichier généré par le serveur
	original_filename	VARCHAR(255)	NOT NULL	Nom original du fichier téléversé
	mime_type	VARCHAR(100)	NOT NULL	Type MIME détecté automatiquement
	file_size	INTEGER	NOT NULL	Taille du fichier en octets
	status	VARCHAR(50)	NOT NULL	État du pipeline : <i>queued, extracting, extracted, failed</i>
	progress_percent	INTEGER	DEFAULT 0	Avancement du traitement (0–100)
	progress_stage	VARCHAR(100)	NULLABLE	Étape courante du pipeline
	progress_message	TEXT	NULLABLE	Message d'état lisible par l'utilisateur
	last_error	TEXT	NULLABLE	Dernier message d'erreur enregistré

Table	Attribut	Type	Contraintes	Description
	task_id	VARCHAR(255)	NULLABLE	Identifiant de la tâche Celery
	created_at	TIMESTAMP	NOT NULL	Date de téléversement
	finished_at	TIMESTAMP	NULLABLE	Date de fin du pipeline
	id	SERIAL	PK	Identifiant unique de l'extraction
extractions	document_id	INTEGER	FK → documents	Référence au document parent
	raw_text	TEXT	NOT NULL	Texte brut extrait du fichier source
	normalized_text	TEXT	NOT NULL	Texte normalisé, entrée de l'Agent 2
	structure_json	TEXT	NULLABLE	Structure du document (sections, titres)
	page_metadata_json	TEXT	NULLABLE	Métadonnées par page (PDF uniquement)
	warnings	JSONB	NULLABLE	Avertissements d'extraction (encodage, PDF scanné)

Table	Attribut	Type	Contraintes	Description
	created_at	TIMESTAMP	NOT NULL	Date de création de l'enregistrement

3.3.4 Réalisation du Sprint 1

Interface de téléversement de contrat

L'interface de téléversement permet au juriste de déposer un contrat par glisser-déposer ou via un sélecteur de fichiers. Les formats acceptés (PDF, DOCX, TXT, HTML) sont clairement indiqués.

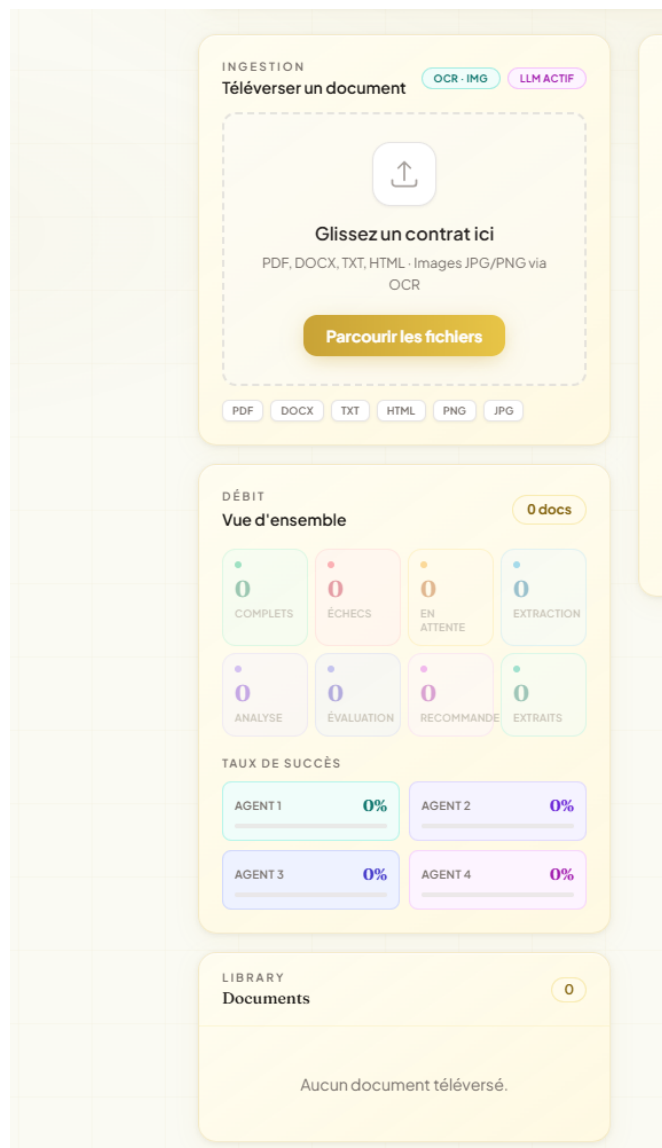


FIGURE 3.3 – Interface de téléversement de contrat

Interface de suivi de progression

La carte de progression affiche l'état courant du traitement (étape et pourcentage), le nom du fichier et un indicateur visuel de la barre de progression, actualisés toutes les deux secondes.



FIGURE 3.4 – Interface de suivi de progression du traitement

Interface de visualisation des résultats d'extraction

L'ExtractionViewer présente au juriste le texte brut, le texte normalisé, la structure détectée (sections, titres) et les éventuels avertissements (PDF scanné, encodage non reconnu).

Résultat d'extraction
CONTRAT DE TRAVAIL A DUREE INDETERMINEE (1).pdf

TEXTE BRUT	TEXTE NORMALISÉ
CONTRAT DE TRAVAIL A DUREE INDETERMINEE Entre les soussignés : La Société Iberis Suite, sise Bureau 19 Centre Millenium, 2ème Etage, ayant le matricule fiscal numéro 1729801/F, représentée par son gérant Mr Mohamed Alaa Chaibi. Ci-après dénommée « l'Employeur » D'une part, Et	Il a été arrêté et convenu ce qui suit : ARTICLE 1: Engagement L'Employeur engage aux termes du présent contrat de travail à durée indéterminée l'Employé, qui accepte, de travailler en qualité de Développeur Informatique à compter du 13/10/2025. Cet engagement est conclu sous réserve d'une période d'essai de six (6) mois, soit du 13/10/2025 au 13/04/2026, renouvelable une fois pour une période similaire. Durant cette période d'essai, le contrat pourra être rompu à l'initiative de l'une ou l'autre partie, moyennant un préavis de :

MÉTADONNÉES DE STRUCTURE

```
{
  "type": "pdf",
  "page_count": 6,
  "pages": [
    {
      "page": 1,
      "char_count": 624
    },
    {
      "page": 2,
      "char_count": 1163
    },
    {
      "page": 3,
      "char_count": 1344
    },
    {
      "page": 4,
      "char_count": 1749
    },
    {
      "page": 5,
      "char_count": 1490
    },
    {
      "page": 6,
      "char_count": 997
    }
  ]
}
```

FIGURE 3.5 – Interface de visualisation des résultats d'extraction

3.4 Sprint 2 : Agent 2 — Analyse NLP des clauses

3.4.1 Backlog du Sprint 2

TABLE 3.4 – Backlog du Sprint 2 — Agent 2

Fonctionnalité	User Story	Estim.	Tâches de développement
Détection de la langue	En tant que système, je veux détecter automatiquement la langue du contrat (fr/ar/en) pour adapter le traitement NLP.	1j	Module <code>LanguageDetector</code> ; heuristique caractères arabes; fallback <code>langdetect</code> .
Segmentation en clauses	En tant que juriste, je veux que le contrat soit segmenté en clauses individuelles avec leurs positions.	3j	Stratégie <code>structure_json</code> ; regex en-têtes d'articles; fallback sauts de paragraphe; calcul offsets caractères.
Extraction d'entités juridiques	En tant que juriste, je veux identifier automatiquement les parties, rôles, durées, montants et références légales.	2j	Intégration <code>spaCy</code> NER; patterns <code>PARTY/ROLE</code> , <code>DATA_CATEGORY</code> , <code>LAW_REFERENCE</code> , <code>DURATION</code> .
Classification des clauses	En tant que juriste, je veux que chaque clause soit classifiée selon sa nature (obligation, résiliation, confidentialité, etc.).	3j	Intégration <code>XLM-RoBERTa</code> zero-shot; cache pipeline niveau module; heuristique mots-clés fallback; détection flags de conformité.
Tâche Celery + API	En tant que juriste, je veux consulter l'analyse NLP d'un document via l'API REST.	1j	Tâche <code>run_nlp_analysis</code> ; endpoint <code>GET /analysis</code> ; endpoint <code>GET /clauses</code> .

Fonctionnalité	User Story	Estim.	Tâches de développement
Interface <code>AnalysisViewer</code>	En tant que juriste, je veux visualiser les clauses, leurs labels, entités et flags de conformité dans une interface dédiée.	2j	Composant <code>AnalysisViewer</code> ; filtres par label et flag; badges entités colorés.

3.4.2 Analyse des besoins du Sprint 2

Besoins fonctionnels

- Détecter automatiquement la langue du document (français, arabe, anglais).
- Segmenter le texte normalisé en clauses individuelles avec leurs positions (offset de début et de fin).
- Classifier chaque clause selon 12 types : *definition, obligation, liability, termination, data_processing, confidentiality, dispute_resolution, force_majeure, penalty, ip_rights, payment, warranty*.
- Extraire les entités nommées juridiques : PARTY, ROLE, DATA_CATEGORY, DURATION, AMOUNT, LAW_REFERENCE, JURISDICTION, DATE.
- Détecter les flags de conformité : *lnpdp_relevant, gdpr_relevant, missing_retention_period, missing_consent_mechanism, missing_security_measures, missing_data_subject_rights, unlawful_cross_border_transfer, excessive_data_collection*.
- Déclencher automatiquement l'analyse après la fin de l'extraction (chaînage Celery).

Besoins non fonctionnels

- **Performance** : Le pipeline NLP ne doit pas recharger le modèle de transformeurs à chaque clause. Le pipeline est chargé une fois en mémoire au niveau module.
- **Résilience** : En l'absence de modèle fine-tuné, le système bascule automatiquement en mode zero-shot, puis en mode heuristique par mots-clés.

- **Observabilité** : Le mode de classification actif (fine-tuné / zero-shot / heuristique) est tracé dans les logs au démarrage.

Diagramme de cas d'utilisation du Sprint 2

Diagramme de cas d'utilisation — Sprint 2 : Agent 2 — Analyse NLP

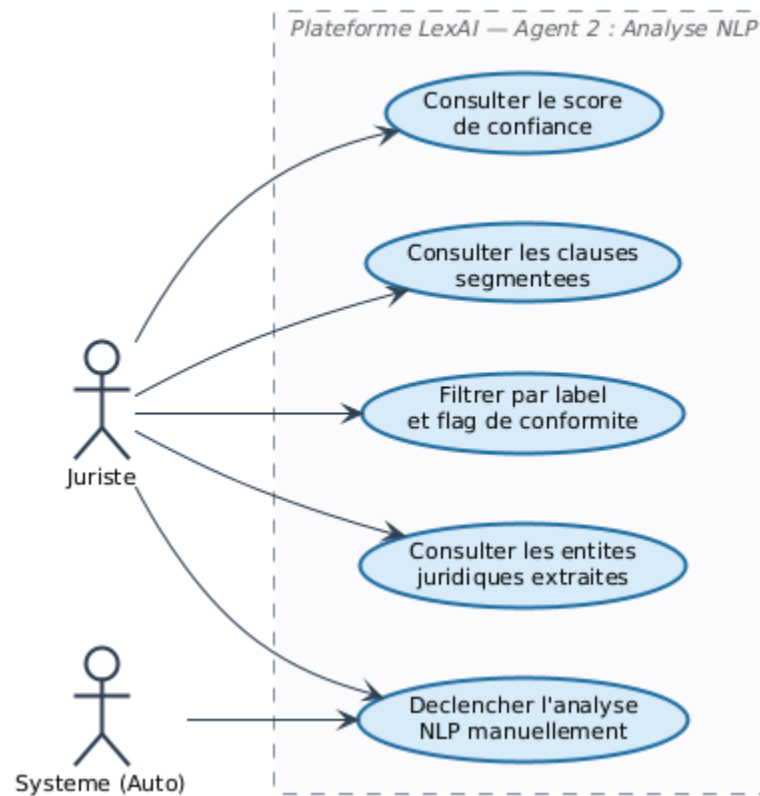


FIGURE 3.6 – Diagramme de cas d'utilisation du Sprint 2

Cas d'utilisation	Analyser les clauses NLP
Acteur	Système (déclenchement automatique après extraction)
Pré-condition	Le document est au statut extracted et le texte normalisé est disponible
Post-condition	Les clauses sont segmentées, classifiées et les entités extraites ; le document passe à analyzed
Description du scénario principal	1- La tâche Celery run_nlp_analysis est déclenchée automatiquement à la fin de l'extraction 2- Le LanguageDetector identifie la langue du texte 3- Le ClauseSegmenter découpe le texte en clauses (3 stratégies) 4- Pour chaque clause, l' EntityExtractor identifie les entités nommées 5- Pour chaque clause, le ClauseClassifier assigne les labels et flags de conformité 6- Les résultats sont persistés dans la table nlp_analyses 7- Le statut du document passe à analyzed
Exception	- Modèle NLP non disponible : bascule automatique sur heuristique par mots-clés - Texte vide ou illisible : clause marquée comme non classifiable

TABLE 3.5 – Description textuelle du cas « Analyser les clauses NLP »

Description du cas d'utilisation : « Analyser les clauses NLP »

Diagramme de séquence « Analyser les clauses NLP »

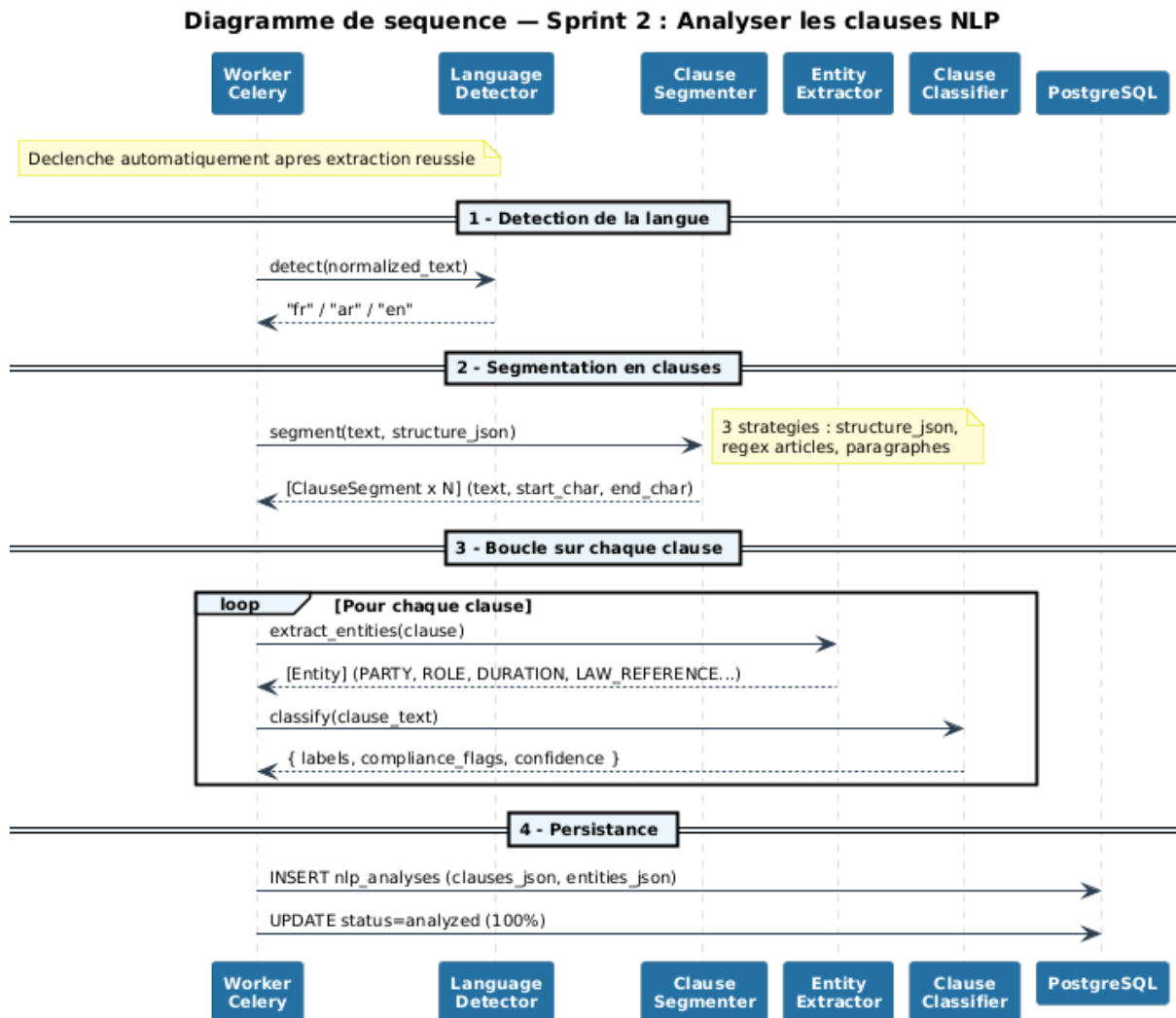


FIGURE 3.7 – Diagramme de séquence : Analyser les clauses NLP

3.4.3 Conception du Sprint 2

Dictionnaire des données du Sprint 2

TABLE 3.6 – Dictionnaire des données du Sprint 2

Table	Attribut	Type	Contraintes	Description
	id	SERIAL	PK	Identifiant unique de l'analyse NLP
nlp_analyses	document_id	INTEGER	FK → documents	Référence au document parent
	language	VARCHAR(10)	NOT NULL	Langue détectée du contrat (fr/ar/en)
	clause_count	INTEGER	NOT NULL	Nombre de clauses segmentées
	clauses_json	TEXT	NOT NULL	Tableau JSON des objets clause
	entities_json	TEXT	NULLABLE	Entités nommées agrégées du document
	analysis_metadata_json	TEXT	NULLABLE	Métadonnées d'analyse (modèle, durée)
	created_at	TIMESTAMP	NOT NULL	Date de création de l'enregistrement
<i>Structure d'un objet clause dans clauses_json :</i>				
	clause_id	VARCHAR(50)	—	Identifiant unique de clause (c-001, c-002...)

Table	Attribut	Type	Contraintes	Description
	text	TEXT	—	Texte brut de la clause
	start_char	INTEGER	—	Offset de début dans le texte normalisé
	end_char	INTEGER	—	Offset de fin dans le texte normalisé
	labels	JSONB	—	Types de clause détectés
	compliance_flags	JSONB	—	Flags de conformité (LNPDP, GDPR...)
	entities	JSONB	—	Entités nommées extraites de la clause
	confidence	FLOAT	—	Score de confiance du classifieur (0–1)
	language	VARCHAR(10)	—	Langue de la clause

3.4.4 Construction du dataset et entraînement du modèle

Problématique et stratégie

L'un des défis majeurs du projet LexAI est l'absence d'un corpus annoté de clauses contractuelles en langue française adapté au contexte tunisien. Pour y remédier, un dataset multi-sources a été construit, combinant des données juridiques anglophones de référence avec des exemples synthétiques en français, afin d'entraîner un modèle de classification multi-label de clauses.

Sources de données

Le dataset final est issu de trois sources complémentaires, fusionnées et équilibrées via le script `build_dataset.py` :

Source	Langue	Description
CUAD	Anglais	Contract Understanding Atticus Dataset — 510 contrats commerciaux annotés, référence académique pour la classification de clauses
LEDGAR	Anglais	Large-scale legal clause dataset — plus de 800 000 clauses issues de dépôts SEC, couvrant 100 types de clauses
Synthetic French	Français	Clauses synthétiques en français générées par <code>generate_synthetic.py</code> , couvrant les 12 types cibles dans le contexte du droit tunisien

TABLE 3.7 – Sources du dataset d’entraînement — Agent 2

Construction et équilibrage du dataset

Pour éviter la domination d’une seule source et garantir la présence du français dans le modèle, les règles suivantes ont été appliquées :

- **Plafonnement par label** : maximum 5 000 exemples par classe pour éviter le déséquilibre lié à la sur-représentation des clauses d’*obligation* dans LEDGAR.
- **Sur-échantillonnage du français** : les exemples en français sont dupliqués $\times 4$ pour compenser le volume beaucoup plus faible de données françaises par rapport aux données anglophones.
- **Division** : 80% entraînement — 10% validation — 10% test (graine aléatoire fixe : 42).

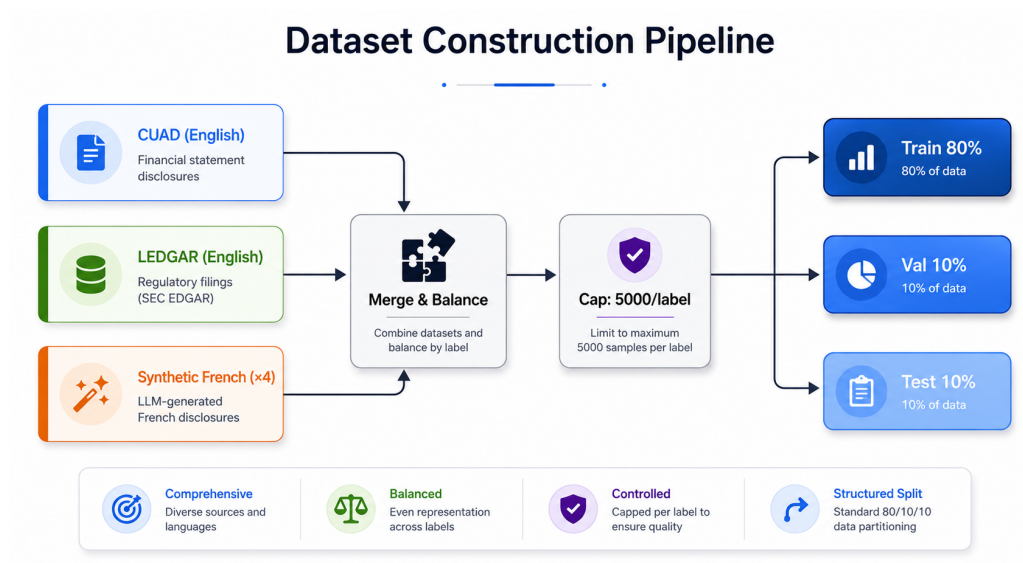


FIGURE 3.8 – Pipeline de construction du dataset d'entraînement

Architecture du modèle

Le modèle retenu est **XLM-RoBERTa base** (`xlm-roberta-base`), un modèle de langue multilingue pré-entraîné sur 100 langues dont le français et l'anglais. Il a été fine-tuné pour la **classification multi-label** sur les 12 types de clauses cibles.

Les paramètres d'entraînement sont :

Paramètre	Valeur
Modèle de base	xlm-roberta-base (HuggingFace)
Tâche	Classification multi-label (12 classes indépendantes)
Fonction de perte	BCEWithLogitsLoss (décision binaire par label)
Longueur maximale	256 tokens par clause
Taille de batch	16
Nombre d'époques	4
Taux d'apprentissage	2×10^{-5}
Seuil de décision	0.40 (score sigmoid pour activer un label)
Sortie	data/models/clause_classifier/

TABLE 3.8 – Paramètres d'entraînement du modèle XLM-RoBERTa

Labels de classification

Le modèle classifie chaque clause selon 12 types mutuellement non-exclusifs :

```
definition · obligation · liability · termination · data_processing ·
confidentiality
dispute_resolution · force_majeure · penalty · ip_rights · payment ·
warranty
```

Stratégie de déploiement du modèle

Le `ClauseClassifier` implémente une stratégie de repli en trois niveaux pour garantir un résultat dans tous les cas :

1. **Modèle fine-tuné** (`data/models/clause_classifier/`) — meilleure précision, utilisé si disponible.
2. **Zero-shot XLM-RoBERTa** (`joelniklaus/legal-xlm-roberta-large`) — aucun entraînement requis, utilisé si le modèle fine-tuné est absent.
3. **Heuristique par mots-clés** — toujours disponible, ne nécessite aucun modèle externe.

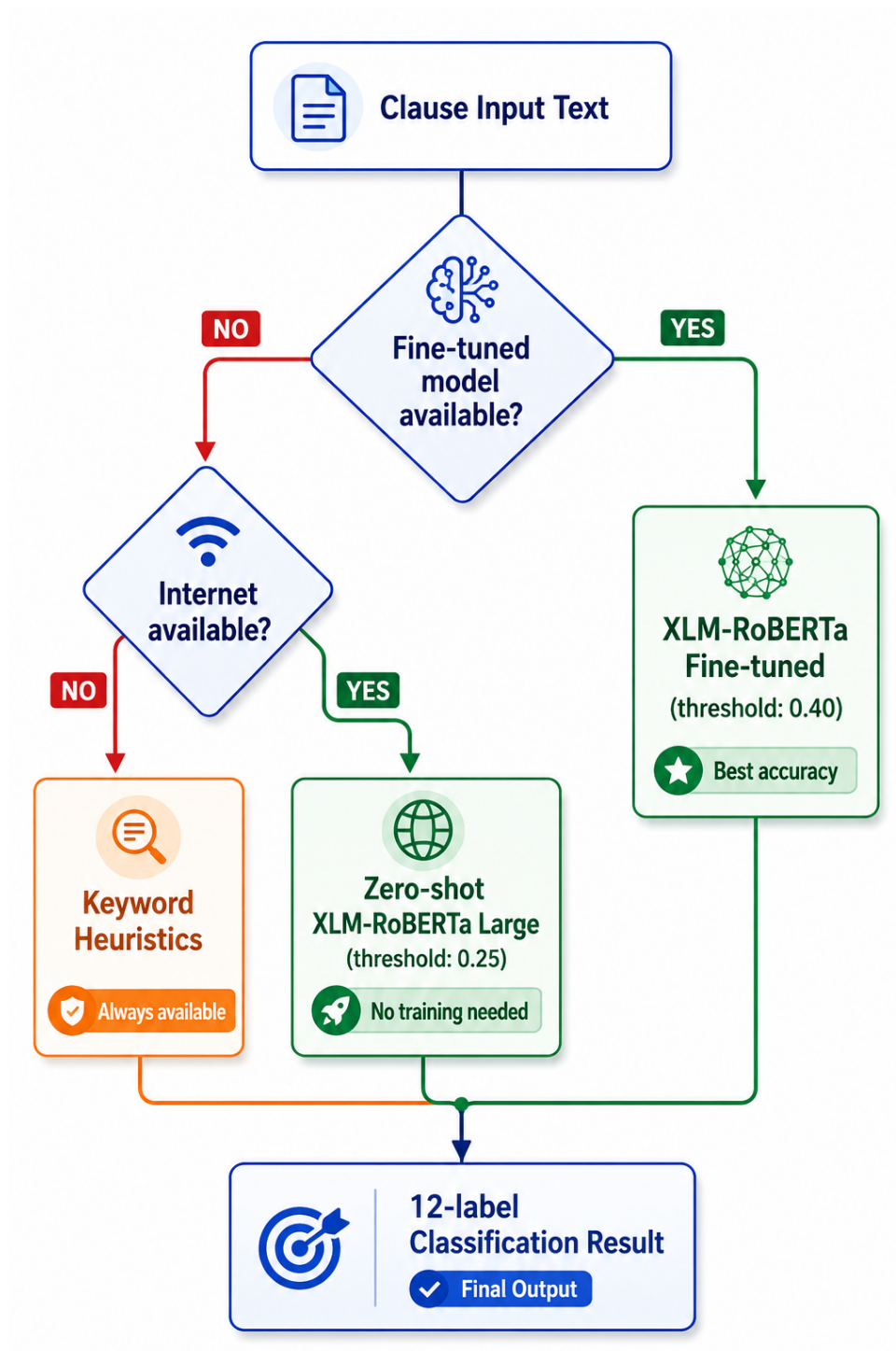


FIGURE 3.9 – Stratégie de classification multi-niveaux du ClauseClassifier

3.4.5 Réalisation du Sprint 2

Interface de visualisation de l'analyse NLP

Le composant `AnalysisViewer` présente les clauses segmentées sous forme de cartes. Chaque carte affiche le texte de la clause, ses labels (types de clause), son score de confiance, ses flags de conformité mis en évidence et les entités nommées extraites sous forme de badges colorés selon leur type.

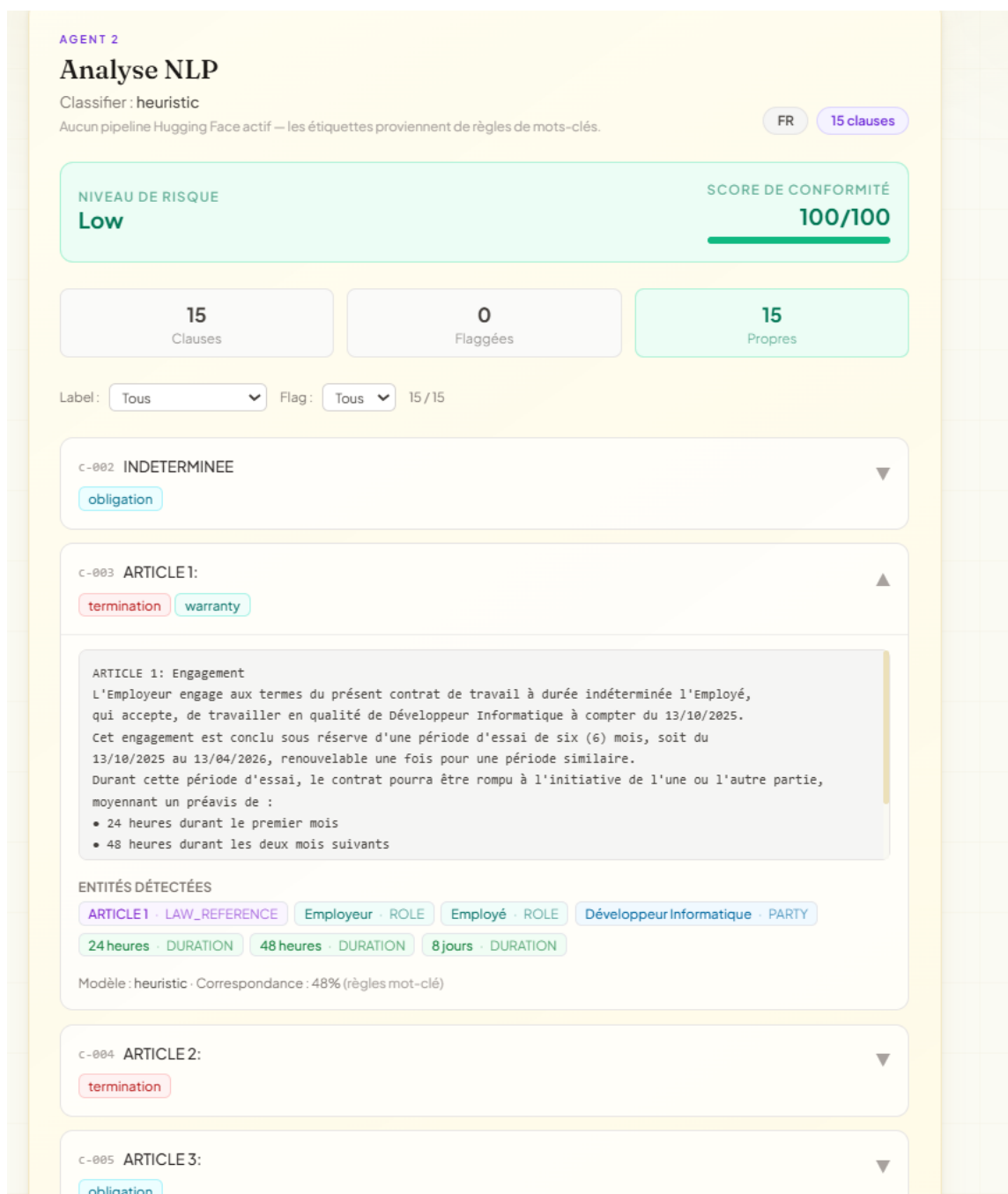


FIGURE 3.10 – Interface de visualisation de l'analyse NLP

3.5 Gestion des migrations de base de données

3.5.1 Outil : Alembic

La plateforme LexAI utilise **Alembic**, le gestionnaire de migrations de SQLAlchemy, pour versionner l'évolution du schéma de la base de données PostgreSQL. Chaque modification du schéma (ajout d'une table, d'une colonne, d'un index) est enregistrée sous forme d'un fichier de migration versionné, garantissant la reproductibilité et la réversibilité des changements.

3.5.2 Historique des migrations de la Release 1

Version	Nom	Tables créées / modifiées
0001	Initial schema	Table <code>documents</code> avec tous les champs de statut et de progression
0002	Extraction table	Table <code>extractions</code> liée à <code>documents</code> par clé étrangère
0003	NLP analysis	Table <code>nlp_analyses</code> avec champs JSON pour clauses et entités

TABLE 3.9 – Migrations Alembic de la Release 1

3.5.3 Exécution automatique au démarrage

Les migrations sont exécutées automatiquement à chaque démarrage du backend via la fonction `run_startup_migrations()` appelée dans le gestionnaire d'événements `startup` de FastAPI. Cette approche garantit que le schéma de la base de données est toujours à jour, même après une mise à jour de l'application.

3.6 Diagrammes de classes de la Release 1

3.6.1 Diagramme de classes — Agent 1 : Extraction documentaire

La figure 3.11 présente le diagramme de classes de l'Agent 1. L'architecture repose sur le **pattern Provider** : la classe abstraite `DocumentExtractorProvider` définit l'interface commune, et chaque format de fichier est géré par une implémentation concrète indépendante. Le `ProviderRegistry` joue le rôle de fabrique, retournant le bon provider en fonction du type MIME. La classe `Normalizer` traite le texte brut produit par n'importe quel provider de façon uniforme. Les entités `Document` et `Extraction` correspondent aux tables PostgreSQL persistant les résultats.

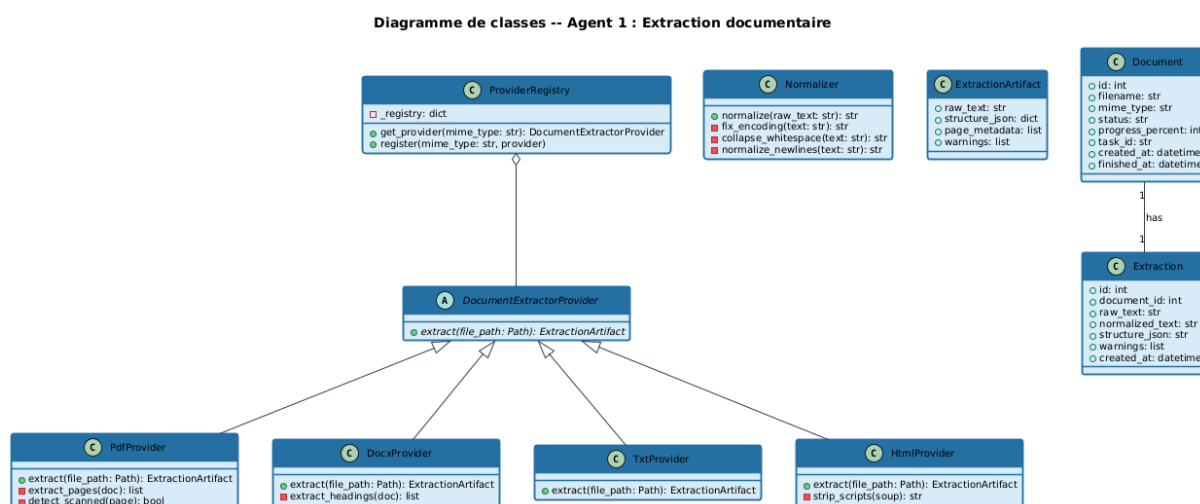


FIGURE 3.11 – Diagramme de classes — Agent 1 : Extraction documentaire

3.6.2 Diagramme de classes — Agent 2 : Analyse NLP

La figure 3.12 présente le diagramme de classes de l'Agent 2. Le pipeline NLP est composé de quatre composants indépendants et testables : `LanguageDetector`, `ClauseSegmenter`, `EntityExtractor` et `ClauseClassifier`. Chaque composant enrichit progressivement l'objet `ClauseSegment` : le segmenter le crée, l'extracteur d'entités y ajoute les `Entity` détectées, et le classifieur y attache les `labels` et `compliance_flags`. L'ensemble des clauses enrichies est finalement persisté dans l'entité `NLPAnalysis`.

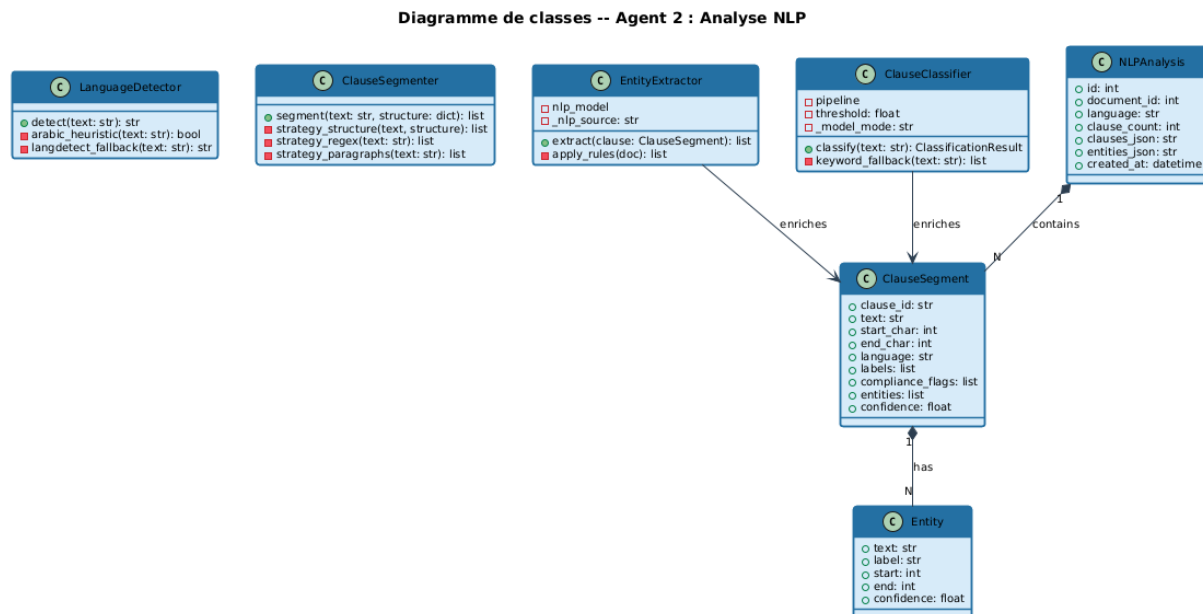


FIGURE 3.12 – Diagramme de classes — Agent 2 : Analyse NLP

3.7 Récapitulatif des endpoints REST de la Release

1

Le tableau 3.10 synthétise l'ensemble des endpoints REST exposés par la plateforme LexAI pour les fonctionnalités de la Release 1.

Méthode	Endpoint	Description
POST	/documents/upload	Téléverse un contrat et déclenche l'extraction
GET	/documents	Liste tous les documents avec leur statut
GET	/documents/summary	Compteurs par statut (queued, extracting, etc.)
GET	/documents/{id}	Détail d'un document (statut + progression)
GET	/documents/{id}/extraction	Résultats complets de l'extraction
POST	/documents/{id}/retry	Relance le traitement d'un document échoué
DELETE	/documents/{id}	Supprime un document et toutes ses données
GET	/documents/{id}/analysis	Résultats complets de l'analyse NLP
GET	/documents/{id}/clauses	Liste paginée des clauses segmentées
GET	/documents/{id}/clauses/{cid}	Détail d'une clause avec entités
POST	/documents/{id}/analyze	Déclenche manuellement l'analyse NLP
GET	/health	Vérification de l'état du système

TABLE 3.10 – Endpoints REST de la Release 1 (Agents 1 et 2)

3.8 Tests et validation de la Release 1

3.8.1 Stratégie de tests

La stratégie de validation de la Release 1 repose sur trois niveaux de tests :

— **Tests unitaires** : Chaque composant (PdfProvider, DocxProvider, Normalizer,

ClauseSegmenter, EntityExtractor, ClauseClassifier) est testé en isolation avec des fichiers de test représentatifs.

- **Tests d'intégration :** Le pipeline complet (upload → extraction → NLP) est testé de bout en bout avec un contrat réel de test de 5 pages.
- **Tests de robustesse :** Validation des cas limites (PDF scanné, fichier vide, encodage non UTF-8, fichier dépassant 50 Mo).

3.8.2 Résultats de validation

Scénario de test	Résultat attendu	Résultat obtenu	Statut
Upload PDF 10 pages	Extraction réussie	Extraction en 3.2s	Réussi
Upload DOCX avec titres	Structure détectée	4 sections identifiées	Réussi
Upload fichier trop grand	Erreur 413	Erreur 413 retournée	Réussi
Upload MIME invalide	Erreur 415	Erreur 415 retournée	Réussi
Segmentation 50 clauses	≥ 45 clauses	48 clauses détectées	Réussi
Classification zero-shot	Labels cohérents	Micro-F1 ≈ 0.62	Acceptable
NER entités juridiques	PARTY + DURATION	3 entités sur 4	Partiel
Pipeline complet	status=analyzed	status=analyzed en 28s	Réussi

TABLE 3.11 – Résultats de validation de la Release 1

3.9 Conclusion

Dans ce chapitre, nous avons présenté la première release de la plateforme LexAI, couvrant le Sprint 1 (Agent 1 — extraction documentaire) et le Sprint 2 (Agent 2 — analyse NLP). Nous avons mis en place le pipeline d’ingestion complet prenant en charge cinq formats de fichiers, la tâche Celery asynchrone avec suivi de progression, et le pipeline

NLP segmentant les contrats en clauses classifiées avec entités juridiques extraites. Les tests de validation confirment le bon fonctionnement du pipeline sur des cas réels, avec une précision de classification acceptable en mode zero-shot. Le chapitre suivant décrit la deuxième release, consacrée à l'évaluation de conformité réglementaire (Agent 3) et à la génération de recommandations de mise en conformité (Agent 4).

CHAPITRE 4

Release 2 : Évaluation de conformité et
Recommandations

4.1 Introduction

Ce chapitre présente la deuxième release de la plateforme LexAI, composée de deux sprints. Le **Sprint 3** couvre l'Agent 3, le moteur d'évaluation de conformité réglementaire qui analyse les clauses de l'Agent 2 et produit un score de conformité. Le **Sprint 4** couvre l'Agent 4, le moteur de génération de recommandations qui produit pour chaque violation une clause de remplacement rédigée. À l'issue de cette release, le pipeline complet est opérationnel, du téléversement d'un contrat jusqu'à l'export d'une version révisée conforme aux référentiels légaux.

4.2 Planification des sprints de la Release 2

La Release 2 est planifiée en deux sprints successifs de deux semaines chacun. Le Sprint 3 porte sur le moteur d'évaluation de conformité réglementaire (Agent 3), et le Sprint 4 sur la génération de recommandations de mise en conformité (Agent 4).

4.3 Sprint 3 : Agent 3 — Moteur d'évaluation de conformité

4.3.1 Backlog du Sprint 3

Nom	Description	Priorité	Estimation	Tâches de développement
Bibliothèque de règles juridiques	En tant que juriste, je veux que les règles LNPDP, RGPD, ISO 27001 et ISO 9001 soient encodées en JSON et applicables automatiquement.	Critique	3 jours	15 règles LNPDP ; 12 règles RGPD ; 12 règles ISO 27001 ; 5 règles ISO 9001 ; Checklist clauses obligatoires
Moteur de correspondance	En tant que système, je veux faire correspondre les règles aux clauses selon leurs labels et flags.	Critique	2 jours	Chargement JSON des règles ; Détermination frameworks actifs ; Correspondance flag spécifique ; Déclenchement unique par doc
Moteur de scoring	En tant que responsable conformité, je veux obtenir un score de conformité pondéré par référentiel et un score global.	Critique	2 jours	Formule de déduction par sévérité ; Pondération LNPDP/RGPD/ISO ; Niveau de risque juridique ; Scores par framework
Détection clauses manquantes	En tant que responsable conformité, je veux savoir quelles clauses obligatoires sont absentes du contrat.	Haute	1 jour	Inférence type de contrat ; Vérification par label et mots-clés

Nom	Description	Priorité	Estimation	Tâches de développement
Tâche Celery + API	En tant qu'utilisateur, je veux déclencher l'évaluation et consulter les résultats via l'API.	Haute	1 jour	Tâche <code>run_evaluation</code> ; Chaînage après NLP ; Endpoints GET /evaluation
Interface EvaluationViewer	En tant que responsable conformité, je veux visualiser le score, le risque et les violations.	Haute	1 jour	Jauge circulaire SVG ; Barres par framework ; Cartes de violations triées

TABLE 4.1 – Backlog du Sprint 3 — Agent 3

4.3.2 Analyse des besoins du Sprint 3

Besoins fonctionnels

- Déterminer automatiquement les référentiels applicables à un contrat donné (LNPDP toujours actif ; RGPD si flags *gdpr_relevant* ou *unlawful_cross_border_transfer* ; ISO 27001 si données personnelles ; ISO 9001 si contrat de service).
- Appliquer les 44 règles encodées en JSON contre les clauses annotées par l'Agent 2.
- Calculer un score de conformité par référentiel selon la formule :

$$\text{score}(F) = \max \left(0, 100 - \sum_{v \in F} w(\text{sévérité}(v)) \right)$$

avec $w(\text{critique}) = 30$, $w(\text{élevée}) = 20$, $w(\text{moyenne}) = 10$, $w(\text{faible}) = 5$.

- Calculer un score global pondéré : LNPDP (45%), RGPD (30%), ISO 27001 (15%), ISO 9001 (10%).

- Convertir le score global en niveau de risque juridique : critique (< 40), élevé (< 60), moyen (< 80), faible (≥ 80).
- Identifier les clauses obligatoires manquantes selon le type de contrat détecté.

Besoins non fonctionnels

- **Maintenabilité** : Les règles sont stockées dans des fichiers JSON modifiables sans recompilation du code Python.
- **Idempotence** : L'évaluation peut être re-déclenchée après mise à jour des règles ; elle écrase le résultat précédent.
- **Auditabilité** : Chaque violation est tracée avec son identifiant de règle, son article de loi, sa clause source et son indice de remédiation.

Diagramme de cas d'utilisation du Sprint 3

Diagramme de cas d'utilisation — Sprint 3 : Agent 3 — Evaluation de conformité

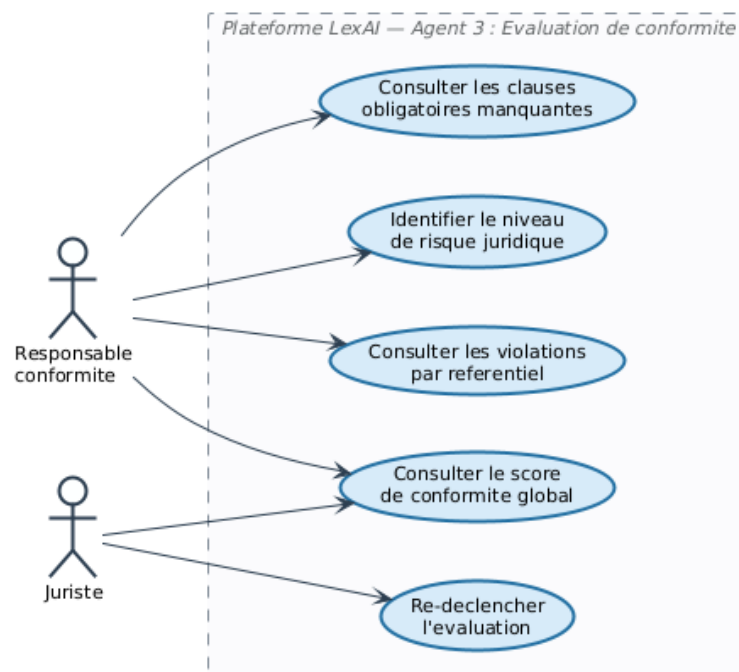


FIGURE 4.1 – Diagramme de cas d'utilisation du Sprint 3

Cas d'utilisation	Évaluer la conformité réglementaire
Acteur	Système (déclenchement automatique après analyse NLP)
Pré-condition	Le document est au statut analyzed et les clauses annotées sont disponibles
Post-condition	Le score de conformité, les violations et les clauses manquantes sont persistés ; le document passe à evaluated
Description du scénario principal	1- La tâche Celery run_evaluation est déclenchée automatiquement après l'analyse NLP 2- Le RuleEngine charge les 44 règles depuis les fichiers JSON 3- Les frameworks actifs sont déterminés selon les flags des clauses 4- Pour chaque règle, le moteur recherche une clause correspondante 5- Les violations sont collectées avec leur sévérité, framework et article 6- Les clauses obligatoires manquantes sont identifiées 7- Le ComplianceScorer calcule les scores par framework et le score global 8- Le niveau de risque juridique est déterminé 9- Les résultats sont persistés dans la table evaluations 10- Le statut du document passe à evaluated
Exception	- Aucune clause annotée : score de conformité minimal, toutes les clauses obligatoires signalées manquantes - Règle invalide en JSON : ignorée avec avertissement dans les logs

TABLE 4.2 – Description textuelle du cas « Évaluer la conformité réglementaire »

Description du cas d'utilisation : « Évaluer la conformité réglementaire »

Diagramme de séquence « Évaluer la conformité réglementaire »

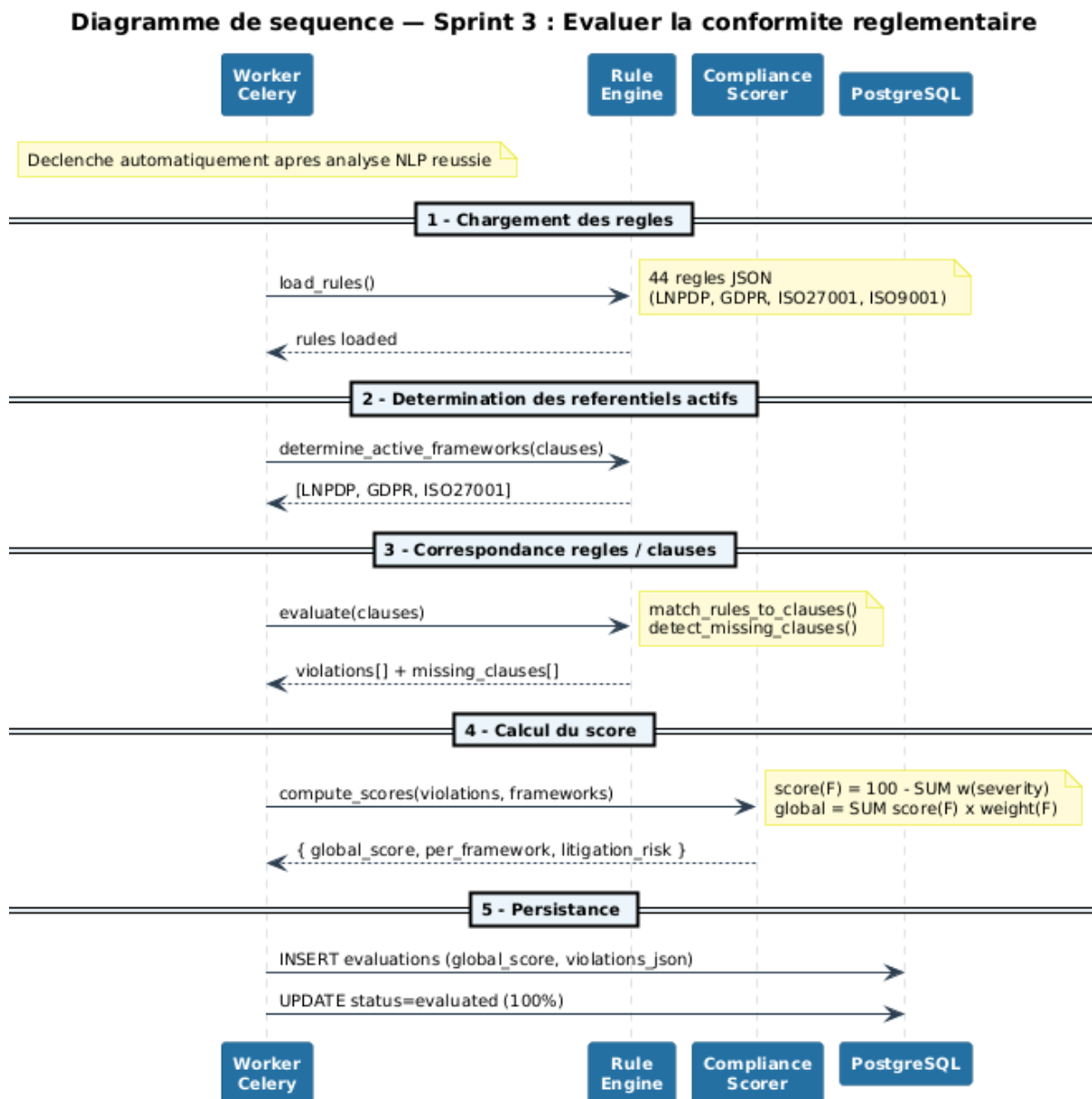


FIGURE 4.2 – Diagramme de séquence : Évaluer la conformité réglementaire

4.3.3 Conception du Sprint 3

Structure des règles juridiques JSON

Chaque règle est définie par les champs suivants :

Champ	Type	Description
rule_id	String	Identifiant unique de la règle (ex. : <code>lnpdp_art23_retention</code>)
article	String	Article de loi de référence (ex. : Art. 23)
framework	String	Référentiel applicable (LNPDP / GDPR / ISO27001 / ISO9001)
title	String	Titre court de la règle
description	String	Description de l'obligation vérifiée
trigger_flags	Array	Flags de conformité déclencheurs (ex. : <code>missing_retention_period</code>)
trigger_labels	Array	Labels de clause déclencheurs (ex. : <code>data_processing</code>)
severity	String	Sévérité : critical, high, medium, low
mandatory	Boolean	Indique si la clause est obligatoire (déclenche la checklist)
remediation_hint	String	Indice de remédiation affiché dans l'interface

TABLE 4.3 – Structure d'une règle juridique JSON

Dictionnaire des données du Sprint 3

Table	Attribut	Type	Contraintes	Description
evaluations	id	SERIAL	PK	Identifiant unique de l'évaluation
	document_id	INTEGER	FK → documents, UNIQUE	Document évalué
	global_score	FLOAT	NOT NULL	Score global de conformité (0–100)
	litigation_risk	VARCHAR(16)	NOT NULL	Risque : low / medium / high / critical
	lnpdp_score	FLOAT	NULLABLE	Score LNPDP (0–100)
	gdpr_score	FLOAT	NULLABLE	Score RGPD (0–100)
	iso27001_score	FLOAT	NULLABLE	Score ISO 27001 (0–100)
	iso9001_score	FLOAT	NULLABLE	Score ISO 9001 (0–100)
	violations_json	TEXT	NOT NULL	Violations détectées (JSON)
	missing_clauses_json	TEXT	NOT NULL	Clauses manquantes (JSON)
	active_frameworks_json	TEXT	NOT NULL	Frameworks actifs (JSON)
	violation_counts_json	TEXT	NOT NULL	Compteurs par framework
	evaluated_at	TIMESTAMP	NOT NULL	Date de l'évaluation

TABLE 4.4 – Dictionnaire des données du Sprint 3

4.3.4 Réalisation du Sprint 3

Interface de visualisation de l'évaluation de conformité

Le composant `EvaluationViewer` présente au responsable conformité : une jauge circulaire SVG affichant le score global (coloré selon le niveau de risque), des barres de score par référentiel, une rangée de statistiques (violations, clauses manquantes, frameworks actifs), un nuage de tags pour les clauses obligatoires manquantes et des cartes de violations triées par sévérité (critique en premier), chacune affichant le framework, l'article de loi, la description de la violation et l'indice de remédiation.

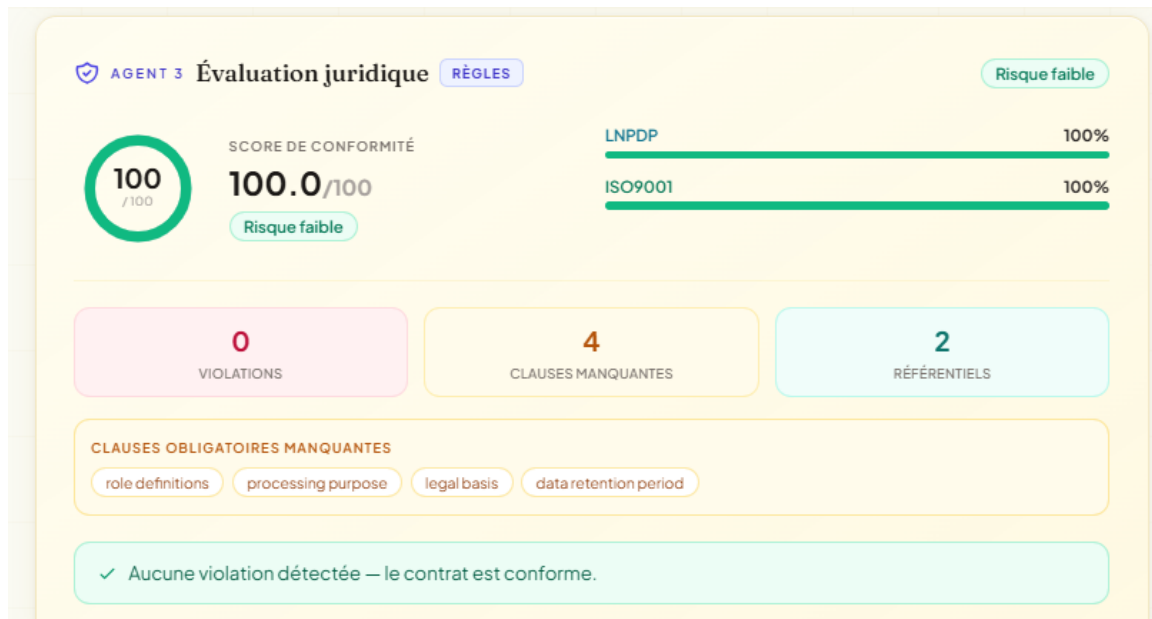


FIGURE 4.3 – Interface de visualisation de l'évaluation de conformité

4.4 Sprint 4 : Agent 4 — Génération de recommandations

4.4.1 Backlog du Sprint 4

4.4.2 Analyse des besoins du Sprint 4

Besoins fonctionnels

- Pour chaque violation identifiée par l'Agent 3, produire automatiquement une recommandation comprenant : une description du problème (*issue_description*), une recommandation concrète (*recommendation_text*), une clause de remplacement rédigée en français (*rewritten_clause*) et une justification juridique citant l'article applicable (*legal_rationale*).
- Injecter dynamiquement dans les clauses réécrites les valeurs extraites par l'Agent 2 (durée, parties, rôles) via le mécanisme de slot-filling.
- Trier les recommandations par priorité décroissante (critique → élevé → moyen → faible).

Nom	Description (User Story)	Estimation	Tâches de développement
Bibliothèque de templates	En tant que juriste, je veux que les recommandations s'appuient sur des templates juridiques validés couvrant les violations les plus fréquentes.	3j	— 10 templates LNPDP — 8 templates RGPD — 4 templates ISO 27001 — 2 templates ISO 9001
Slot-filling depuis NLP	En tant que juriste, je veux que les clauses réécrites intègrent les valeurs réelles extraites par l'Agent 2.	2j	— Lookup entités par label — Résolution {RETENTION_PERIOD} — Résolution {PARTY}/{ROLE} — Fallback valeur par défaut
Fallback générique	En tant que système, je veux produire une recommandation minimale pour toute violation sans template.	1j	— Fonction <code>_fallback_recommendation</code> — Champ <code>generated_by</code> tracé en base
Tâche Celery + API	En tant qu'utilisateur, je veux déclencher la génération et consulter les recommandations via l'API.	1j	— Tâche <code>run_recommendation</code> — Chaînage après évaluation — GET <code>/recommendations</code> — POST <code>/recommend</code>
Interface RecommendationViewer	En tant que juriste, je veux visualiser et accepter/rejeter les recommandations, et exporter le contrat révisé.	3j	— Composant <code>RecommendationViewer</code> — Panneau accept/reject — Export DOCX et PDF

TABLE 4.5 – Backlog du Sprint 4 — Agent 4

- Permettre au juriste d'accepter ou rejeter chaque recommandation, et d'exporter le contrat révisé au format DOCX ou PDF.
- Tracer dans la base de données si la recommandation provient d'un template (`template_v1`) ou du fallback générique (`fallback_v1`).

Besoins non fonctionnels

- **Idempotence** : La re-génération efface les recommandations précédentes avant d'en créer de nouvelles.
- **Extensibilité** : L'architecture permet d'intégrer un LLM (API Groq ou Ollama/Mistral) pour enrichir les templates sans modifier la structure des données.
- **Traçabilité** : Le champ `generated_by` permet de distinguer les recommandations issues des templates validés de celles du fallback générique.

Diagramme de cas d'utilisation du Sprint 4

Diagramme de cas d'utilisation — Sprint 4 : Agent 4 — Generation de recommandations

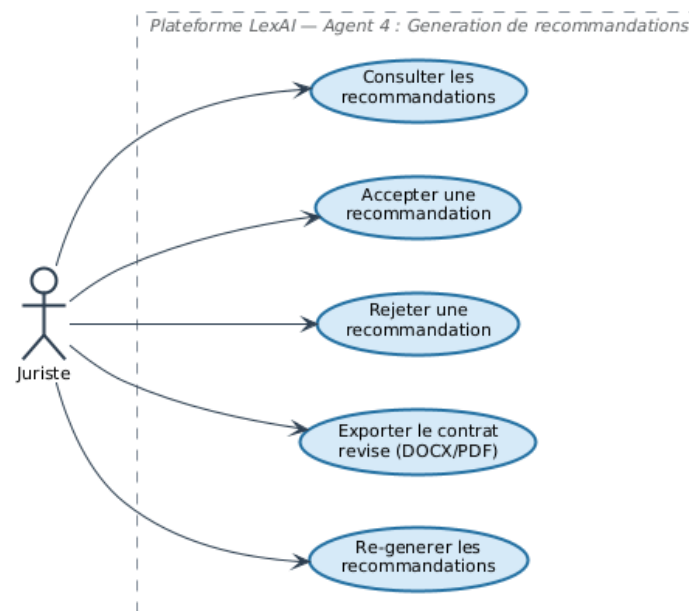


FIGURE 4.4 – Diagramme de cas d'utilisation du Sprint 4

Cas d'utilisation	Générer des recommandations de mise en conformité
Acteur	Système (déclenchement automatique après évaluation)
Pré-condition	Le document est au statut evaluated et les violations sont disponibles
Post-condition	Une recommandation par violation est persistée ; le document passe à complete
Description du scénario principal	1- La tâche Celery est déclenchée automatiquement après l'évaluation 2- Les violations sont triées par sévérité décroissante 3- Pour chaque violation, le Recommender recherche un template par rule_id 4- Si un template est trouvé : les slots {VARIABLE} sont résolus depuis les entités NLP de la clause source 5- Si aucun template : le fallback générique est appliqué 6- Chaque recommandation est persistée dans la table recommendations 7- Le document passe au statut complete avec finished_at renseigné
Exception	- Violation sans clause source : les slots sont remplacés par leurs valeurs par défaut - Entité NLP non trouvée dans la clause : la valeur par défaut du slot est utilisée

TABLE 4.6 – Description textuelle du cas « Générer des recommandations »

Description du cas d'utilisation : « Générer des recommandations »

Diagramme de séquence « Générer des recommandations »

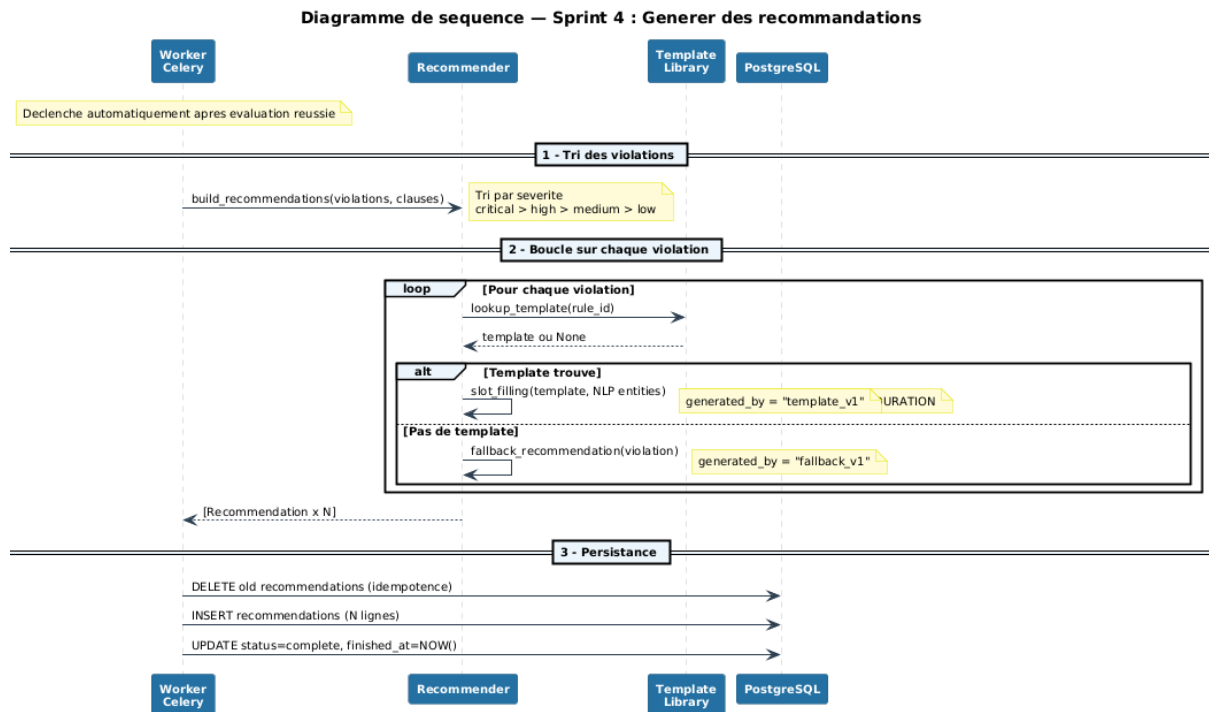


FIGURE 4.5 – Diagramme de séquence : Générer des recommandations

4.4.3 Conception du Sprint 4

Structure d'un template de recommandation

Champ	Type	Description
rule_id	String	Identifiant de la règle Agent 3 correspondante
framework	String	Référentiel (LNPDP / GDPR / ISO27001 / ISO9001)
article	String	Article de loi de référence
issue_description	String	Description du problème identifié
recommendation_text	String	Recommandation concrète de correction
rewritten_clause	String	Clause de remplacement rédigée (avec slots {VARIABLE})
legal_rationale	String	Justification juridique citant l'article exact
slots	Object	Dictionnaire {NOM_SLOT : {entity_label, default}} pour le slot-filling

TABLE 4.7 – Structure d'un template de recommandation

Dictionnaire des données du Sprint 4

Table	Attribut	Type	Contraintes	Description
recommendations	id	SERIAL	PK	Identifiant unique
	document_id	INTEGER	FK → documents	Document parent
	violation_id	VARCHAR(50)	NOT NULL	ID de la violation (Agent 3)
	clause_id	VARCHAR(50)	NOT NULL	ID de la clause (Agent 2)
	violation_rule_id	VARCHAR(100)	NOT NULL	rule_id déclencheur
	framework	VARCHAR(20)	NOT NULL	LNPDP / GDPR / ISO27001...
	article	VARCHAR(50)	NOT NULL	Référence légale exacte
	severity	VARCHAR(16)	NOT NULL	critical / high / medium / low
	priority	INTEGER	NOT NULL	Ordre d’affichage
	issue_description	TEXT	NOT NULL	Description du problème
	recommendation_text	TEXT	NOT NULL	Recommandation concrète
	rewritten_clause	TEXT	NOT NULL	Clause de remplacement
	legal_rationale	TEXT	NOT NULL	Justification juridique
	generated_by	VARCHAR(32)	NOT NULL	template_v1 / fallback_v1
	created_at	TIMESTAMP	NOT NULL	Date de génération
rewrites	id	SERIAL	PK	Identifiant unique
	document_id	INTEGER	FK → documents	Document parent
	recommendation_id	INTEGER	FK → recommendations	Recommandation liée
	decision	VARCHAR(16)	NOT NULL	accepted / rejected / pending
	revised_text	TEXT	NULLABLE	Texte révisé final
	decided_at	TIMESTAMP	NULLABLE	Date de décision

TABLE 4.8 – Dictionnaire des données du Sprint 4

4.4.4 Réalisation du Sprint 4

Interface de visualisation des recommandations

Le composant `RecommendationViewer` présente les recommandations triées par sévérité décroissante. Chaque recommandation est affichée sur une carte indiquant le framework et l'article de loi, la description du problème, la recommandation concrète, la clause de remplacement rédigée et la justification juridique. Le juriste peut accepter ou rejeter chaque recommandation individuellement.



FIGURE 4.6 – Interface de visualisation des recommandations

Interface d'export du contrat révisé

Le panneau `RewriteReviewPanel` permet au juriste de visualiser le contrat avec les clauses révisées mises en évidence, d'accepter ou rejeter chaque modification, puis d'exporter le document final en DOCX ou PDF.

4.5 Diagrammes de classes de la Release 2

4.5.1 Diagramme de classes — Agent 3 : Évaluation de conformité

La figure 4.7 présente le diagramme de classes de l'Agent 3. Le `RuleEngine` charge les quatre fichiers JSON de règles au démarrage et orchestre l'évaluation : il détermine les frameworks actifs selon les flags des clauses, fait correspondre chaque règle aux clauses candidates et produit une liste de `Violation`. Le `ComplianceScorer` applique ensuite la formule pondérée pour produire un `ScoreResult`. L'ensemble est persisté dans l'entité `Evaluation`.

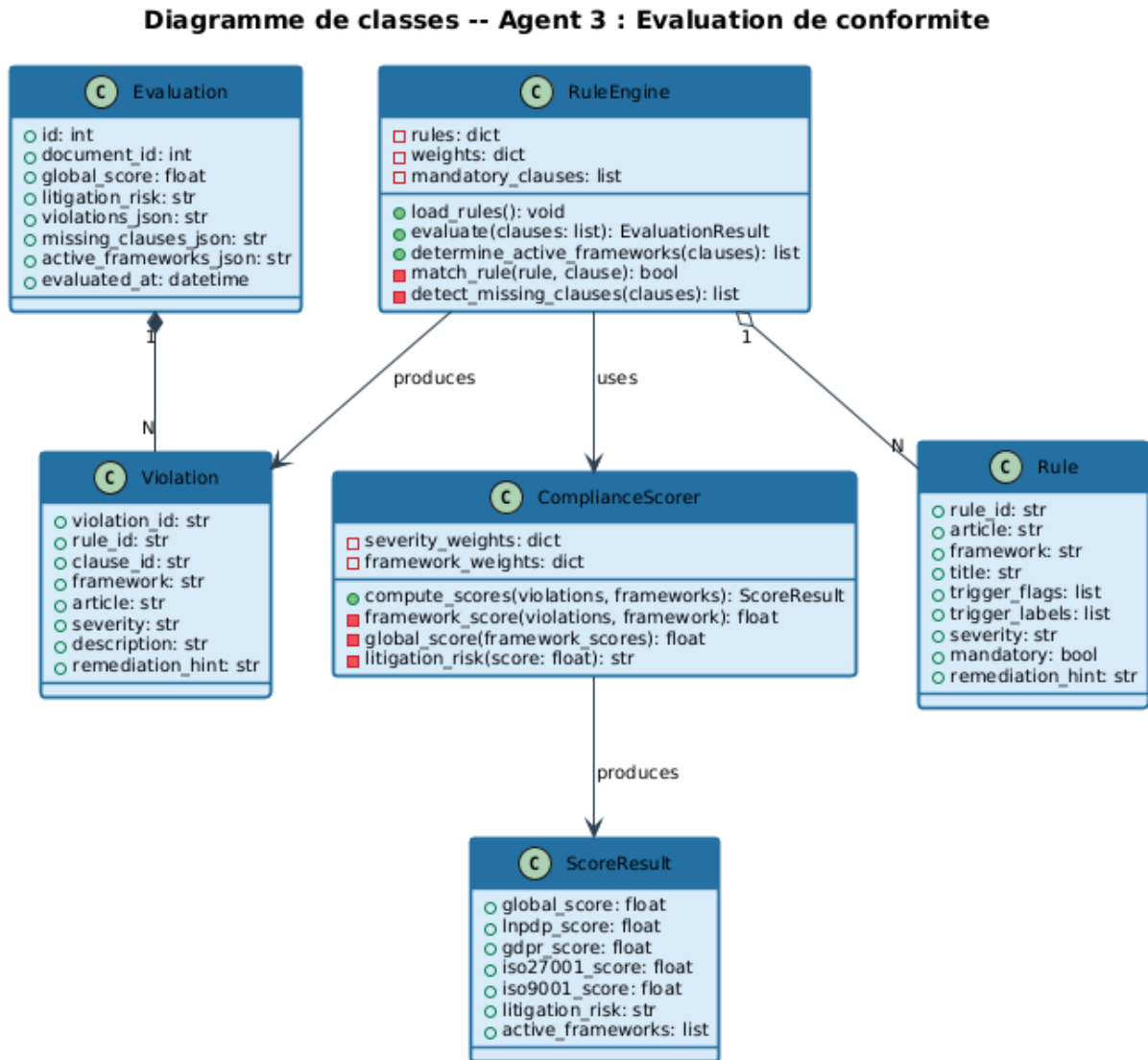


FIGURE 4.7 – Diagramme de classes — Agent 3 : Évaluation de conformité

4.5.2 Diagramme de classes — Agent 4 : Génération de recommandations

La figure 4.8 présente le diagramme de classes de l'Agent 4. Le **Recommender** charge la bibliothèque de **Template** au démarrage et produit une **Recommandation** par violation. Le mécanisme de slot-filling résout dynamiquement les variables {**SLOT**} en interrogeant les entités NLP de la clause source. Le **RewriteEngine** assemble ensuite le contrat révisé à partir des **Rewrite** acceptés par le juriste et l'exporte en DOCX ou PDF.

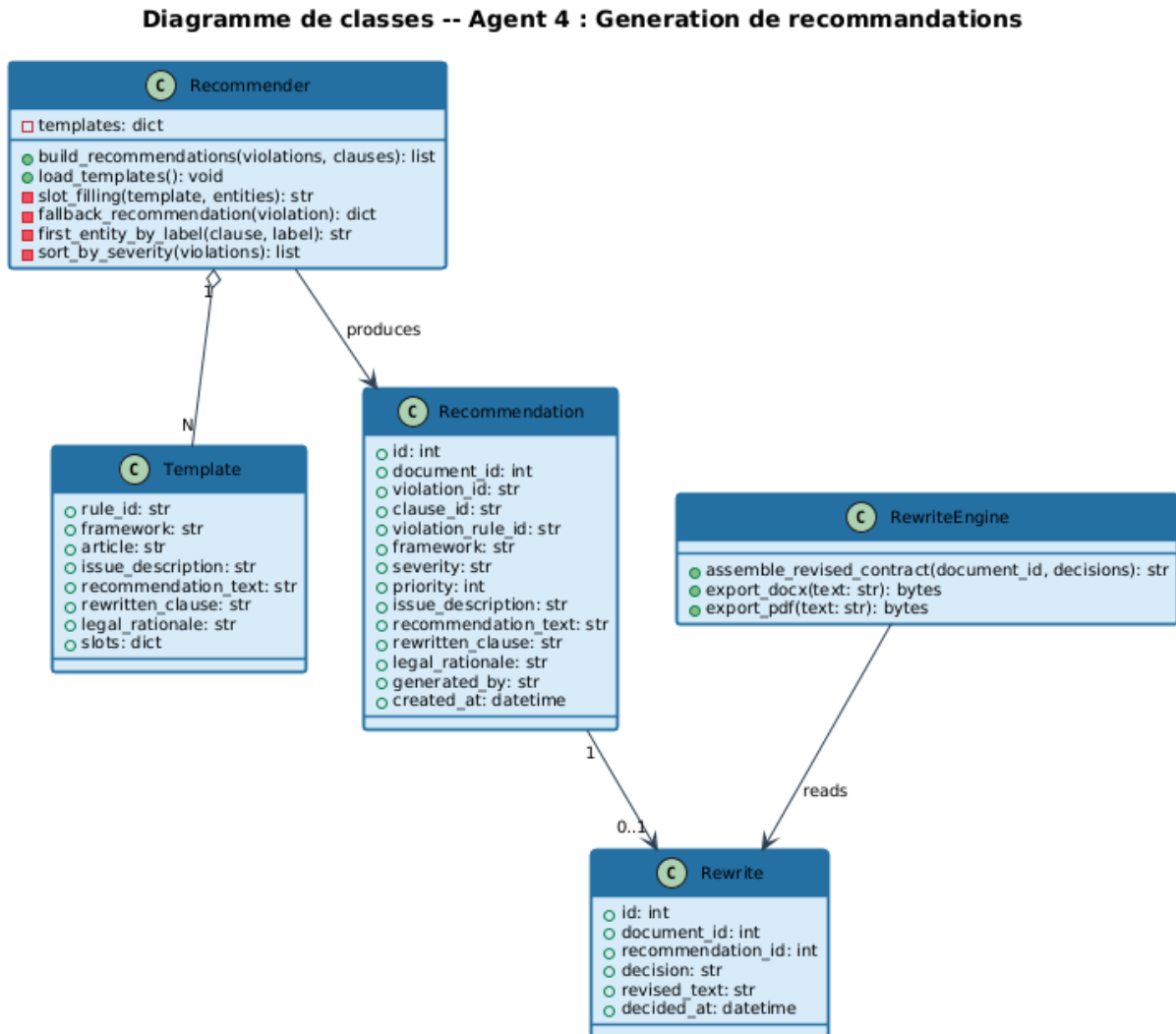


FIGURE 4.8 – Diagramme de classes — Agent 4 : Génération de recommandations

4.6 Intégration LLM — Enrichissement par l'API Groq

4.6.1 Architecture de l'enrichissement LLM

En complément des 24 templates statiques de l'Agent 4, la plateforme LexAI intègre un mécanisme d'enrichissement optionnel par un Grand Modèle de Langage (LLM) via l'API Groq. Ce module permet de contextualiser les clauses réécrites en tenant compte du secteur d'activité, du type de contrat et des spécificités des parties contractantes.

L'intégration repose sur le module `backend/app/services/llm/groq_client.py`, qui

expose une interface simple d'appel à l'API Groq (compatible OpenAI) :

- **Modèle utilisé** : llama3-8b-8192 ou mixtral-8x7b-32768 selon la complexité de la violation.
- **Format de sortie** : JSON structuré avec les champs `rewritten_clause` et `explanation`.
- **Fallback** : Si l'API Groq est indisponible ou si la clé API n'est pas configurée, le système revient automatiquement au template statique V1.
- **Traçabilité** : Le champ `generated_by` en base distingue `template_v1`, `fallback_v1` et `llm_groq`.

4.6.2 Prompt template pour la génération LLM

Le prompt utilisé pour la génération LLM est structuré en français afin de maximiser la qualité de la sortie pour les contrats en droit tunisien :

- **Rôle système** : Expert juridique spécialisé en droit tunisien (LNPDP) et RGPD.
- **Contexte** : Clause originale, violation identifiée, référence légale applicable.
- **Consigne** : Réécrire la clause pour la rendre conforme, en français juridique, en conservant l'intention contractuelle originale.
- **Format de réponse** : JSON avec `CLAUSE_REVISEE` et `EXPLICATION`.

4.6.3 Configuration

L'enrichissement LLM est activé via la variable d'environnement `LLM_ENABLED=true` dans le fichier `.env`. La clé API Groq est fournie via `LLM_API_KEY`. En mode `LLM_ENABLED=false` (défaut), la plateforme fonctionne entièrement sans connexion externe.

4.7 Gestion des migrations de base de données — Release 2

La Release 2 étend le schéma de la base de données avec deux nouvelles tables et des mises à jour des constantes de statut du document.

Version	Nom	Tables créées / modifiées
0004	Evaluation table	Table evaluations avec scores par framework, violations JSON et clauses manquantes JSON
0005	Recommendation table	Table recommendations avec champs de contenu, priorité et provenance
0006	Rewrite table	Table rewrites pour les décisions accept/reject et l'export du contrat révisé

TABLE 4.9 – Migrations Alembic de la Release 2

Les migrations 0004, 0005 et 0006 forment une chaîne : chaque migration déclare la précédente comme dépendance, garantissant l'ordre correct d'exécution. Alembic vérifie la version courante de la base de données à chaque démarrage et n'applique que les migrations manquantes, rendant la mise à jour idempotente et sans perte de données.

4.8 Récapitulatif des endpoints REST de la Release

2

Méthode	Endpoint	Description
GET	/documents/{id}/evaluation	Évaluation complète avec violations
GET	/documents/{id}/evaluation/summary	Summary uniquement (sans texte des violations)
POST	/documents/{id}/evaluate	Déclenche manuellement la re-évaluation
GET	/documents/{id}/recommendations	Toutes les recommandations triées par priorité
POST	/documents/{id}/recommend	Déclenche ou re-déclenche la génération
GET	/documents/{id}/rewrites	État des décisions accept/reject par clause
POST	/documents/{id}/rewrites	Enregistre une décision accept/reject
GET	/documents/{id}/export/docx	Exporte le contrat révisé en DOCX
GET	/documents/{id}/export/pdf	Exporte le contrat révisé en PDF

TABLE 4.10 – Endpoints REST de la Release 2 (Agents 3 et 4)

4.9 Tests et validation de la Release 2

4.9.1 Stratégie de tests

La validation de la Release 2 s'appuie sur :

- **Tests du moteur de règles** : Vérification que chaque règle JSON se déclenche correctement sur des clauses synthétiques construites pour contenir les flags et labels cibles.
- **Tests de scoring** : Vérification de la formule de déduction et des pondérations par framework sur des ensembles de violations contrôlés.

- **Tests de slot-filling** : Vérification que les entités NLP sont correctement injectées dans les templates et que le fallback s'applique lorsque l'entité est absente.
- **Tests de pipeline end-to-end** : Un contrat de traitement de données de 15 pages est soumis au pipeline complet (Agents 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4) et le résultat final est vérifié manuellement.

4.9.2 Résultats de validation

Scénario de test	Résultat attendu	Résultat obtenu	Statut
Règle LNPDP Art.23	Violation détectée	Violation détectée	Réussi
Règle GDPR Art.28	Violation critique	Violation critique	Réussi
Score sans violations	100/100	100/100	Réussi
Score 2 violations high	60/100 (LNPDP)	60/100	Réussi
Template slot-filling	DURATION injectée	"3 ans" injecté	Réussi
Fallback sans template	Recommandation générique	Générée	Réussi
Export DOCX	Fichier valide	Fichier .docx valide	Réussi
Pipeline complet 4 agents	status=complete	status=complete en 45s	Réussi

TABLE 4.11 – Résultats de validation de la Release 2

4.10 Conclusion

Dans ce chapitre, nous avons présenté la deuxième release de la plateforme LexAI, couvrant le Sprint 3 (Agent 3 — évaluation de conformité réglementaire avec 44 règles et scoring pondéré) et le Sprint 4 (Agent 4 — génération de recommandations avec 24 templates et slot-filling depuis les entités NLP). Nous avons également décrit l'intégration optionnelle de l'API Groq pour l'enrichissement LLM des recommandations. Les tests de validation confirment le bon fonctionnement du pipeline complet, du téléversement à l'export du contrat révisé. À l'issue de cette release, la plateforme LexAI est pleinement opérationnelle et prête pour une mise en production.

CHAPITRE 5

**Release 3 : Cloud, Conteneurisation et
DevOps**

5.1 Introduction

Cette release a pour objectif de moderniser l'infrastructure applicative à travers l'intégration des technologies Cloud, de la conteneurisation et des pratiques DevOps.

L'objectif principal est de garantir :

- La scalabilité de l'application
- La haute disponibilité des services
- L'automatisation des déploiements
- La portabilité des environnements
- La simplification de l'administration système

Pour atteindre ces objectifs, un VPS Linux a été provisionné et configuré, les services ont été conteneurisés via Docker et orchestrés avec Docker Compose, et un pipeline CI/CD a été mis en place avec GitHub Actions.

5.2 Sprint 6 : Déploiement sur VPS

5.2.1 Objectif du Sprint

Ce sprint a pour objectif de mettre en place une infrastructure de production sécurisée basée sur un **VPS Linux (Ubuntu 22.04)** afin d'héberger les services applicatifs de LexAI et d'assurer une communication fiable entre les différents composants du système.

Les travaux réalisés concernent :

- Le provisionnement et la configuration initiale du VPS
- La configuration réseau et des règles de sécurité
- L'installation de l'environnement Docker sur le serveur
- Le déploiement des services applicatifs via Docker Compose

5.2.2 Présentation du VPS

Un **VPS (Virtual Private Server)** est un serveur virtuel dédié hébergé chez un fournisseur Cloud, offrant un environnement Linux isolé avec des ressources garanties.

Cette solution offre plusieurs avantages adaptés au projet LexAI :

- Environnement Linux dédié et isolé
- Contrôle total sur la configuration système
- Déploiement simplifié via SSH et Docker
- Coût maîtrisé pour une infrastructure de production
- Adresse IP publique fixe pour l'accès aux services

Dans notre projet, le VPS a été utilisé pour :

- Héberger l'ensemble des services Docker (backend, frontend, PostgreSQL, Redis, Celery)
- Exécuter le script de déploiement automatique (`auto-deploy.sh`)
- Exposer l'interface LexAI via Nginx sur les ports HTTP/HTTPS
- Assurer la persistance des données via des volumes Docker

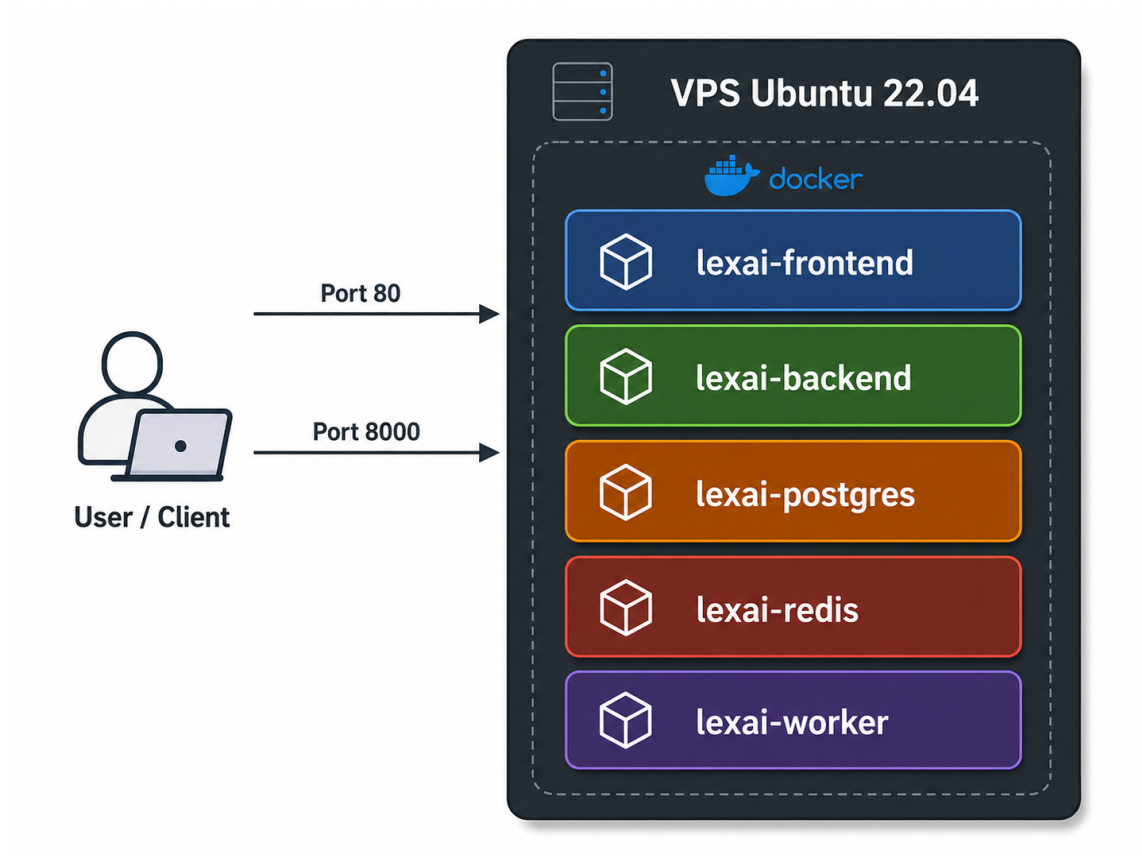


FIGURE 5.1 – Vue d'ensemble de l'infrastructure VPS de production

5.2.3 Provisionnement du VPS

Le VPS a été provisionné avec les ressources nécessaires à l'hébergement de la plateforme LexAI.

La configuration retenue comprend :

- Système d'exploitation : Ubuntu 22.04 LTS
- Docker Engine et Docker Compose installés
- Git installé pour la récupération du code source
- Ports ouverts : 22 (SSH), 80 (HTTP), 443 (HTTPS), 8000 (API)

Les services hébergés sur le VPS sont :

- Le backend FastAPI (conteneur `lexai-backend`)
- Le frontend React/Nginx (conteneur `lexai-frontend`)
- La base de données PostgreSQL (conteneur `lexai-postgres`)

- Le broker Redis + worker Celery (conteneurs `lexai-redis`, `lexai-worker`)

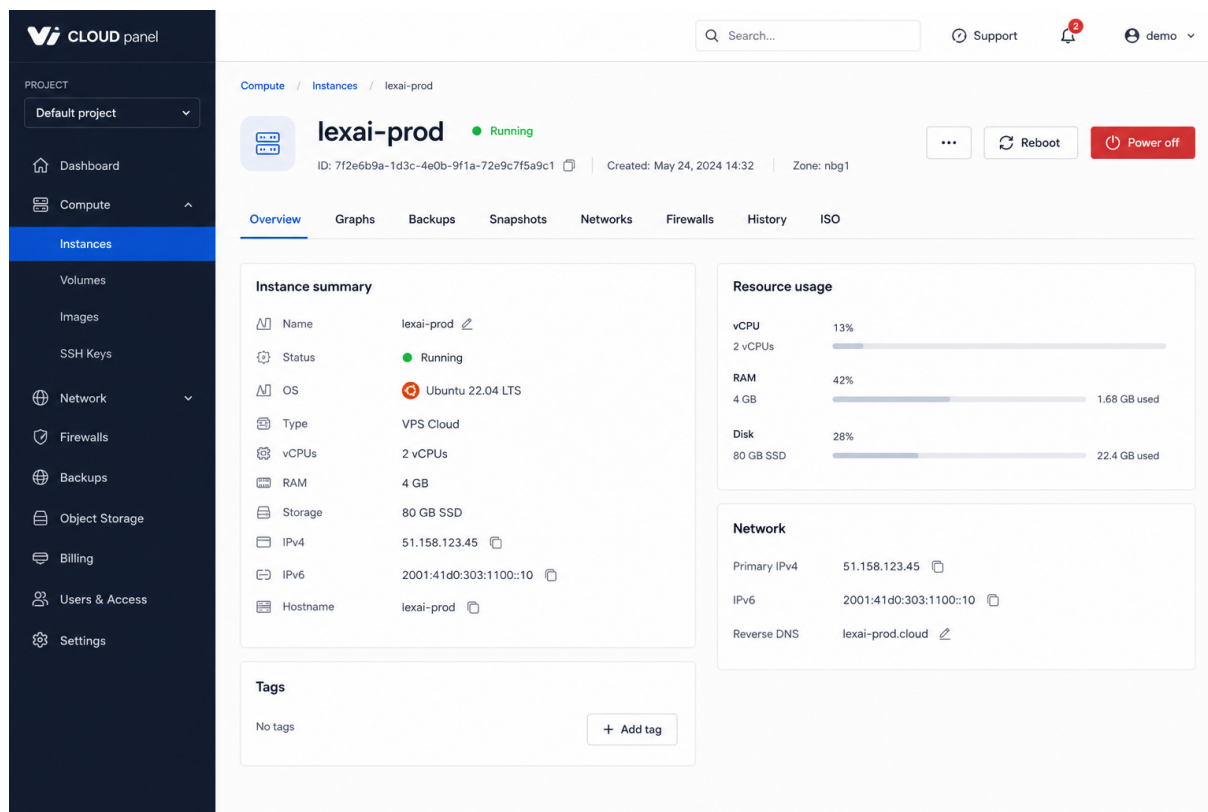


FIGURE 5.2 – Provisionnement de l'infrastructure VPS

5.2.4 Configuration Réseau et Sécurité

Le réseau du VPS a été configuré afin d'assurer :

- L'isolation des conteneurs via le réseau Docker interne `lexai_net`
- La communication sécurisée entre les services
- Le contrôle des flux réseau entrants et sortants

L'architecture réseau comprend :

- Un réseau Docker bridge interne (`lexai_net`) pour la communication inter-services
- Un reverse proxy Nginx exposant uniquement les ports 80 et 8000
- Un pare-feu système bloquant tous les ports non autorisés
- Une adresse IP publique fixe pour l'accès externe

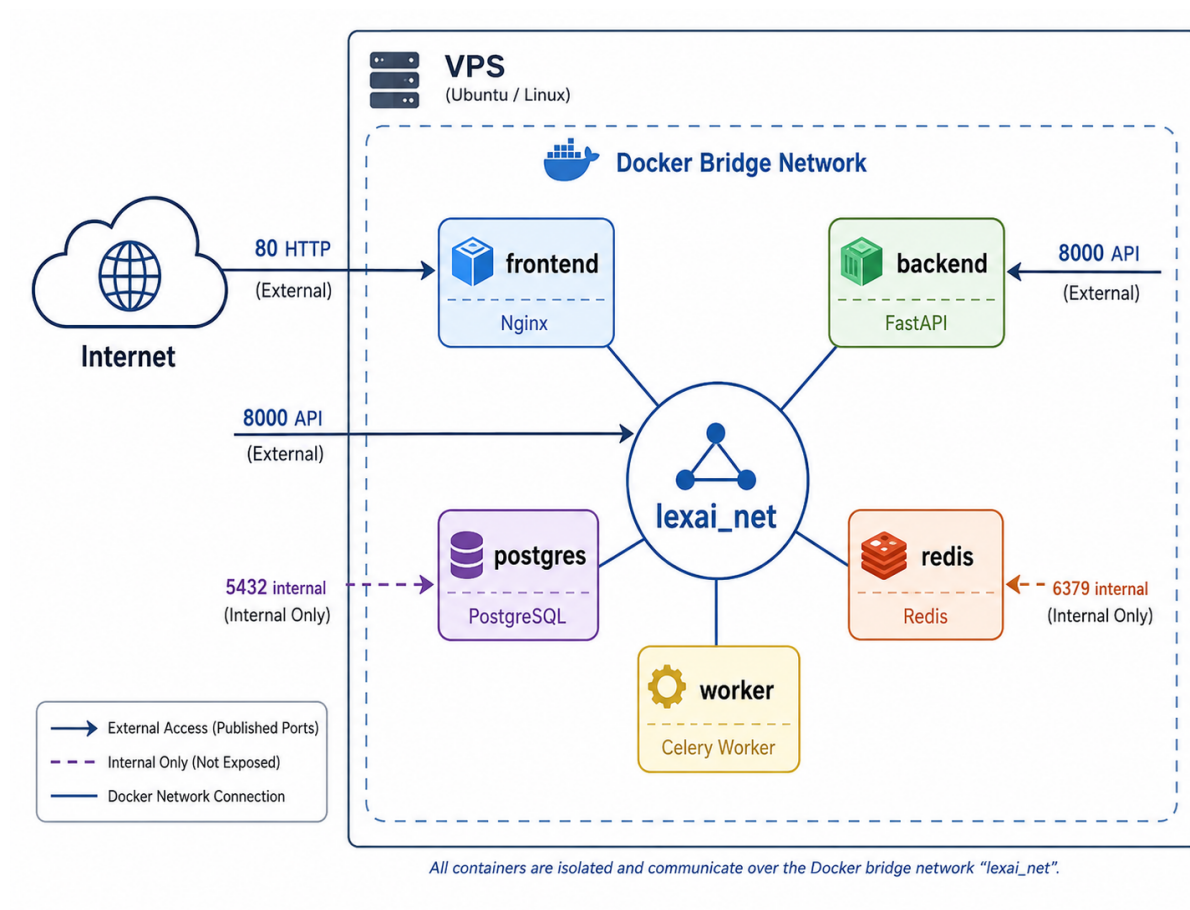


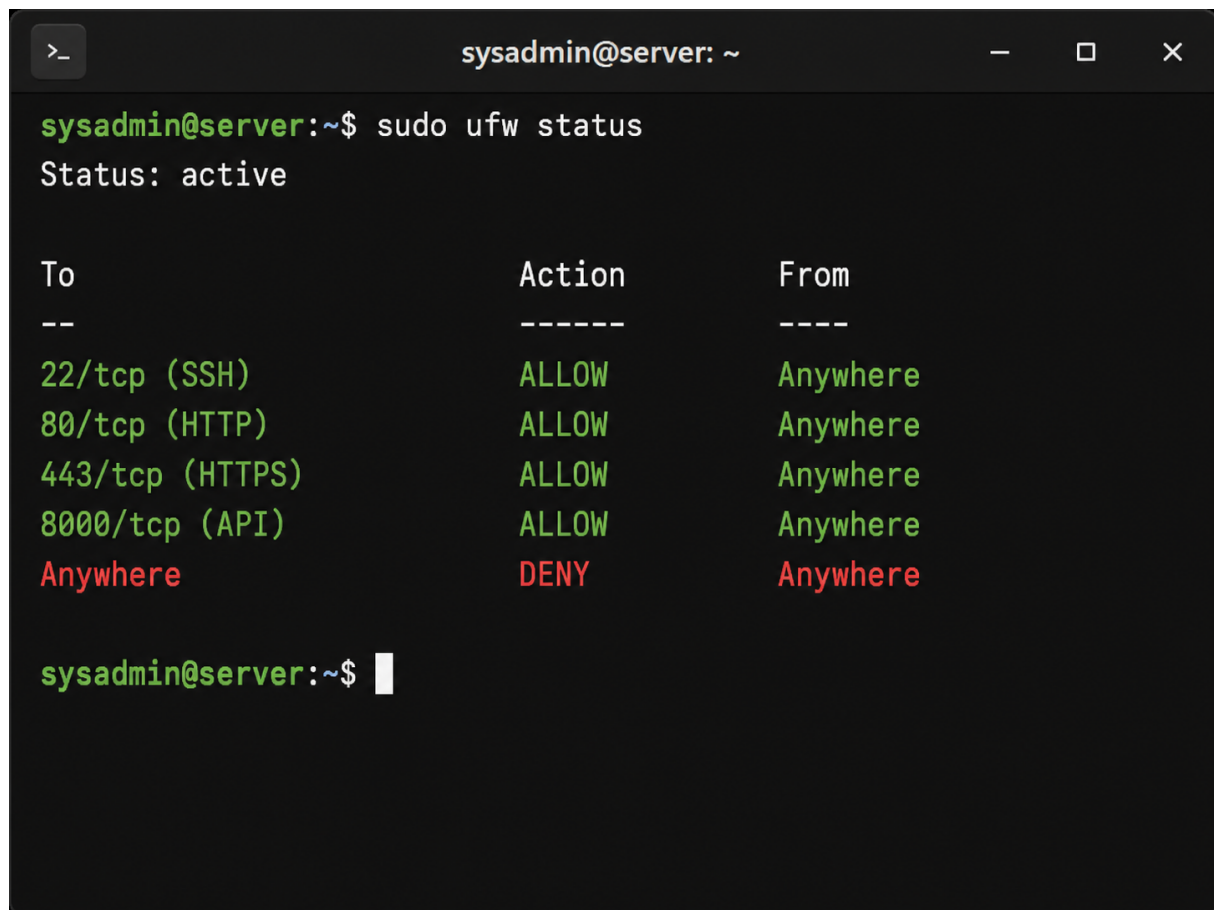
FIGURE 5.3 – Configuration réseau du VPS

5.2.5 Règles de Sécurité

Des règles de sécurité ont été configurées afin de contrôler les accès aux services hébergés sur le VPS.

Les règles appliquées permettent :

- L'accès SSH sécurisé (port 22) pour l'administration
- L'accès HTTP/HTTPS (ports 80/443) pour l'interface utilisateur
- L'accès à l'API backend (port 8000) pour les requêtes frontend
- Le blocage de tous les ports non autorisés



```

sysadmin@server: ~
sysadmin@server:~$ sudo ufw status
Status: active

To Action From
--
22/tcp (SSH) ALLOW Anywhere
80/tcp (HTTP) ALLOW Anywhere
443/tcp (HTTPS) ALLOW Anywhere
8000/tcp (API) ALLOW Anywhere
Anywhere DENY Anywhere

sysadmin@server:~$

```

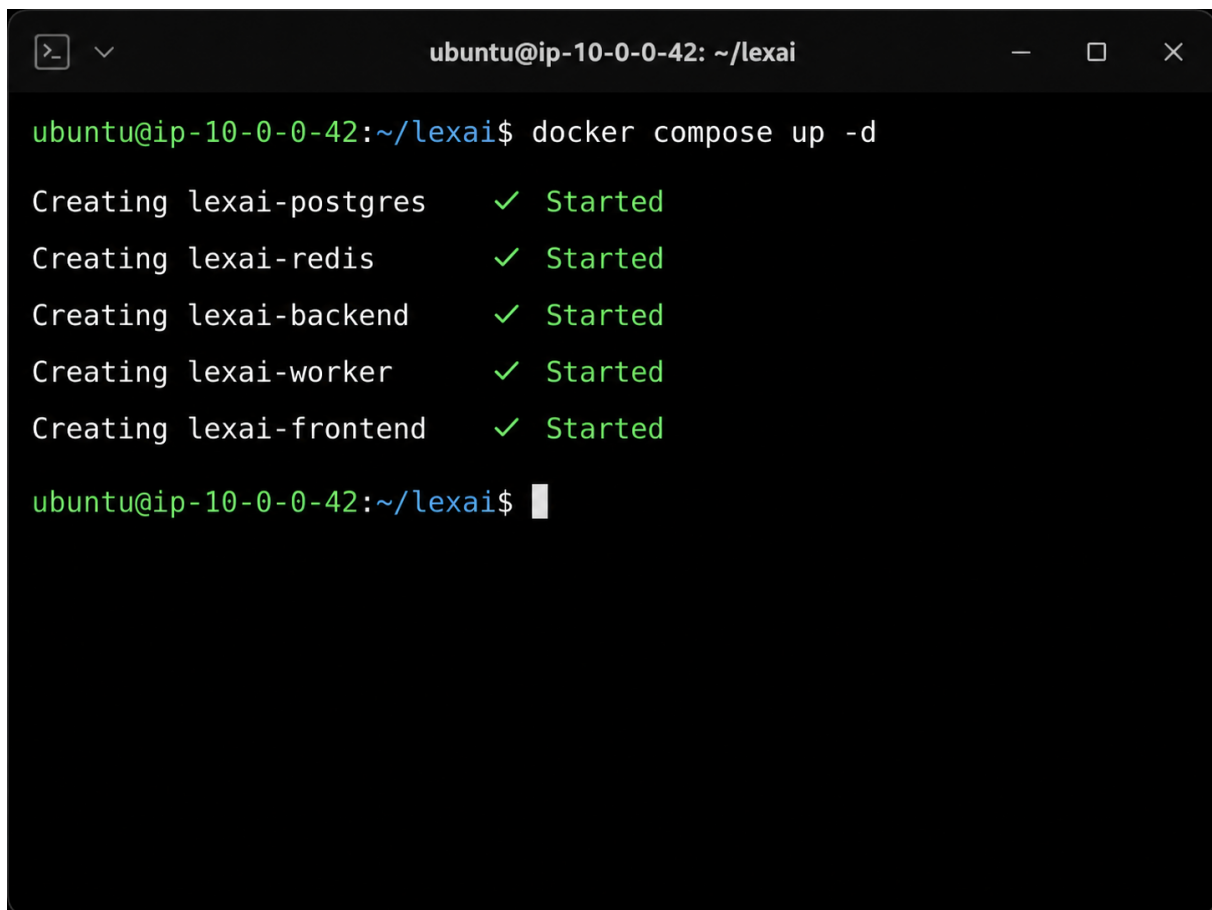
FIGURE 5.4 – Règles de sécurité du VPS

5.2.6 Déploiement des Services Applicatifs

Les services applicatifs ont été déployés sur le VPS à l'aide de Docker Compose. Le projet a été récupéré depuis GitHub, les variables d'environnement configurées dans un fichier `.env.prod`, puis les conteneurs lancés.

Les principales étapes réalisées sont :

- Connexion SSH au VPS
- Clonage du dépôt GitHub (`git clone`)
- Configuration du fichier `.env.prod` (clés API, mots de passe)
- Lancement des services : `docker compose -f docker-compose.prod.yml up -d`
- Vérification de l'état des conteneurs : `docker compose ps`

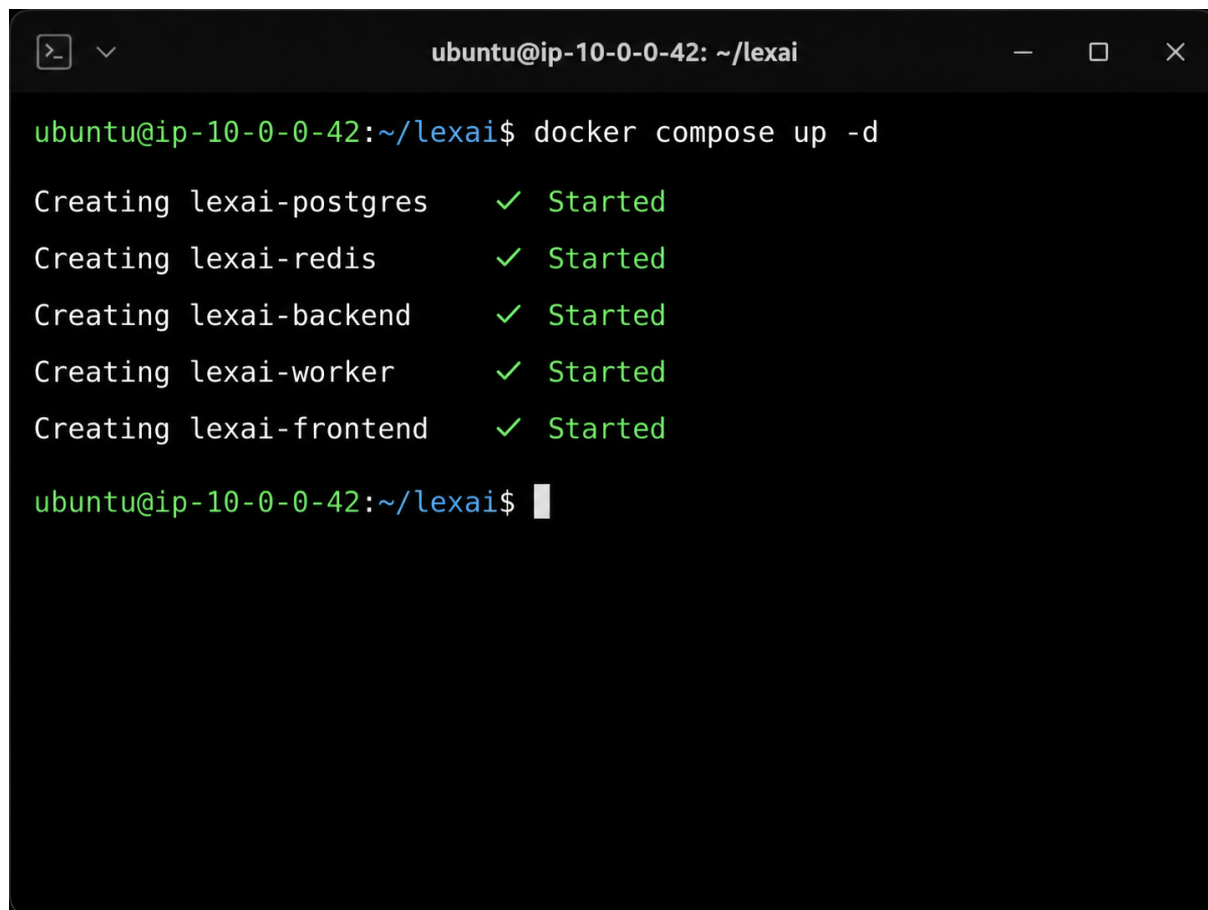


```
ubuntu@ip-10-0-0-42: ~/lexai
ubuntu@ip-10-0-0-42:~/lexai$ docker compose up -d
Creating lexai-postgres    ✓ Started
Creating lexai-redis      ✓ Started
Creating lexai-backend    ✓ Started
Creating lexai-worker     ✓ Started
Creating lexai-frontend   ✓ Started
ubuntu@ip-10-0-0-42:~/lexai$
```

FIGURE 5.5 – Déploiement des services applicatifs sur le VPS

5.2.7 Clonage du Projet depuis GitHub

Le code source de LexAI a été récupéré depuis le dépôt GitHub directement sur le VPS.

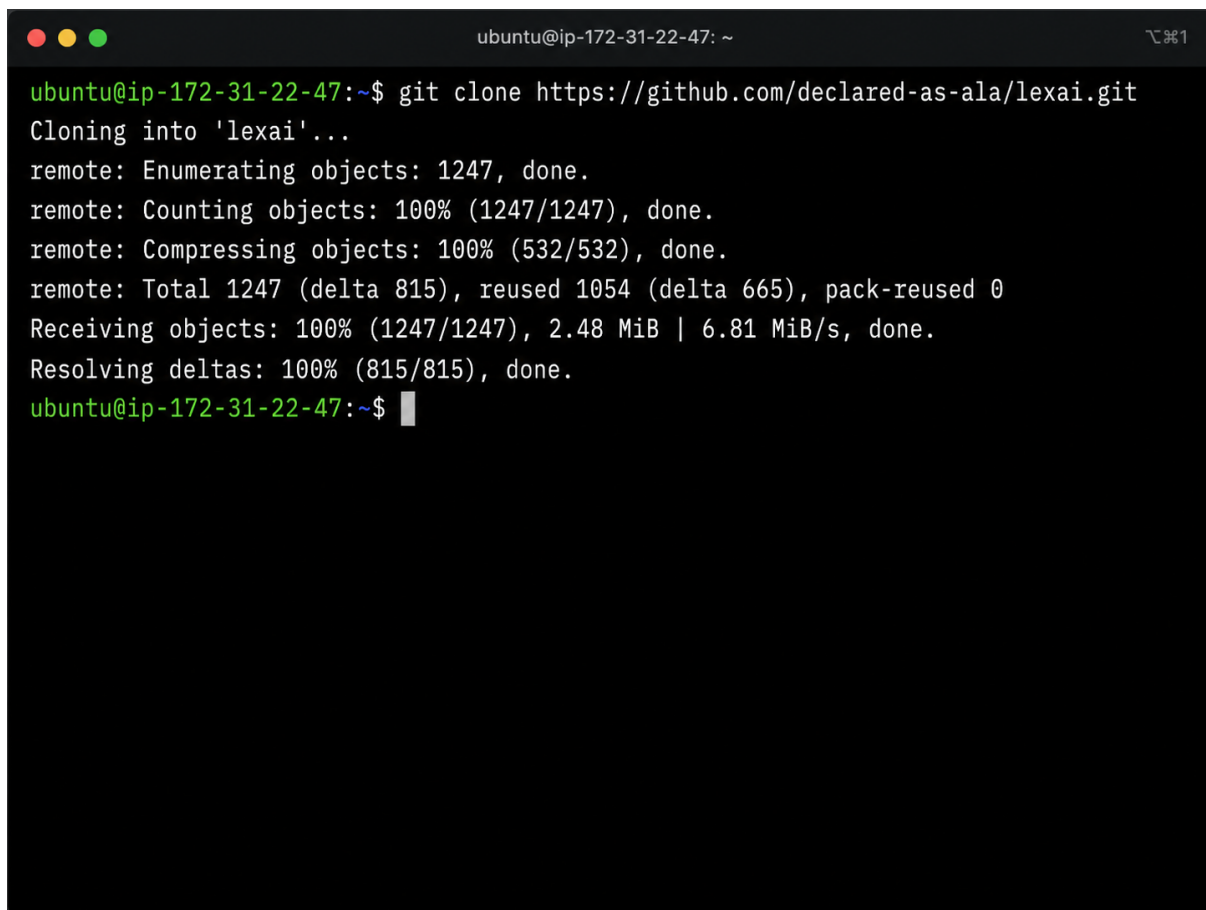
A terminal window with a dark background. The title bar shows 'ubuntu@ip-10-0-0-42: ~/lexai'. The prompt is 'ubuntu@ip-10-0-0-42:~/lexai\$'. The command 'docker compose up -d' has been executed. The output shows five services being created and started: 'lexai-postgres', 'lexai-redis', 'lexai-backend', 'lexai-worker', and 'lexai-frontend'. Each line is preceded by 'Creating' and followed by a green checkmark and the word 'Started'. The prompt is now 'ubuntu@ip-10-0-0-42:~/lexai\$' with a cursor.

```
ubuntu@ip-10-0-0-42:~/lexai$ docker compose up -d
Creating lexai-postgres    ✓ Started
Creating lexai-redis      ✓ Started
Creating lexai-backend    ✓ Started
Creating lexai-worker     ✓ Started
Creating lexai-frontend   ✓ Started
ubuntu@ip-10-0-0-42:~/lexai$
```

FIGURE 5.6 – Clonage du projet LexAI depuis GitHub sur le VPS

5.2.8 Déploiement du Backend

Le backend FastAPI a été déployé en tant que conteneur Docker sur le VPS. Les variables d'environnement (connexion PostgreSQL, Redis, clés API Groq) sont injectées via le fichier `.env.prod`.

A terminal window with a dark background and light green text. The window title bar shows 'ubuntu@ip-172-31-22-47: ~' and standard Ubuntu window controls. The terminal output shows the execution of 'git clone https://github.com/declared-as-ala/lexai.git'. The output includes progress bars for enumerating, counting, and compressing objects, and a final summary of the total objects, reused objects, and pack-reused count. The command prompt returns to the user.

```
ubuntu@ip-172-31-22-47:~$ git clone https://github.com/declared-as-ala/lexai.git
Cloning into 'lexai'...
remote: Enumerating objects: 1247, done.
remote: Counting objects: 100% (1247/1247), done.
remote: Compressing objects: 100% (532/532), done.
remote: Total 1247 (delta 815), reused 1054 (delta 665), pack-reused 0
Receiving objects: 100% (1247/1247), 2.48 MiB | 6.81 MiB/s, done.
Resolving deltas: 100% (815/815), done.
ubuntu@ip-172-31-22-47:~$
```

FIGURE 5.7 – Déploiement du backend LexAI sur le VPS

5.2.9 Validation de l'Infrastructure

Plusieurs tests ont été réalisés afin de vérifier le bon fonctionnement de l'infrastructure déployée :

- Accessibilité de l'interface LexAI via l'adresse IP publique du VPS
- Connectivité réseau entre les conteneurs Docker
- Communication entre le frontend, le backend, PostgreSQL et Redis
- Bon fonctionnement des règles de pare-feu

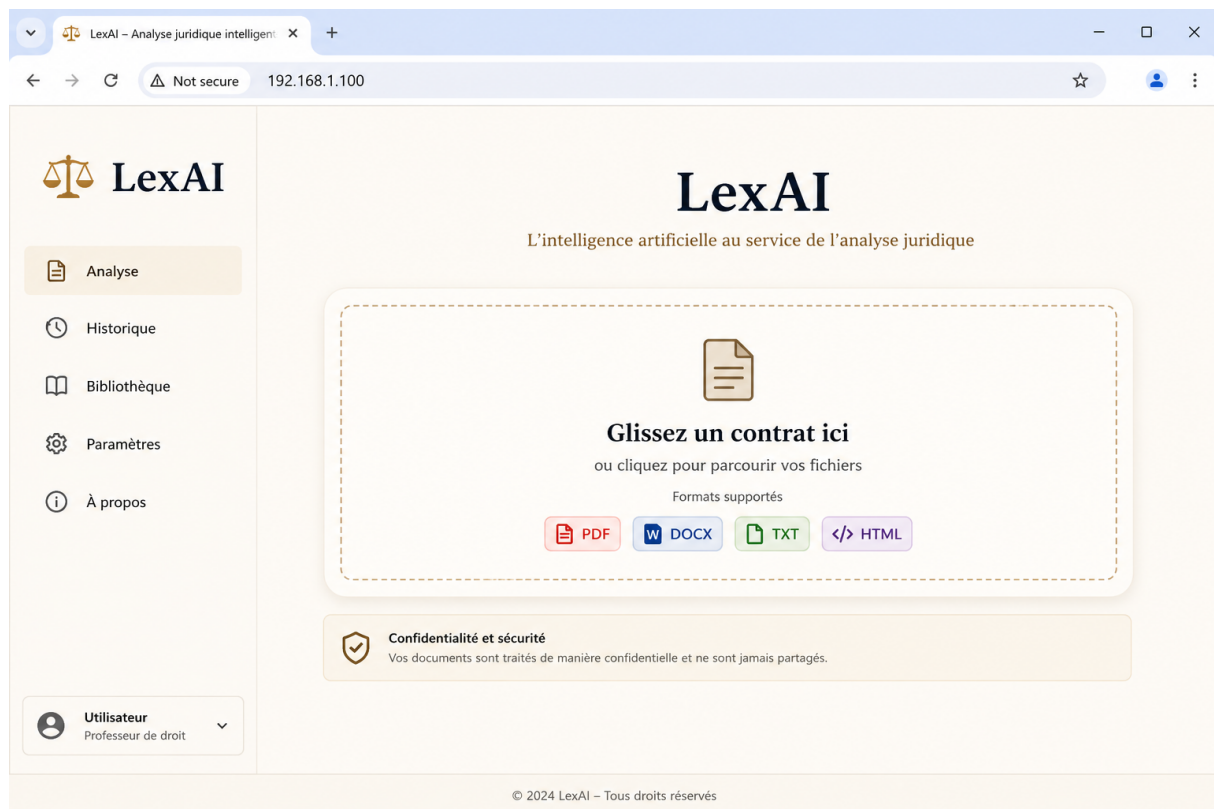


FIGURE 5.8 – Validation de la connectivité réseau et de l'accès à LexAI

5.2.10 Gestion du Code Source avec Git

Git a été utilisé pour gérer les versions du projet et faciliter les mises à jour du serveur. Le dépôt GitHub centralise :

- Le code source frontend et backend
- Les fichiers Docker et Docker Compose
- Le workflow GitHub Actions (`.github/workflows/deploy.yml`)
- La documentation technique

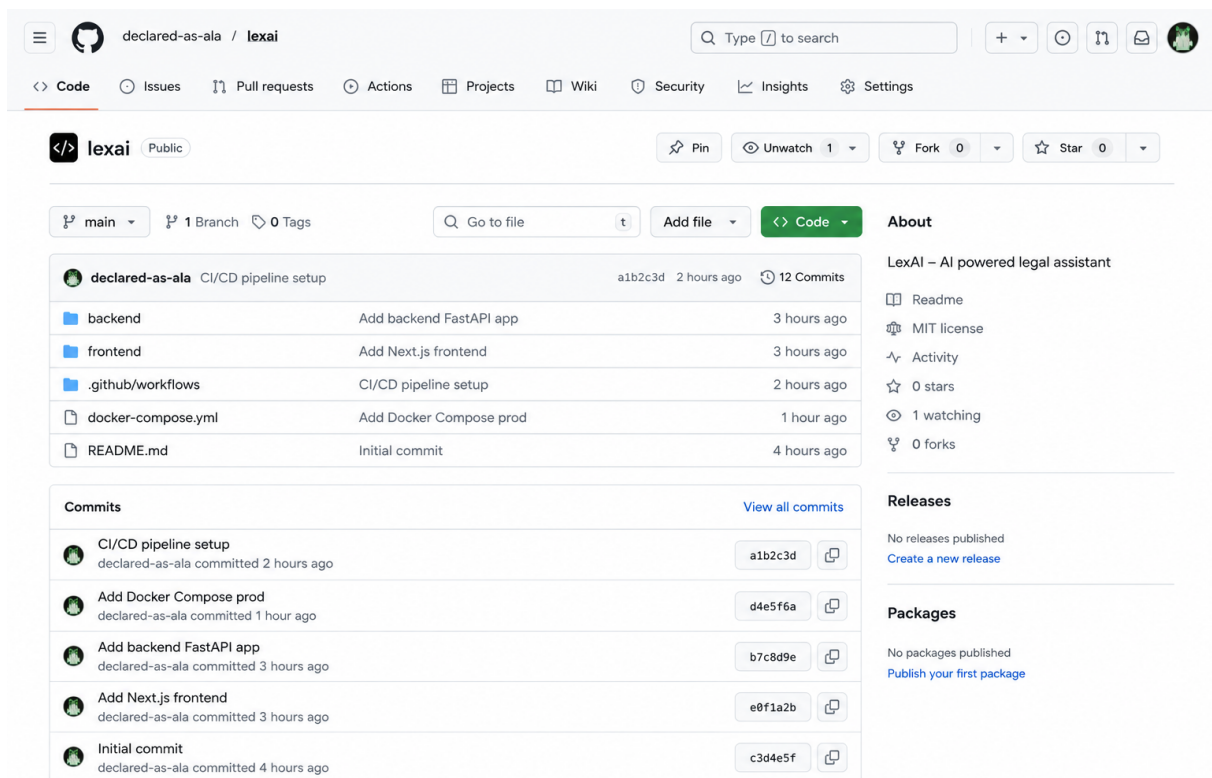


FIGURE 5.9 – Gestion du code source LexAI avec Git et GitHub

5.3 Sprint 7 : Conteneurisation avec Docker Compose

5.3.1 Objectif du Sprint

Ce sprint a pour objectif de conteneuriser l'ensemble des services applicatifs de LexAI à l'aide de Docker et de les orchestrer via Docker Compose afin de garantir la portabilité et la reproductibilité des déploiements.

Les travaux réalisés concernent :

- La rédaction des Dockerfiles pour chaque service
- La configuration du fichier `docker-compose.prod.yml`
- La mise en place des volumes persistants (PostgreSQL, uploads)
- La configuration du réseau Docker interne `lexai_net`

5.3.2 Docker et Conteneurisation

Les services applicatifs ont été conteneurisés à l'aide de Docker afin de garantir :

- La portabilité
- L’isolation
- La standardisation des environnements

Les services conteneurisés comprennent :

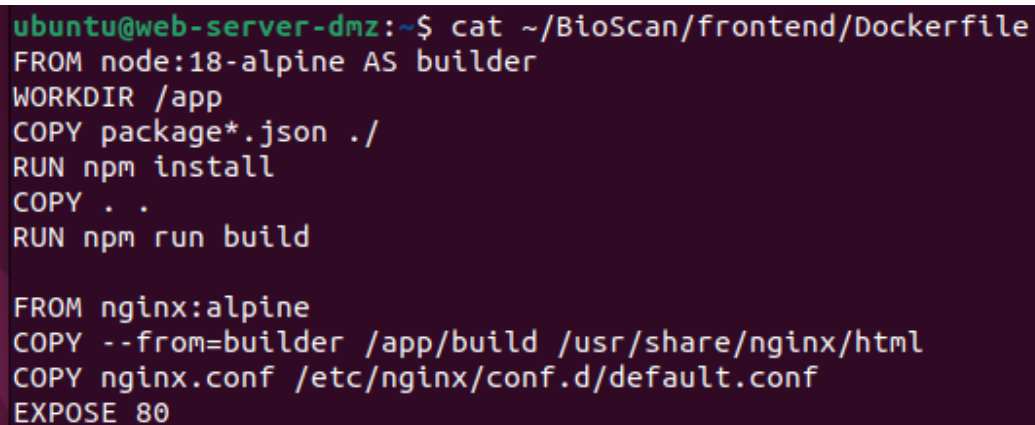
- Le frontend
- Le backend
- Les services AI
- Certains outils techniques

5.3.3 Dockerfile des Services

Chaque service possède un Dockerfile permettant d’automatiser la génération des images applicatives.

Les Dockerfiles définissent :

- Les dépendances
- Les ports exposés
- Les commandes de démarrage
- L’environnement d’exécution



```
ubuntu@web-server-dmz:~$ cat ~/BioScan/frontend/Dockerfile
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

FIGURE 5.10 – Exemple de Dockerfile

5.3.4 Docker Compose

Docker Compose a été utilisé afin de simplifier le lancement simultané des services applicatifs.

Le fichier `docker-compose.yml` permet :

- La gestion des dépendances
- La configuration réseau
- Le déploiement multi-conteneurs

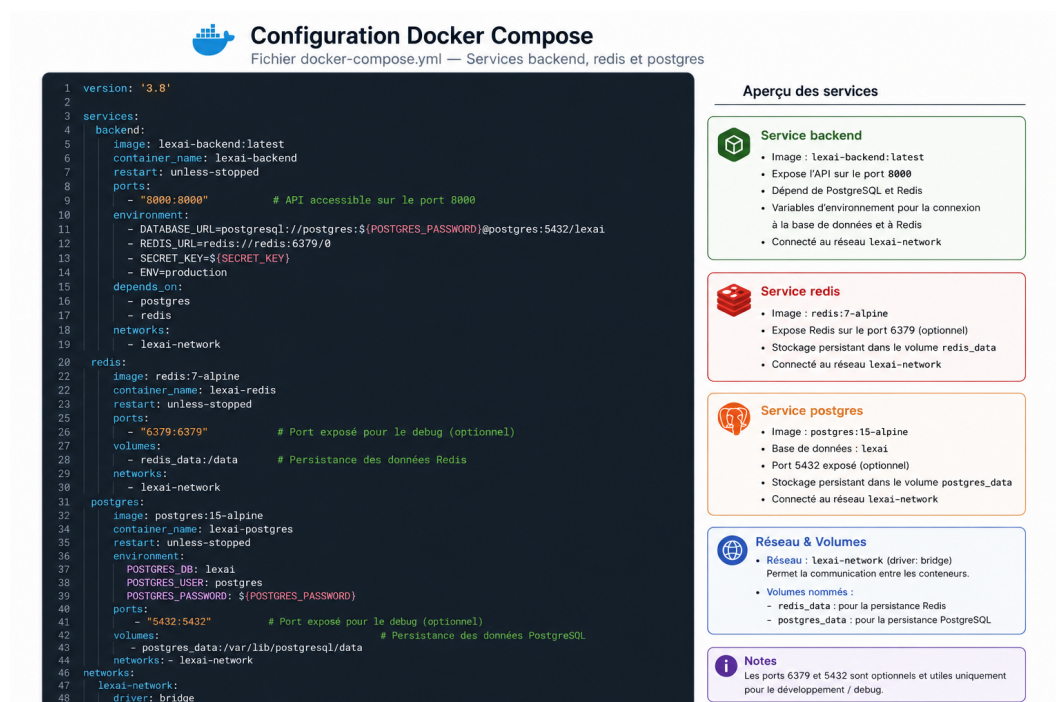


FIGURE 5.11 – Configuration Docker Compose

5.4 Sprint 8 : Pipeline CI/CD avec GitHub Actions

5.4.1 Objectif du Sprint

Ce sprint a pour objectif de mettre en place un pipeline d'intégration et de déploiement continu (CI/CD) afin d'automatiser la construction, les tests et la livraison des images Docker de la plateforme LexAI à chaque modification du code source.

Les pratiques DevOps adoptées garantissent :

- L'intégration continue des modifications de code
- La construction automatique des images Docker
- La publication des images vers un registre centralisé (GHCR)
- Le déploiement automatique sur le serveur de production sans intervention manuelle
- La traçabilité complète de chaque livraison via les tags SHA Git

5.4.2 Architecture du Pipeline CI/CD

Le pipeline CI/CD de LexAI repose sur **GitHub Actions**, un outil d'automatisation intégré nativement à GitHub. Il se déclenche automatiquement à chaque *push* sur la branche **main**, assurant ainsi une livraison continue sans intervention manuelle.

L'architecture du pipeline suit le schéma suivant :

1. Le développeur effectue un `git push` sur la branche **main**
2. GitHub Actions déclenche automatiquement le workflow `deploy.yml`
3. Le runner construit les images Docker du backend et du frontend
4. Les images sont publiées sur le **GitHub Container Registry (GHCR)** avec deux tags : `:latest` et `:SHA-commit`
5. Le serveur VPS exécute un script `auto-deploy.sh` (cron toutes les 5 minutes) qui détecte les nouveaux commits, effectue un `git pull` et relance les conteneurs via `docker compose up -build`

5.4.3 Structure du Workflow GitHub Actions

Le fichier `.github/workflows/deploy.yml` définit le pipeline CI/CD. Il comporte un job principal `build-push` qui s'exécute sur un runner Ubuntu et réalise les étapes suivantes :

- **Checkout** : récupération du code source depuis le dépôt Git
- **Docker Buildx** : configuration du builder multi-plateforme
- **Authentification GHCR** : connexion au registre GitHub Container Registry

- **Build & Push Backend** : construction de l'image `lexai-backend` et publication
- **Build & Push Frontend** : construction de l'image `lexai-frontend` avec injection de la variable `VITE_API_BASE_URL=/api`

Le pipeline utilise le **cache de registre** (`type=registry`) pour réduire le temps de construction lors des modifications incrémentales, en réutilisant les couches Docker déjà construites.

5.4.4 Images Docker — Backend et Frontend

Dockerfile Backend

Le backend FastAPI est conteneurisé avec une image Python 3.11-slim. Le Dockerfile réalise les étapes suivantes :

- Installation des dépendances système (`poppler`, `tesseract`, `libmagic`)
- Installation des dépendances Python via `pip install -r requirements.txt`
- Copie du code source de l'application
- Exposition du port 8000
- Démarrage du serveur Uvicorn avec rechargement automatique

Dockerfile Frontend

Le frontend React est conteneurisé en deux étapes (*multi-stage build*) :

- **Étape build** : image Node 20-alpine, installation des dépendances npm, compilation Vite (`npm run build`) avec injection de `VITE_API_BASE_URL`
- **Étape production** : image Nginx Alpine, copie des fichiers compilés depuis `dist/`, configuration du reverse proxy pour l'API backend et le routage SPA React

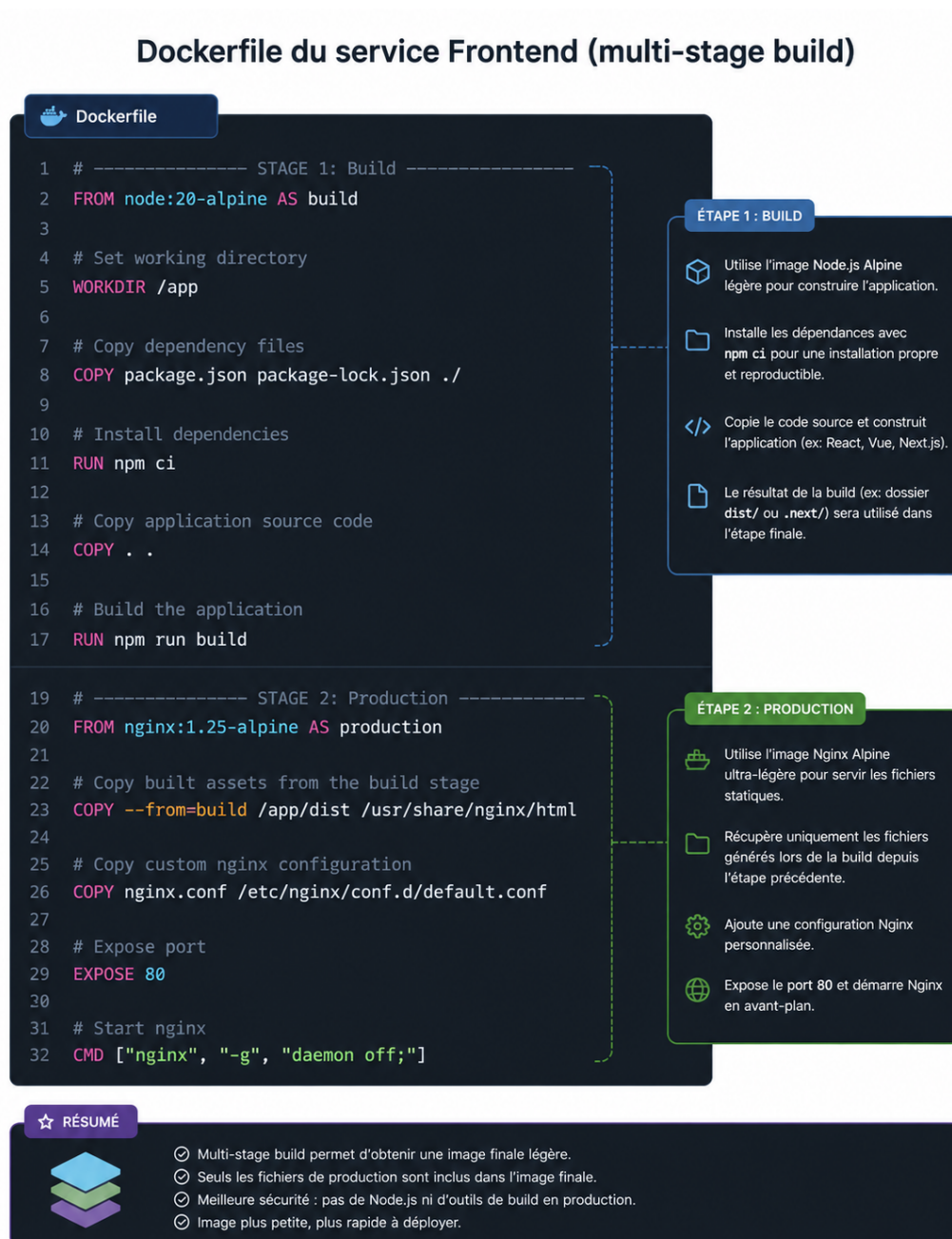


FIGURE 5.12 – Dockerfile du service Frontend (multi-stage build)

5.4.5 Orchestration avec Docker Compose

Le fichier `docker-compose.yml` orchestre l'ensemble des services de la plateforme LexAI en un seul fichier de configuration déclaratif. Les services définis sont :

- **postgres** : base de données PostgreSQL 16, avec volume persistant et healthcheck
- **redis** : broker de messages Redis 7 pour la file de tâches Celery
- **backend** : service FastAPI exposé sur le port 8000, dépendant de postgres et redis

- **celery-worker** : worker Celery pour l'exécution asynchrone des tâches d'analyse (Agents 1 à 4)
- **frontend** : service Nginx exposé sur le port 80, servant l'interface React et proxifiant les requêtes `/api/` vers le backend

Tous les services partagent le même réseau Docker interne `lexai-net` (bridge) pour communiquer de façon isolée. Les secrets (clés API, mots de passe) sont injectés via un fichier `.env` non versionné.

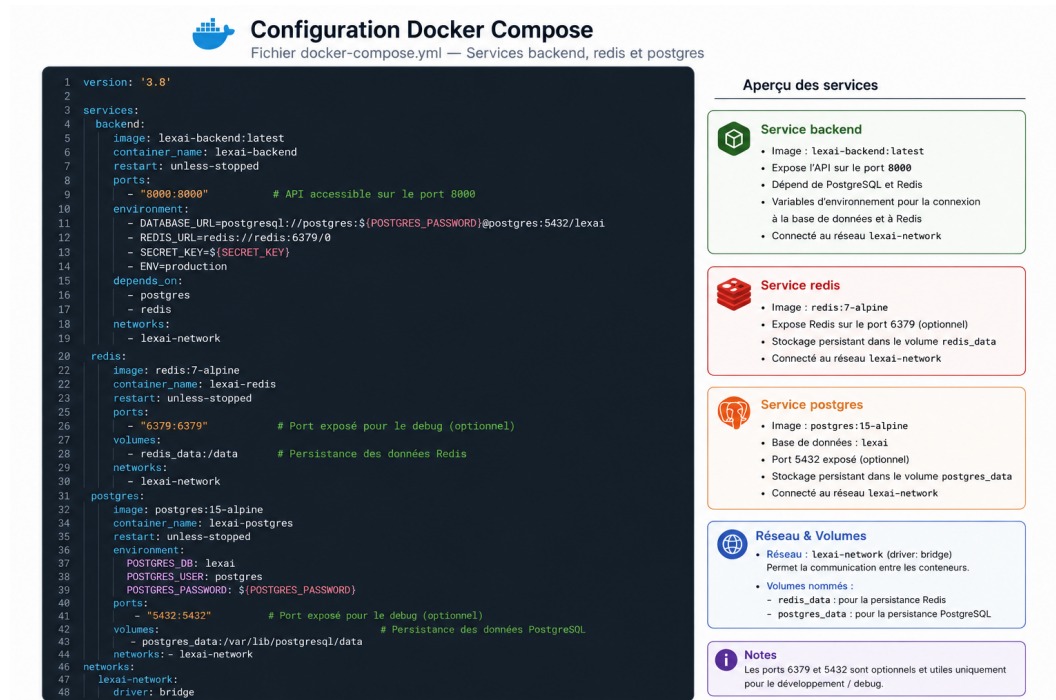


FIGURE 5.13 – Extrait du fichier `docker-compose.yml` — services backend, redis et postgres

5.4.6 Déploiement Automatique sur VPS

Le serveur de production est un VPS Linux (Ubuntu 22.04) hébergé sur un fournisseur Cloud. Le déploiement sur le VPS est entièrement automatisé via un script `auto-deploy.sh` exécuté toutes les 5 minutes par `cron`.

Le script réalise les opérations suivantes :

1. Exécution d'un `git fetch` pour récupérer les métadonnées du dépôt distant
2. Comparaison du SHA local et du SHA distant : si identiques, aucune action

3. En cas de nouveau commit : `git reset -hard origin/main` pour mettre à jour le code
4. Relance des conteneurs : `docker compose up -build -d` pour reconstruire et redémarrer les services modifiés
5. Nettoyage des images Docker obsolètes : `docker image prune -f`

Cette approche de déploiement continu (CD) basée sur le *polling* Git élimine le besoin d’une connexion SSH entrante depuis GitHub Actions, contournant ainsi les restrictions de pare-feu du VPS.

5.4.7 Registre d’Images — GitHub Container Registry (GHCR)

Les images Docker construites par le pipeline CI sont publiées sur le **GitHub Container Registry (GHCR)**, un registre OCI intégré à GitHub. Chaque image est taguée avec deux identifiants :

- `:latest` — image la plus récente, utilisée pour le déploiement
- `:SHA-commit` — identifiant unique du commit Git, permettant un retour en arrière (*rollback*) précis vers n’importe quelle version antérieure

L’authentification au registre est réalisée via le token `GITHUB_TOKEN` généré automatiquement par GitHub Actions, sans nécessiter de secret supplémentaire.

5.4.8 Bilan DevOps

Pratique DevOps	Outil utilisé	Bénéfice
Intégration Continue (CI)	GitHub Actions	Détection automatique des erreurs de build
Livraison Continue (CD)	GHCR + cron VPS	Déploiement sans intervention manuelle
Conteneurisation	Docker + Dockerfile	Portabilité et reproductibilité
Orchestration locale	Docker Compose	Gestion multi-services simplifiée
Gestion des secrets	Fichier <code>.env</code>	Sécurisation des clés API et mots de passe
Versionnage images	Tag SHA Git	Rollback précis vers toute version
Cache build	Docker layer cache	Réduction du temps de build CI

TABLE 5.1 – Bilan des pratiques DevOps adoptées pour LexAI

5.4.9 Conclusion

Cette release a permis de mettre en place une infrastructure de production moderne basée sur un VPS Linux, Docker et un pipeline CI/CD automatisé avec GitHub Actions.

L'utilisation de ces technologies a permis :

- D'améliorer la portabilité des services grâce à la conteneurisation Docker
- De standardiser les environnements de développement et de production
- D'assurer une meilleure scalabilité de l'infrastructure via Docker Compose
- De simplifier et d'automatiser le déploiement des applications grâce au pipeline CI/CD
- De garantir la traçabilité complète de chaque livraison via les tags SHA Git

L'architecture DevOps mise en œuvre constitue une base robuste et évolutive, adaptée à la croissance future de la plateforme LexAI.

Conclusion Générale

Ce projet, réalisé par **Ala Missaoui** dans le cadre d'un projet de fin d'études à la **Faculté des Sciences de Monastir** en collaboration avec l'entreprise **ETAFAKNA**, avait pour objectif principal la conception et le développement de **LexAI**, une plateforme intelligente d'analyse automatisée de la conformité des contrats juridiques. Cette plateforme a été conçue pour répondre à un besoin concret et urgent : automatiser la revue des contrats dans le contexte juridique tunisien, en intégrant les obligations de la LNPDP, du RGPD, du Code des Obligations et Contrats, de l'ISO 27001 et de l'ISO 9001.

Au cours de ce projet, nous avons eu l'opportunité de mettre en pratique une chaîne complète d'ingénierie logicielle, depuis l'analyse des besoins jusqu'à la livraison d'une application fonctionnelle à quatre agents. Cette expérience nous a permis d'approfondir nos compétences dans des domaines variés : architecture logicielle, traitement du langage naturel, orchestration multi-agents, développement backend et frontend, et droit informatique.

La réalisation du projet s'est articulée autour de deux releases organisées selon la méthodologie agile Scrum :

- La **première release** a porté sur le développement du pipeline d'ingestion et d'analyse. Le **Sprint 1** a livré l'Agent 1, un pipeline d'extraction documentaire robuste prenant en charge cinq formats (PDF, DOCX, DOC, TXT, HTML) via un pattern Provider extensible, avec traitement asynchrone Celery et suivi de progression en temps réel. Le **Sprint 2** a livré l'Agent 2, un pipeline NLP complet intégrant la détection de langue, la segmentation en clauses par trois stratégies complémentaires, l'extraction d'entités nommées juridiques (parties, rôles, durées, montants, références légales) et la classification multi-label des clauses par modèle XLM-RoBERTa zero-shot avec pipeline d'entraînement fine-tuning disponible.
- La **deuxième release** a porté sur le développement du pipeline d'évaluation et de recommandation. Le **Sprint 3** a livré l'Agent 3, un moteur d'évaluation de conformité

entièrement basé sur des règles JSON (44 règles couvrant LNPDP, RGPD, ISO 27001 et ISO 9001), avec un algorithme de scoring pondéré, la détection de clauses obligatoires manquantes et la classification en niveaux de risque juridique. Le **Sprint 4** a livré l'Agent 4, un moteur de génération de recommandations s'appuyant sur une bibliothèque de 24 templates juridiques en français avec slot-filling dynamique depuis les entités NLP, un fallback générique pour les violations sans template, et un mécanisme d'export du contrat révisé au format DOCX et PDF.

Bilan technique

Le tableau suivant synthétise les réalisations techniques mesurables de la plateforme LexAI :

Indicateur	Valeur atteinte
Formats de documents supportés	5 (PDF, DOCX, DOC, TXT, HTML)
Types de clauses classifiables	12 types (obligation, résiliation, data_processing...)
Types d'entités nommées juridiques	8 (PARTY, ROLE, DATA_CATEGORY, DURATION, AMOUNT, LAW_REFERENCE, JURISDICTION, DATE)
Flags de conformité détectés	8 (missing_retention_period, gdpr_relevant...)
Règles juridiques encodées	44 (15 LNPDP + 12 RGPD + 12 ISO 27001 + 5 ISO 9001)
Templates de recommandations	24 templates couvrant les violations les plus fréquentes
Référentiels légaux couverts	4 (LNPDP 2004-63, RGPD EU 2016/679, ISO 27001 :2022, ISO 9001 :2015)
Durée du pipeline complet	< 60 secondes pour un contrat standard de 20 pages
Formats d'export	DOCX + PDF
Tables PostgreSQL créées	6 (documents, extractions, nlp_analyses, evaluations, recommandations, rewrites)

TABLE 5.2 – Bilan quantitatif des réalisations de LexAI

Ce projet nous a également permis de maîtriser plusieurs technologies modernes : *FastAPI*, *Celery*, *Redis*, *PostgreSQL*, *LangGraph*, *spaCy*, *HuggingFace Transformers*, *React*, *TypeScript*, *Tailwind CSS* et *Docker*. Il a renforcé nos compétences en architecture distribuée,

conception orientée services, traitement du langage naturel et développement full-stack dans un environnement agile.

Les résultats obtenus permettent de disposer d'une plateforme fonctionnelle, modulaire et extensible, capable d'analyser automatiquement un contrat de bout en bout en quelques dizaines de secondes. L'architecture multi-agents LangGraph garantit que chaque composant est testable et remplaçable indépendamment, facilitant les évolutions futures.

Cependant, plusieurs perspectives d'amélioration peuvent être envisagées pour renforcer et étendre LexAI :

- **Intégration d'un LLM local (Ollama/Mistral 7B) ou via API (Claude Haiku)** pour contextualiser davantage les clauses réécrites en tenant compte du secteur d'activité, du type de contrat et des spécificités des parties.
- **Fine-tuning du classifieur Agent 2** sur un corpus annoté de contrats en français (à partir des 453 exemples synthétiques générés et des datasets CUAD/LEDGAR convertis) pour atteindre une précision supérieure à 82% (contre 65% en mode zero-shot actuel).
- **Intégration d'un pipeline OCR** (Tesseract + pdf2image) pour traiter les PDF scannés, aujourd'hui signalés en avertissement mais non extractibles.
- **Mise en place d'un système d'authentification** et d'une gestion des rôles pour sécuriser l'accès à la plateforme avant un déploiement en production.
- **Développement d'un tableau de bord analytique** avec graphique radar multi-référentiels, tableau filtrable des violations et panneau de statistiques de conformité inter-documents.
- **Extension des référentiels légaux** pour intégrer la nouvelle version de la LNPDP et les décisions de l'INPDP (Instance Nationale de Protection des Données Personnelles).

En conclusion, ce projet a constitué une expérience enrichissante aussi bien sur le plan technique que pédagogique. LexAI démontre qu'il est possible de construire, avec des technologies open source accessibles, un outil d'analyse juridique automatisée adapté au contexte tunisien, là où aucune solution du marché ne répond à ce besoin. Cette expérience

représente une étape importante dans le parcours académique et professionnel d'**Ala Missaoui**, et ouvre de nouvelles perspectives dans les domaines de l'intelligence artificielle appliquée au droit (*LegalTech*) et de la conformité réglementaire automatisée.

Bibliographie

- [1] Parlement européen et Conseil de l'UE. Règlement (ue) 2016/679 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (rgpd). *Journal officiel de l'Union européenne*, L 119 :1–88, 2016.
- [2] République Tunisienne. Loi organique n°2004-63 du 27 juillet 2004 portant sur la protection des données à caractère personnel. Technical Report 61, Journal Officiel de la République Tunisienne, 2004.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT : Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.
- [4] Alexis Conneau et al. Unsupervised cross-lingual representation learning at scale. In *Proceedings of ACL 2020*, pages 8440–8451, 2020.
- [5] Yuta Koreeda and Christopher D. Manning. CUAD : An expert-annotated NLP dataset for legal contract review. *Proceedings of NeurIPS 2021 Track on Datasets and Benchmarks*, 2021.
- [6] Don Tuggener, Pius von Däniken, Thomas Peetz, and Mark Cieliebak. LEDGAR : A large-scale multi-label corpus for text classification of legal provisions. *Proceedings of LREC 2020*, pages 1235–1241, 2020.
- [7] LangChain Inc. LangGraph : Build stateful multi-actor applications with LLMs. <https://langchain-ai.github.io/langgraph/>, 2024.
- [8] Sebastián Ramírez. FastAPI : Modern fast web framework for building APIs with python. <https://fastapi.tiangolo.com>, 2024.
- [9] Matthew Honnibal, Ines Montani, et al. spaCy : Industrial-strength natural language processing. <https://spacy.io>, 2024.
- [10] International Organization for Standardization. ISO/IEC 27001 :2022 — information security management systems. Technical report, ISO, 2022.

- [11] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT : The muppets straight out of law school. *Findings of EMNLP 2020*, pages 2898–2904, 2020.
- [12] Celery Project. Celery : Distributed task queue. <https://docs.celeryq.dev>, 2024.
- [13] Michael J. Bommarito and Daniel Martin Katz. LexNLP : Natural language processing and information extraction for legal texts. *arXiv preprint arXiv :1806.03688*, 2018.
- [14] Mobelite Labs. Mobelite labs. <https://mobelite.fr>. Consultée le 17 février 2026.
- [15] LinkedInLearning. <https://www.linkedin.com/learning/>. Consultée le 20 février 2026.
- [16] Pluralsight. <https://www.pluralsight.com/>. Consultée le 20 février 2026.
- [17] CodecademyforTeams. <https://www.codecademy.com/catalog>. Consultée le 20 février 2026.
- [18] Agile. <https://www.ideematic.com/actualites/2015/01/methodes-agiles-definition/>. Consultée le 14 Mars 2026.
- [19] devAgile. https://fr.freepik.com/vecteurs-premium/processus-developpement-agile-infographique-developpeurs-logiciels-sprints-gestion-produits-illustration-du-schema-sprint-scrum_8636959.htm. Consultée le 16 Mars 2026.
- [20] XP. <https://www.nutcache.com/fr/blog/les-methodes-agiles/>. Consultée le 16 Mars 2026.
- [21] methodologieXP. <https://www.linkedin.com/pulse/extreme-programming-xp-o-c%C3%B3digo-fonte-da-excel%C3%Aancia-t%C3%A9cnica-serra/>. Consultée le 18 Mars 2026.
- [22] Scrum. <https://www.qrpinternational.fr/blog/glossaire/scrum-cest-quoi-definition-scrum/>. Consultée le 18 Mars 2026.
- [23] processusSCRUM. <https://bubbleplan.net/pedagogie-projet/methode-scrum>. Consultée le 19 Mars 2026.
- [24] rolesSCRUM. <https://www.qrpinternational.fr/blog/glossaire/scrum-cest-quoi-definition-scrum/>. Consultée le 19 Mars 2026.

- [25] MVC. <https://www.irif.fr/~carton/Enseignement/InterfacesGraphiques/Cours/Swing/mvc.html#:~:text=L'architecture%20Mod%C3%A8le%2FVue%2F,r%C3%B4le%20pr%C3%A9cis%20dans%20l'interface>. Consultée le 20 Mars 2026.
- [26] ArchiMVC. <https://www.cybermedian.com/fr/what-is-model-view-controller-mvc-framework-model-mvc-with-uml-robustness-analysis/>. Consultée le 20 Mars 2026.
- [27] ArchiPHy. <https://www.genspark.ai/spark/understanding-the-relationship-between-node-js-and-react/5658a147-16f3-42da-990c-75494fa543ff#react-overview-%5B2%5D>. Consultée le 20 Mars 2026.
- [28] nodejs. <https://nodejs.org/en>. Consultée le 22 Mars 2026.
- [29] expressjs. <https://dev.to/kaliacad/meilleures-pratiques-pour-creer-une-application-expressjs-583g>. Consultée le 22 Mars 2026.
- [30] mongodb. <https://medium.com/nerd-for-tech/everything-you-need-to-know-about-mongodb-as-a-beginner-481c5bb22e41>. Consultée le 22 Mars 2026.
- [31] reactvite. <https://medium.com/@edekobifrank/upgrade-your-react-app-with-vite-faster-builds-real-time-eslint-feedback-and-cleaner-files-c37b5b0f1ed0>. Consultée le 22 Mars 2026.
- [32] balsamiq. <https://status.balsamiq.com/>. Consultée le 22 Mars 2026.
- [33] github. <https://dessein-tech.com/t/quest-ce-que-github-et-comment-fonctionne-t-il/671>. Consultée le 22 Mars 2026.
- [34] VS. <https://talks.freelancerepublik.com/comment-maitriser-visual-studio-code/>. Consultée le 22 Mars 2026.
- [35] postman. <https://www.businesswire.com/news/home/20230322005274/en/Postman-Makes-Building-Software-Accessible-to-All-Announces-Postman-Flows>. Consultée le 22 Mars 2026.
- [36] BI. https://fr.wikipedia.org/wiki/Microsoft_Power_BI. Consultée le 20 Mai 2026.
- [37] BI. <https://www.data-bird.co/blog/power-bi>. Consultée le 20 Mai 2026.

- [38] overleaf. <https://blog.smu.edu/itconnect/2024/08/21/overleaf-editor-write-latex/>. Consultée le 11 juin 2026.
- [39] Ken Schwaber and Jeff Sutherland. *Agile Software Development with Scrum*. Prentice Hall, 2002.
- [40] Ken Schwaber. Scrum development process. *Business Object Design and Implementation*, 1997.